LESERT Aymeric

S.G.B.D.

SYSTEME DE GESTION DE BASES DE DONNEES

V 1.0



1994 - 1995

SYSTEME DE GESTION DE BASES DE DONNEES

SUJET

1. Objectif

Le produit, concernant la version 1.0, sera le noyau d'un système de gestion de bases de données (SGBD) de type relationnel. Il possédera uniquement les outils de gestions des fichiers de la base, ceci pourra permettre à un programmeur utilisant un compilateur C ou C++ d'exploiter en tout quiétude ce produit.

2. Pour Qui ? Pour Quoi faire ?

Elle sera conçue pour offrir des outils de programmation d'une base de données en C ou pour réaliser des programmes simples.

Exemple : Réalisation d'une base de données pour gérer une collection d'étiquettes de vin, ou pour réaliser une gestion de devis de matériel ou de personnel, ...

Elle sera adaptée aux bases de données moyenne contenant au maximum 32.000 enregistrements par table, pouvant manipuler 99 tables avec 99 clés et 99 champs. Ceci constitue les limites de ce S.G.B.D. donc il est adapté aux petites bases. De plus, il est facile de concevoir des filtres entre les différentes bases de données déjà existantes.

3. Les contraintes matérielles

Son manque de module en assembleur permet une portabilité de la base sur de nombreux systémes d'exploitation tel que MS-DOS, Linux , Unix, OS/2 et windows.
Cette portabilité pourra s'effectuer en positionnant les INCLUDEs appropriés.

Ce S.G.B.D. ne génera pas trop vos applications puisqu'il ne réclame pas beaucoup de mémoires (au maximum 30Ko), toutes les données seront conservées sur disque dur qui sera utilisé comme mémoire auxiliaire.

4. Exemple pour accélerer le traitement du S.G.B.D.

Il est possible de créer un disque virtuel (RAMDRIVE : MS-DOS), de charger la base de données dans ce disque, puis de travail avec. En fin de traitement, vous pouvez recopier toutes les données du disque virtuel sur le disque réel.

SYSTEME DE GESTION DE BASES DE DONNEES

OBJECTIFS DE CE PROJET

Ce projet a pour but la constitution d' outils permettant de gérer un S.G.B.D de type relationnel dans un programme C ou C++.

Remarque :Nous allons juste signaler qu' un projet est une application constituée d' une base de données et de ces outils.

Dans **un premier temps**, nous allons concevoir la gestion d' une application, c' est à dire :

- Création d' une application et de son arborescence
- Ouverture d' une application dans le cas où elle existe déjà
- Sauvegarde des modifications de l' application
  (Sauvegarde du dictionnaire de données)
- Destruction d' une application.

Dans **un deuxième temps**, nous allons mettre au point des outils permettant une modification du schéma de la base (modification des paramètres d' une table, d' un champ ou d' une clé). Dans cette partie, il ne sera pas question d' optimisation car le soucis principal est de ne pas perdre de données (i.e. offrir une garantie de sécurité).

* Les opérations concernées seront :
  - La création d' une relation (ou table)
  - La suppression d' une relation (ou table)
  - L' ajout d' une clé ou d' un champ
    (car pour une relation il existe toujours un identifiant et
    pas toujours des propriétés)
  - La suppression d' une clé ou d' un champ
  - La modification du contenu des informations que peuvent
    contenir une clé ou un champ (paramètres).

Remarque : Il faudra faire de son mieux pour éviter la perte d' informations en réalisant des tests judicieux et en sauvegardant les données dans des tampons, ainsi modifiées pour ne pas les perdre bêtement.

* En parallèle, nous allons développer des outils permettant la mise à jour et la consultation d' enregistrements dans une base définie par les outils précédemment développés. Ceci permet un meilleur contrôle du développement de l'application.

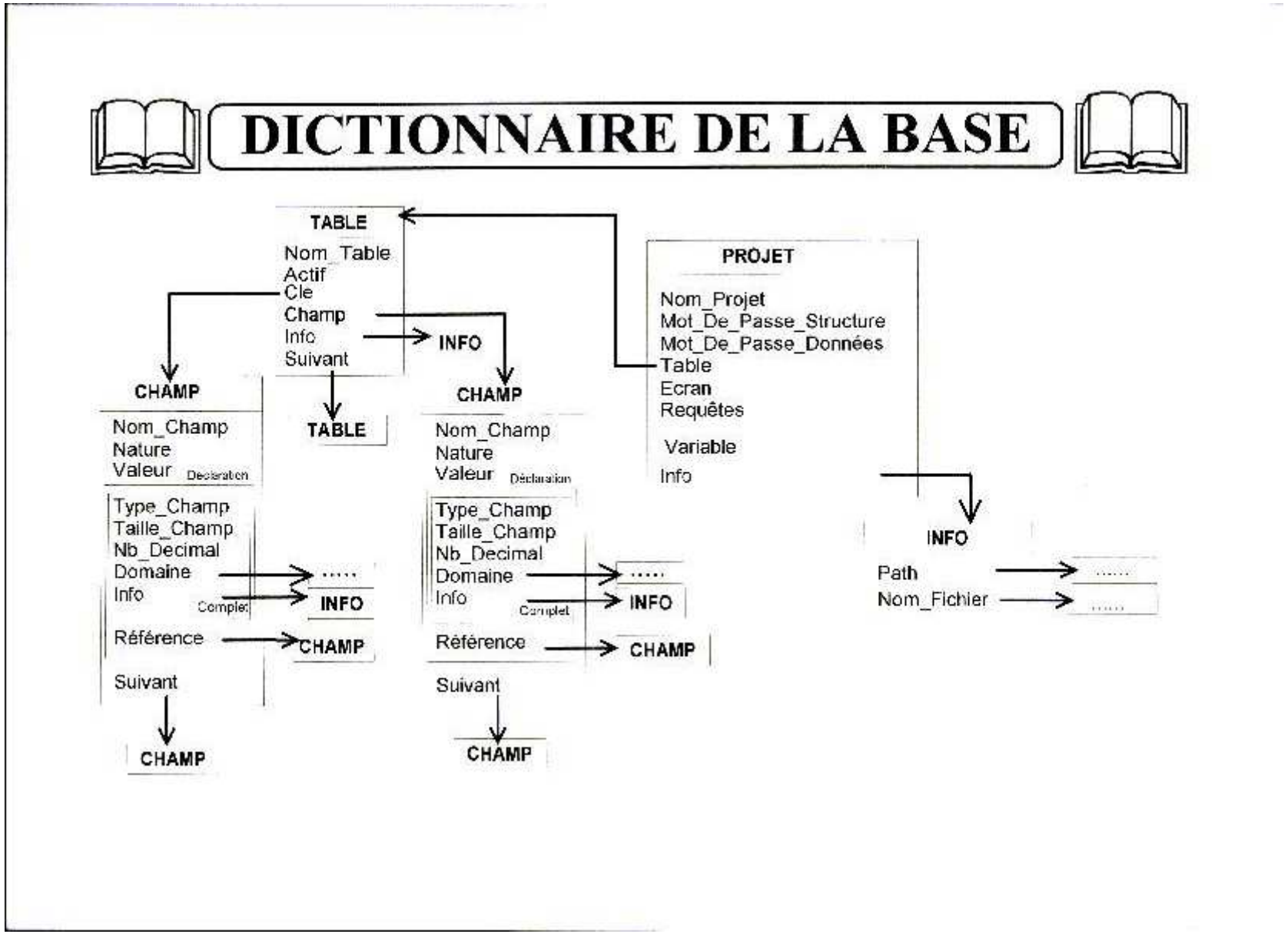
Dans **un troisième temps**, nous allons concevoir un outil permettant la consultation du schéma des relations ou plutôt des données que peut contenir le schéma des relations. Cet outil sera un SQL : langage de requêtes.

* Dans une première phase de développement, le SQL s' effectuera sur un niveau. Le but recherché sera un accès rapide à une question précise.

* Dans une deuxième phase, nous envisagerons la possibilité d' un SQL à plusieurs niveaux à travers divers algorithmes proposés.

Dans **un quatrième temps**, il s'agira de concevoir une application manipulant cette librairie afin de montrer comment l'utiliser, quels sont ces avantages et ces inconvénients et enfin, mettre en avant toutes les qualités qui en découlent.

A chaque niveau d' étude, correspondra un niveau d' approfondissement et, pour cela, il faudra peu à peu compléter le dictionnaire des données avec les nouveaux paramètres. Ces nouveaux paramètres devront être clairement définis afin de sauvegarder, en intégralité, l' ensemble d' une application sans dommage de cette dernière et ainsi offrir une garantie de sécurité pour tout usager d' une telle application.


DICTIONNAIRE DE LA BASE
TABLE
Nom_Table
Actif
Clé
Champ
Info
Suivant
INFO
TABLE
CHAMP
Nom_Champ
Nature
Valeur
Declaration
Type_Champ
Taille_Champ
Nb_Decimal
Domaine
Info
Complet
.....
INFO
Référence
CHAMP
Suivant
CHAMP
CHAMP
Nom_Champ
Nature
Valeur
Déclaration
Type_Champ
Taille_Champ
Nb_Decimal
Domaine
Info
Complet
.....
INFO
Référence
CHAMP
Suivant
CHAMP
PROJET
Nom_Projet
Mot_De_Passe_Structure
Mot_De_Passe_Données
Table
Ecran
Requêtes
Variable
Info
INFO
Path
Nom_Fichier
.....
.....

```
/* ************************************************************************ */
/*                Systéme de Gestion de Bases de Données                    */
/*                              Version 1.0                                 */
/*                                                                          */
/* Date : 01/06/95                                                          */
/*                                                                          */
/*                                                        par LESERT Aymeric */
/* ************************************************************************ */

/* ************************************************************************ */
/*                                  UNITE                                   */
/*                déclaration des constantes exploitables                   */
/* ************************************************************************ */

/* macro définissant un intervalle fermée */

#define between(x,a,y)  ((x<=a)&&(a<=y))

/* constante de l'entête d'un enregistrement */

#define SGBD_Caractere_Actif   ' '
#define SGBD_Caractere_Inactif 'X'

/* constante de Taille */

#define SGBD_Taille_Nom            32
#define SGBD_Nb_Champ_Ecran_Maxi   127
#define SGBD_Taille_Mot_Passe       5

/* definition des domaines de definitions */

#define SGBD_Domaine_Path_Projet   "[A-Za-z0-9\\\\_:]*"
#define SGBD_Domaine_Nom_Fichier   "[A-Za-z0-9]+"
#define SGBD_Domaine_Nom_Projet    "+"
#define SGBD_Domaine_Mot_De_Passe  "+"
#define SGBD_Domaine_Nom_Table     "[A-Za-z0-9_]*"
#define SGBD_Domaine_Nom_Champ     "[A-Za-z0-9_]*"
#define SGBD_Domaine_SGBD_Domaine_Champ "*"
#define SGBD_Domaine_Caractere     "*"
#define SGBD_Domaine_Numerique     "[\\+\\-0-9]*"
#define SGBD_Domaine_Reel          "[\\+\\-0-9,]*"
#define SGBD_Domaine_Date          "[0-9]{4}[01][0-9][0-3][0-9]"
#define SGBD_Domaine_Memo          "*"
#define SGBD_Domaine_Nom_Menu      "[A-Za-z0-9_ ]*"
#define SGBD_Domaine_Nom_SQL       "[A-Za-z0-9_ ]*"
#define SGBD_Domaine_Nom_Ecran     "[A-Za-z0-9_ ]*"
#define SGBD_Domaine_Nom_Variable  "[A-Za-z0-9_]*"
#define SGBD_Domaine_Nom_Bouton    "[A-Za-z0-9_]*"

/* constante booleenne */

#define Vrai 1
#define Faux 0
```

```
/* constante de la nature d'une information */

#define SGBD_Cle_Croissante       'C'
#define SGBD_Cle_Decroissante     'D'
#define SGBD_Cle_Aleatoire        'A'
#define SGBD_Cle_Croissante_Ref   'c'
#define SGBD_Cle_Decroissante_Ref 'd'
#define SGBD_Cle_Aleatoire_Ref    'a'
#define SGBD_Champ_Normal         'N'
#define SGBD_Champ_Reference      'R'
#define SGBD_Champ_Variable       'V'

/* constante de typage d'un champ */

#define SGBD_Numerique 'N'
#define SGBD_Caractere 'C'
#define SGBD_Reel      'R'
#define SGBD_Date      ‘D’
#define SGBD_Memo     ‘M'
```

```
/* ************************************************************************ */
/*                Systéme de Gestion de Bases de Données                    */
/*                              Version 1.0                                 */
/*                                                                          */
/* Date : 01/06/95                                                          */
/*                                                                          */
/*                                                        par LESERT Aymeric */
/* ************************************************************************ */

/* ************************************************************************ */
/*                                UNITE                                     */
/*                           déclaration des types                          */
/* ************************************************************************ */

/* tout ce qui est décrit ici est utilisé dans les différents modules */

#include <stdio.h>
#include <string.h>
#include <alloc.h>

#define between(x,a,y)  ((x<=a)&&(a<=y))

 /* ************************************************************************ */
 /*                        définition des constantes                         */
 /* ************************************************************************ */

/* constante du SGF */

#define Caractere_Actif   ' '
#define Caractere_Inactif 'X'

/* constante de la Taille_Nom */

#define Taille_Nom            32
#define Taille_Mot_Passe       5

/* definition des domaines de definitions */

#define Domaine_Path_Projet   "[A-Za-z0-9\\\\_:]*"
#define Domaine_Nom_Fichier   "[A-Za-z0-9]+"
#define Domaine_Nom_Projet    "+"
#define Domaine_Mot_De_Passe  "+"
#define Domaine_Nom_Table     "[A-Za-z0-9_]*"
#define Domaine_Nom_Champ     "[A-Za-z0-9_]*"
#define Domaine_Domaine_Champ "*"
#define Domaine_Caractere     "*"
#define Domaine_Numerique    "[\\+\\-0-9]*"
#define Domaine_Reel         "[\\+\\-0-9,]*"
#define Domaine_Date          "[0-9]{4}[01][0-9][0-3][0-9]"
#define Domaine_Memo          "*"
#define Domaine_Nom_Menu      "[A-Za-z0-9_ ]*"
#define Domaine_Nom_SQL       "[A-Za-z0-9_ ]*"
#define Domaine_Nom_Ecran     "[A-Za-z0-9_ ]*"
#define Domaine_Nom_Variable  "[A-Za-z0-9_]*"
#define Domaine_Nom_Bouton    "[A-Za-z0-9_]*"
```

```
/* constante booleenne */

#define Vrai 1
#define Faux 0

/* constante de la nature d'une information */

#define Cle_Croissante       'C'
#define Cle_Decroissante     'D'
#define Cle_Aleatoire        'A'
#define Cle_Croissante_Ref   'c'
#define Cle_Decroissante_Ref 'd'
#define Cle_Aleatoire_Ref    'a'
#define Champ_Normal         'N'
#define Champ_Reference      'R'
#define Champ_Variable       'V'

/* constante de typage d'un champ */

#define Numerique 'N'
#define Caractere 'C'
#define Reel      'R'
#define Date  'D'
#define Memo  'M'

/* constante de nature d'un noeud d'un arbre */

#define Nature_Conjonction   'C'
#define Nature_Signe         'S'
#define Nature_Condition_Si  'D'

#define Conjonction_Et       'E'
#define Conjonction_Ou       'O'
#define Conjonction_Non      'N'

#define Tautologie           'T'
#define Antilogie            'A'
#define Signe_Egal           '='
#define Signe_Non_Egal       '!'
#define Signe_Superieur      '>'
#define Signe_Inferieur      '<'
#define Signe_Superieur_Egal '_'
#define Signe_Inferieur_Egal '_'
```

```
 /* ************************************************************************ */
 /*                            définition des types                          */
 /* ************************************************************************ */

/* ----------------------- */
/* définition du type info */
/* ----------------------- */

typedef struct
  {
    char *Path;
    char *Nom_Fichier;
  } TCellule_Info,*TFichier_Info;

#define Fichier_Info_Vide (TFichier_Info)NULL

             /* ******************************************** */
             /* Définition de tout ce qui concerne une relation */
             /* ******************************************** */

/* --------------------- */
/* définition d'un champ */
/* --------------------- */

typedef struct L_Champ
  {
    char Nom_Champ[Taille_Nom];
    short Nature;
    union
      {
        struct
          {
            short Type_Champ;
            int Taille_Champ;
            int Nb_Decimal;
            char *Domaine;
            TFichier_Info Info;
          } Complet;
        struct L_Champ* Reference;
      } Declaration;
    char *Valeur;
    struct L_Champ* Suivant;
  } TCellule_Champ,*TListe_Champ;

#define Liste_Champ_Vide (TListe_Champ)NULL
```

```
/* ---------------------- */
/* définition d'une table */
/* ---------------------- */

typedef struct L_Table
  {
    char Nom_Table[Taille_Nom];
    int Actif; /* Enregistrement Actif ou effacé */
    TListe_Champ Cle;
    TListe_Champ Champ;
    TFichier_Info Info;
    struct L_Table* Suivant;
  } TCellule_Table,*TListe_Table;

#define Liste_Table_Vide (TListe_Table)NULL

/* ------------------------------------------------- */
/* définition de la structure d'un index pour un champ */
/* ------------------------------------------------- */

typedef struct
  {
    unsigned int Indice;
    unsigned int Nb_Element;
  } TIndex_Champ;

                        /* ********************** */
                        /* définition d'une requête */
                        /* ********************** */

/* ------------------------------------------------------------------ */
/* définition de la constitution d'un arbre binaire pour la sélection */
/* ------------------------------------------------------------------ */

typedef struct
  {
    TListe_Champ Champ;
  } TCellule_Select_Champ,*TTableau_Select_Champ;
#define Tableau_Select_Champ_Vide (TTableau_Select_Champ)NULL

typedef struct
  {
    TListe_Champ Champ;      /* pointeur sur le champ appartenant */
                              /* à au moins une condition */
    unsigned int Indice;     /* Valeur lue dans l'enregistrement */
    int Descripteur_Fichier; /* descripteur au besoin du fichier champ */
    int Position;            /* position de Indice dans l'enregistrement */
    int Taille_Champ;        /* longueur réelle du champ concerné */
  } TCellule_Tableau_Champ,*TTableau_Champ;
#define Tableau_Champ_Vide (TTableau_Champ)NULL

typedef struct
  {
    TListe_Table Table;      /* pointeur sur la table concernée */
    unsigned int n;          /* n iéme enregistrement en cours de lecture */
  } TCellule_Tableau_Table,*TTableau_Table;
#define Tableau_Table_Vide (TTableau_Table)NULL
```

```
typedef struct
  {
    int Signe; /* signe de la comparaison */
    TTableau_Champ Membre_Gauche;
    struct
      {
        char *Valeur;
        short Existance_Insertion;
        int Indice;
      } Membre_Droit;
  } TCellule_Signe,*TArbre_Signe;
#define Arbre_Signe_Vide (TArbre_Signe)NULL

typedef struct A_Conjonction
  {
    short Nature_Noeud;
    short Nature_Noeud_Gauche;
    struct
      {
        TArbre_Signe Signe;
        struct A_Conjonction* Conjonction;
      } Membre_Gauche;

    short Nature_Noeud_Droit;
    struct
      {
        TArbre_Signe Signe;
        struct A_Conjonction* Conjonction;
      } Membre_Droit;
  } TCellule_Conjonction,*TArbre_Conjonction;
#define Arbre_Conjonction_Vide (TArbre_Conjonction)NULL

typedef struct A_Selection
  {
    short Nb_Champ_Select;
    TTableau_Select_Champ Select_Champ;
    short Nb_Champ;
    TTableau_Champ Tableau_Champ;
    short Nb_Table;
    TTableau_Table Tableau_Table;
    int Prescan;
    short Nature_Noeud;
    union
      {
        TArbre_Signe       Signe;
        TArbre_Conjonction Conjonction;
      } Membre;
  } TCellule_Selection,*TArbre_Selection;
#define Arbre_Selection_Vide (TArbre_Selection)NULL
```

```
/* ------------------------ */
/* définition d'une requête */
/* ------------------------ */

typedef struct L_Requete
  {
    char Nom_Requete[Taille_Nom];
    TArbre_Selection Selection; /* arbre binaire de la requête */
    TFichier_Info Info;         /* contiendra le nom de la requete associée */
    struct L_Requete* Suivant;
  } TCellule_Requete,*TListe_Requete;

#define Liste_Requete_Vide (TListe_Requete)NULL

                            /* ******************** */
                            /* définition du projet */
                            /* ******************** */

/* ------------------------------ */
/* définition du type d'un projet */
/* ------------------------------ */

typedef struct
  {
    char Nom_Projet[Taille_Nom]; /* nom logique d'un projet */
    char Mot_De_Passe_Donnee[Taille_Mot_Passe]; /* mot de passe du projet */
    char Mot_De_Passe_Structure[Taille_Mot_Passe];
    TListe_Table Table;       /* liste des tables */
    TListe_Requete Requete;   /* liste des requêtes */
    TListe_Champ Variable;    /* liste des variables globales */
    TFichier_Info Info;
  } TProjet;
```

SYSTEME DE GESTION DE BASES DE DONNEES

GESTION D'UNE APPLICATION

1. Qu' est-ce que représente une application ?

Une application correspond à un schéma de relations, des requêtes de consultations, des masques d' écrans permettant la saisie mais aussi et l' affichage des résultats de requêtes. Les deux derniers cas devront être réalisés par le programmeur avec les outils qu'il jugera adéquate.

2. Quels sont les objectifs de la conception d' une application ?

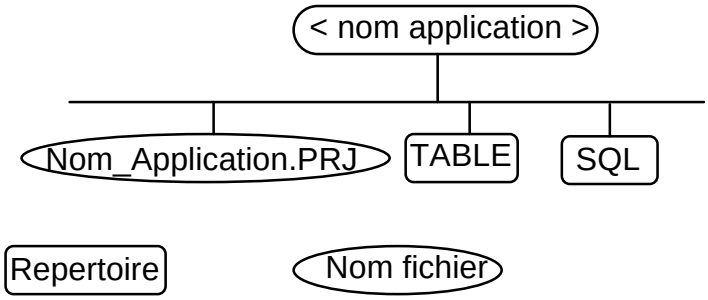
La conception d' une application permettra la création d' une arborescence propre où chaque fichier lié à l' application trouve sa place :

- Le dictionnaire dans le répertoire d'accès de l' application,
- Les fichiers correspondant aux tables, dans un répertoire TABLE
- Les fichiers SQL (Requêtes) dans un répertoire SQL

Tels sont les objectifs de la conception d' une application.

3. Quels sont les traitements applicables à une application ?

1. Création d' une application
   - génération de l' arborescence en vue de son utilisation



2. Modification de certains éléments d' une application
   - Modification d' informations élémentaires sur lesquels reposent l' application
   - Gestion d' un mot de passe pour l'accès aux données (Saisie, Modification, Suppression)
   - Gestion d' un mot de passe pour les modifications de structures
   - Gestion du nom physique de l' application
3. Ouverture et Fermeture d' une application
   - Chargement, vérification puis construction logique du dictionnaire
   - Sauvegarde physique du dictionnaire puis destruction logique.

4. L' organisation du dictionnaire

Cette organisation doit permettre à un utilisateur de visionner rapidement ce à quoi correspond le dictionnaire mais aussi, elle doit permettre de sauvegarder correctement toute la structure mémoire sur disque sans perdre d' informations qui peuvent être capitales pour la bonne exécution et la bonne utilisation de l' application.

Le dictionnaire est un fichier d' extension .PRJ

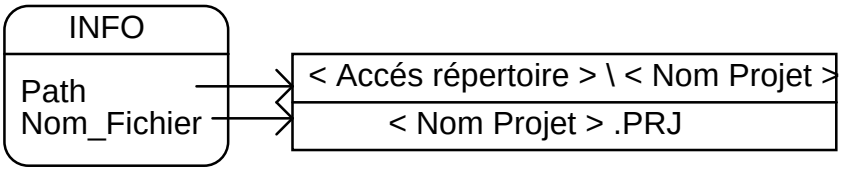
Ce sera un fichier séquentiel en mode ASCII de type texte lisible et compréhensible.

REMARQUE : L' organisation de ce dictionnaire se fera au fur et à mesure de l' avance dans le projet et dans l' analyse de la question. L' analyse se fera par étape et chaque niveau expliquera l' implantation de leurs informations dans le dictionnaire.

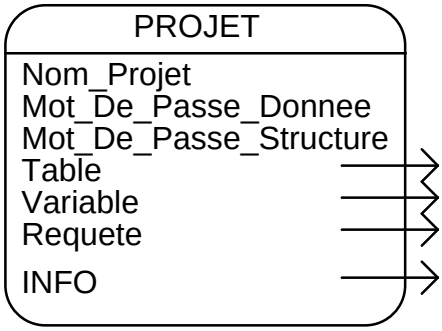
Contenu du dictionnaire pour une application :

PROJET:< nom complet du projet >[: S :< Mot de passe structure >] [:D:< Mot de passe données >]

5. Contenu de la structure info dans la cellule PROJET



6. Description de la structure projet



```
/* ************************************************************************ */
/*                Systéme de Gestion de Bases de Données                    */
/*                              Version 1.0                                 */
/*                                                                          */
/* Date : 01/06/95                                                          */
/*                                                                          */
/*                                                        par LESERT Aymeric */
/* ************************************************************************ */

/* ************************************************************************ */
/*                     module de gestion d'une application                  */
/* Date : 23/04/94                                                          */
/* ************************************************************************ */

/* les fonctions développées dans ce module sont :  */
/*       - création d'une application,              */
/*       - ouvrir une application,                  */
/*       - fermer une application,                  */
/*       - modification d'une application           */

/* ---------------------------------------------------------------- */
/* fonction : int Creer_Application(char *Path,char *Nom_Physique,  */
/*                                  char *Nom_Complet);             */
/* Les élèments indispensables pour la création d'une application : */
/*    - Nom complet du projet,                                      */
/*    - Nom physique du projet,                                     */
/*    - Path d'accés du projet                                      */
/* Cette fonction retourne 0 dans le cas où il n'y pas d'erreurs :  */
/*    - 1 : Un projet est ouvert                                    */
/*    - 2 : Path incorrect                                          */
/*    - 3 : Nom Physique déjà existant                              */
/*    - 4 : Allocation impossible                                   */
/*    - 5 : Nom Complet du projet trop long                         */
/*    - 6 : Nom Physique du projet trop long                        */
/*    - 7 : Syntaxe du Nom Physique incorrecte                      */
/*    - 8 : Création répertoire impossible                          */
/*    - 9 : Création du dictionnaire impossible                     */
/*    - 10: Le Nom complet ne peut pas être vide                    */
/*    - 11: Le Nom complet est mal défini                           */
/* ---------------------------------------------------------------- */

extern "C" int Creer_Application(char *Path,char *Nom_Physique,
                                                        char *Nom_Complet);
```

```
/* ------------------------------------------------------------------ */
/* fonction : int Ouvrir_Application(char *Path,char *Nom_Physique); */
/* Les élèments indispensables pour l'ouverture d'une application :  */
/*    - Nom physique du projet,                                       */
/*    - Path d'accés du projet                                        */
/* Cette fonction retourne 0 dans le cas où il n'y pas d'erreurs :   */
/*    - 1 : Un projet est déjà ouvert                                 */
/*    - 2 : Le répertoire d'accés n'est pas correct                   */
/*    - 3 : Le Nom_Physique n'est pas une application                 */
/*    - 5 : Allocation impossible                                     */
/*    - 6 : Le dictionnaire n'existe pas                              */
/*    - 7 : Le dictionnaire ne peut pas être ouvert en lecture        */
/*    - 8 : Mauvaise définition du projet                             */
/*    - 9 : Caractére incorrect 'S' ou 'D' attendu                    */
/*    - 10: Nom de l'application trop long                            */
/*    - 11: Nom de l'application mal définie                          */
/*    - 12: Mot de passe donnée trop long                             */
/*    - 13: Mot de passe donnée mal définie                           */
/*    - 14: Mot de passe structure trop long                          */
/*    - 15: Mot de passe structure mal définie                        */
/*    - 16: Double définition d'un projet                             */
/*    - 17: élèments inconnus dans "TABLE"                            */
/*    - 18: Composition inconnue pour une table                       */
/*    - 19: deux tables non successives                               */
/*    - 20: table déjà existante                                      */
/*    - 21: Le nom d'une table ne peut pas être vide                  */
/*    - 22: Le nom de la table est trop long                          */
/*    - 23: Le nom de la table est mal défini                         */
/*    - 24: Allocation impossible                                     */
/*    - 25: Repertoire d'accés à l'application inaccessible           */
/*    - 26: Repertoire des tables inaccessible                        */
/*    - 27: Fichier de données inconnu                                */
/*    - 28: Fichier d'index inconnu                                   */
/*    - 29: Répertoire de la table inaccessible                       */
/*    - 30: Une table ne contient que des CHAMPS ou des CLES          */
/*    - 31: Nom d'une clé ne peut pas être vide                       */
/*    - 32: Nom de la clé trop long                                   */
/*    - 33: Nom de la clé mal définié                                 */
/*    - 34: Composant inconnu pour une clé                            */
/*    - 35: Nom de la référence ne peut pas être vide                 */
/*    - 36: Nom de la référence trop long                             */
/*    - 37: Nom de la référence mal défini                            */
/*    - 38: Insertion d'une clé dans une table non définie            */
/*    - 39: Deux clés successives                                     */
/*    - 40: Clé déjà définie                                          */
/*    - 41: Ordre de rangement des clés non respecté                  */
/*    - 42: Nom d'un champ ne peut pas être vide                      */
/*    - 43: Nom d'un champ trop long                                  */
/*    - 44: Nom d'un champ mal défini                                 */
/*    - 45: Composant inconnu pour un champ                           */
/*    - 46: Deux champs non successifs                                */
/*    - 47: Champ déjà défini                                         */
/*    - 48: Nom déjà défini                                           */
/*    - 49: Type d'une clé incorrect                                  */
```

```
/*    - 50: Taille d'une clé incorrecte                                 */
/*    - 51: Nombre de décimal d'une clé incorrect                       */
/*    - 52: Domaine d'une cle ne doit pas être vide                     */
/*    - 53: Type d'un champ incorrect                                   */
/*    - 54: Taille d'un champ incorrect                                 */
/*    - 55: Nombre de décimal d'un champ incorrect                      */
/*    - 56: Domaine d'un champ ne doit pas être vide                    */
/*    - 57: La taille est comprise entre 0 et 80                        */
/*    - 58: Le nombre de décimal est compris entre 0 et 78              */
/*    - 59: Le fichier de données de la clé est inexistant              */
/*    - 60: Le fichier index de la clé est inexistant                   */
/*    - 61: Le fichier de données du champ est inexistant               */
/*    - 62: Le fichier index du champ est inexistant                    */
/*    - 63: Nombre de composants incorrect pour une requête             */
/*    - 64: Rang de la requête incorrect                                */
/*    - 65: Rang de deux SQL non consécutif                             */
/*    - 66: Rang de la requête déjà existant                            */
/*    - 67: Longueur du nom de la requête trop long                     */
/*    - 68: Longueur du nom de la requête ne peut pas être 0            */
/*    - 69: Domaine de définition non respecté                          */
/*    - 70: Nom de la requête déjà existant                             */
/*    - 71: Allocation impossible                                       */
/*    - 72: Composant SQL incorrect                                     */
/*    - 73: Requête de rang non défini                                  */
/*    - 74: Composant inconnu pour une requête                          */
/*    - 77: Nombre de composants incorrect pour une variable            */
/*    - 78: Le nom de la variable ne peut pas être vide                 */
/*    - 79: Longueur du nom trop long                                   */
/*    - 80: Nom non conforme au domaine de définition                   */
/*    - 81: Nom de variable déjà existant                               */
/*    - 82: Type de variable inconnu                                    */
/*    - 83: Taille_Champ non conforme                                   */
/*    - 84: Nb_Decimal non conforme                                     */
/*    - 85: Taille_Champ non compris entre 1 et 78                      */
/*    - 86: (Caractere,Numerique) Nb_Decimal doit être nulle            */
/*    - 87: (Reel) Nb_Decimal trop important                            */
/*    - 88: (Date) Taille ou Nb_Decimal incorrect                       */
/*    - 89: Trop grand nombre de lignes pour un Memo                    */
/*    - 90: Allocation impossible                                       */
/*    -204: Element inconnu dans le dictionnaire                        */
/*    -205: Un des noms de champ référence n'existe pas                 */
/* ---------------------------------------------------------------- */

extern "C" int Ouvrir_Application(char *Path,char *Nom_Physique);

/* ---------------------------------------------------------------- */
/* fonction : int Fermer_Application();                             */
/* Cette fonction libére l'espace mémoire occupée par une application */
/* Cette fonction retourne 0 quand pas d'erreur sinon :             */
/*    - 1 : Pas de projet ouvert                                    */
/*    - 2 : Répertoire d'accés incorrect                            */
/*    - 3 : Dictionnaire impossible d'ouvrir en écriture            */
/*    - 4 : Nom du projet ne doit pas être vide                     */
/* ---------------------------------------------------------------- */

extern "C" int Fermer_Application();
```

```
/* ------------------------------------------------------------ */
/* fonction : int Modification_Application(char *Mot_De_Passe,    */
/*                   int Commande,char *Nouvelle_Donnee);        */
/* Modification des paramétres d'une application                 */
/* Cette fonction retourne 0 quand tout c'est bien passée sinon : */
/*    - 1 : Pas de projet ouvert                                 */
/*    - 2 : Mot de passe invalide                                */
/*    - 3 : Commande incorrecte 1..3                             */
/*    - 4 : Nouvelle donnée trop longue                          */
/*    - 5 : Nouvelle donnée mal définie                          */
/*    - 6 : Répertoire d'accés incorrect                         */
/*    - 7 : Dictionnaire impossible d'ouvrir en écriture         */
/*    - 8 : Nom du projet doit être non vide                     */
/* ------------------------------------------------------------ */

extern "C" int Modification_Application(char *Mot_De_Passe,int Commande,
                                                    char *Nouvelle_Donnee);

/* ------------------------------------------------------------------- */
/* fonction : int Application_Ouverte();                               */
/* Cette fonction retourne Vrai si une application est ouverte sinon Faux */
/* ------------------------------------------------------------------- */

extern "C" int Application_Ouverte();
```

SYSTEME DE GESTION DE BASES DE DONNEES

GESTION DU DICTIONNAIRE

Le dictionnaire est l' organe principal d' un S.G.B.D. . Cette importance résulte du fait qu' il sauvegarde l' architecture d' une base de données. Cette sauvegarde permet ainsi de la recharger pour l' utiliser à des fins de mises à jour ou de consultations, tout dépend ensuite de ce que veut l' utilisateur (Celui qui manipule les données).

La gestion de ce dictionnaire réside dans deux opérations principales et une troisième secondaire :

- Mise à jour du dictionnaire
- Chargement du dictionnaire
- Libération de l' architecture logique

Dans ce projet, le dictionnaire sera un fichier texte ASCII, lisible et compréhensible pour ceux qui connaissent les différents éléments qui le composent. Il se nommera "< nom de l' application > .PRJ"

Cette partie est un élément interne aux autres modules, c' est à dire, que l' utilisateur des outils conçus dans ce projet, ne pourra pas avoir accès à cette gestion du dictionnaire.

1. Mise à jour du dictionnaire

Cette opération consiste à écrire dans ce fichier tous les éléments architecturaux d' une application. Chaque chapitre de ce dossier présentera tous les éléments qui le composent. Les risques d' erreurs sont très restreints, du fait, du grand nombre de tests effectués lors de la construction ou modification de l' architecture.

Les erreurs pouvant peut-être survenir sont :

- Accès au répertoire accès à l' application inconnue
  (Changement de disquette ...)
- Fichier dictionnaire impossible d' ouvrir en écriture
  (Dans ce cas, c' est plus préoccupant ...),
- Les attributs ont été modifiés,
  (Perte d' informations mémoires ...)
- Le nom de l' application est vide

2. Lecture du dictionnaire

Cette opération nécessite de multiples tests concernant chaque ligne car ce fichier est un fichier de type ASCII donc facilement modifiable avec un traitement de texte ou un simple éditeur en ligne.

Ces tests seront en fonction des besoins de l' architecture. Il faudra vérifier la validité des informations contenues dans le fichier. Ceci peut être long mais comme cette opération ne s' effectue que lors de l' ouverture d' une application.

Il ne faudra rien laisser au hasard car il faut éviter au maximum la corruption de l' architecture et par conséquent, la corruption des données ce qui peut être plus grave.

3. Libération de l' architecture logique

Cette opération est fonction de la présentation de l' architecture. Sa tâche sera de rendre la mémoire aussi propre qu' avant l'utilisation de cette dernière par l' architecture.

Remarque : Ce chapitre au niveau de la programmation nécessitera de nombreux commentaires pour ne pas se perdre dans l' ensemble des vérifications à réaliser avant la validation.

```
/* ********************************************************************** */
/*                Systéme de Gestion de Bases de Données                  */
/*                              Version 1.0                               */
/*                                                                        */
/* Date : 01/06/95                                                        */
/*                                                                        */
/*                                                    par LESERT Aymeric */
/* ********************************************************************** */

/* ********************************************************************** */
/*          MODULE D'EXPLOITATION DU DICTIONNAIRE D'UNE APPLICATION       */
/*        MODULE INTERNE AU SGBD , Non exploitable par l'extérieur !!!    */
/* Date : 23/04/94                                                        */
/* ********************************************************************** */

/* ---------------------------------------------------------------------- */
/* procédure : Libération_Dictionnaire(TProjt **Projet);                  */
/* Cette procédure libére la mémoire de l'architecture du dictionnaire */
/* ---------------------------------------------------------------------- */

extern "C" void Liberation_Dictionnaire(TProjet **Projet);

/* ------------------------------------------------------------ */
/* fonction : int Lecture_Dictionnaire(TProjet *Projet);        */
/* Cette fonction retourne 0 au succés sinon :                  */
/*    - 1 : Le dictionnaire ne peut pas être ouvert en lecture */
/*    - 2 : Mauvaise définition du projet                       */
/*    - 3 : Caractére incorrect 'S' ou 'D' attendu              */
/*    - 4 : Nom de l'application trop long                      */
/*    - 5 : Nom de l'application mal définie                    */
/*    - 6 : Mot de passe donnée trop long                       */
/*    - 7 : Mot de passe donnée mal définie                     */
/*    - 8 : Mot de passe structure trop long                    */
/*    - 9 : Mot de passe structure mal définie                  */
/*    - 10: Double définition d'un projet                       */
/*    - 11: élèments inconnus dans "TABLE"                      */
/*    - 12: Composition inconnue pour une table                 */
/*    - 13: deux tables non successives                         */
/*    - 14: table déjà existante                                */
/*    - 15: Le nom d'une table ne peut pas être vide            */
/*    - 16: Le nom de la table est trop long                    */
/*    - 17: Le nom de la table est mal défini                   */
/*    - 18: Allocation impossible                               */
/*    - 19: Repertoire d'accés à l'application inaccessible     */
/*    - 20: Repertoire des tables inaccessible                  */
/*    - 21: Fichier de données inconnu                          */
/*    - 22: Fichier d'index inconnu                             */
/*    - 23: Répertoire de la table inaccessible                 */
/*    - 24: Une table ne contient que des CHAMPS ou des CLES    */
/*    - 25: Nom d'une clé ne peut pas être vide                 */
/*    - 26: Nom de la clé trop long                             */
/*    - 27: Nom de la clé mal définié                           */
/*    - 28: Composant inconnu pour une clé                      */
/*    - 29: Nom de la référence ne peut pas être vide           */
/*    - 30: Nom de la référence trop long                       */
/*    - 31: Nom de la référence mal défini                      */
/*    - 32: Insertion d'une clé dans une table non définie      */
/*    - 33: Deux clés successives                               */
/*    - 34: Clé déjà définie                                    */
/*    - 35: Ordre de rangement des clés non respecté            */
/*    - 36: Nom d'un champ ne peut pas être vide                */
```

```
/*    - 37: Nom d'un champ trop long                              */
/*    - 38: Nom d'un champ mal défini                             */
/*    - 39: Composant inconnu pour un champ                       */
/*    - 40: Deux champs non successifs                            */
/*    - 41: Champ déjà défini                                     */
/*    - 42: Nom déjà défini                                       */
/*    - 43: Type d'une clé incorrect                              */
/*    - 44: Taille d'une clé incorrecte                           */
/*    - 45: Nombre de décimal d'une clé incorrect                 */
/*    - 46: Domaine d'une cle ne doit pas être vide               */
/*    - 47: Type d'un champ incorrect                             */
/*    - 48: Taille d'un champ incorrect                           */
/*    - 49: Nombre de décimal d'un champ incorrect                */
/*    - 50: Domaine d'un champ ne doit pas être vide              */
/*    - 51: La taille est comprise entre 0 et 80                  */
/*    - 52: Le nombre de décimal est compris entre 0 et 78        */
/*    - 53: Le fichier de données de la clé est inexistant        */
/*    - 54: Le fichier index de la clé est inexistant             */
/*    - 55: Le fichier de données du champ est inexistant         */
/*    - 56: Le fichier index du champ est inexistant              */
/*    - 57: Nombre de composants incorrect pour une requête       */
/*    - 58: Rang de la requête incorrect                          */
/*    - 59: Rang de deux SQL non consécutif                       */
/*    - 60: Rang de la requête déjà existant                      */
/*    - 61: Longueur du nom de la requête trop long               */
/*    - 62: Longueur du nom de la requête ne peut pas être 0      */
/*    - 63: Domaine de définition non respecté                    */
/*    - 64: Nom de la requête déjà existant                       */
/*    - 65: Allocation impossible                                 */
/*    - 66: Composant SQL incorrect                               */
/*    - 67: Requête de rang non défini                            */
/*    - 68: Composant inconnu pour une requête                    */
/*    - 69: Ecran déjà défini pour une requête                    */
/*    - 70: Ecran inconnu pour une requête                        */
/*    - 71: Nombre de composants incorrect pour une variable      */
/*    - 72: Le nom de la variable ne peut pas être vide           */
/*    - 73: Longueur du nom trop long                             */
/*    - 74: Nom non conforme au domaine de définition             */
/*    - 75: Nom de variable déjà existant                         */
/*    - 76: Type de variable inconnu                              */
/*    - 77: Taille_Champ non conforme                             */
/*    - 78: Nb_Decimal non conforme                               */
/*    - 79: Taille_Champ non compris entre 1 et 78                */
/*    - 80: (Caractere,Numerique) Nb_Decimal doit être nulle      */
/*    - 81: (Reel) Nb_Decimal trop important                      */
/*    - 82: (Date) Taille ou Nb_Decimal incorrect                 */
/*    - 83: Trop grand nombre de lignes pour un Memo              */
/*    - 84: Allocation impossible                                 */
/*   -198: Element inconnu dans le dictionnaire                   */
/*   -199: Un des noms de champ référence n'existe pas            */
/* ------------------------------------------------------------- */

extern "C" int Lecture_Dictionnaire(TProjet *Projet);
```

```
/* ------------------------------------------------------------------------ */
/* procedure : Mise_A_Jour_Dictionnaire(TProjet Projet);                    */
/* sortie : Le dictionnaire est créé s'il n'existait pas sinon il est effacé */
/*                 puis réécrit.                                            */
/* Erreur : 0 : Pas d'erreur                                                */
/*          1 : Répertoire d'accés au dictionnaire incorrect                */
/*          2 : dictionnaire impossible d'ouvrir en écriture                */
/*          3 : Le Nom Complet d'un projet ne peut être vide                */
/* ------------------------------------------------------------------------ */

extern "C" int Mise_A_Jour_Dictionnaire(TProjet* Projet);
```

SYSTEME DE GESTION DE BASES DE DONNEES

MODIFICATION DU SCHEMA DE LA BASE

Cette étape permettra de manipuler physiquement, mais aussi logiquement, une table ou une relation dans le schéma des relations. Les outils qui seront mis à disposition de l' utilisateur seront :

- ajout d' une relation
- ajout d' un champ ou d' une clé
- modification d' information(s) d' une relation
- modification d' information(s) d' une clé ou d' un champ
- suppression d' une relation dans la base
- suppression d' un champ ou d' une clé

Il faudra faire attention à ne pas perdre de données dans la base. Pour garantir un maximum de sécurité, à chaque traitement qui sera exécuté, il faudra réaliser des tests de faisabilité (exemple : non-suppression d' un champ ou d' une clé qui sert de référence, suppression d' une clé unique, ...)

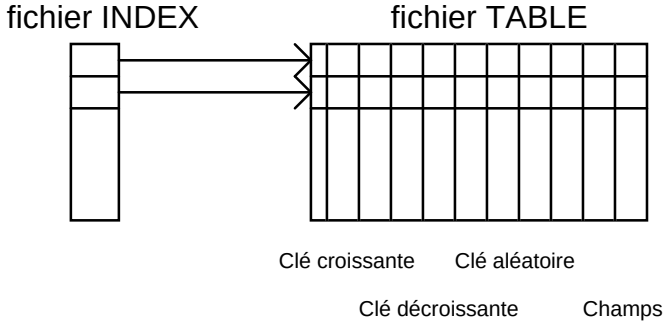
Toutes les modifications physiques des données se feront par des fichiers temporaires qui seront validés seulement à la fin de la mise à jour. De plus à chacun de ces traitements réussis, il ne faudra pas oublier de sauvegarder la nouvelle structure ainsi obtenue dans le dictionnaire des données.

1. Organisation physique d' une relation (ou d' une table)

Rappel : Une relation sera synonyme d' une table.

Chaque table se constituera de clé(s) et/ou de champ(s), il faudra donc stocker chacune de ces informations en évitant les redondances d' informations et en permettant un accès rapide aux données.

Ces deux impératifs, on le sait, ne sont pas vraiment compatibles en terme de performances. Aussi, je préfère perdre du temps dans la sauvegarde des informations plutôt que dans la recherche de ces dernières : Ceci est un choix stratégique.



Le premier caractère de chaque enregistrement sera un caractère qui indiquera l' état de l' enregistrement à un moment précis de son utilisation. Chaque enregistrement possédera deux états, le premier, l' état actif (l' enregistrement est consultable), et le deuxième, l' état passif (enregistrement théoriquement supprimé de la base).

Lorsqu' un enregistrement est inactif (passif), cela signifie qu' il pourra être récupéré.

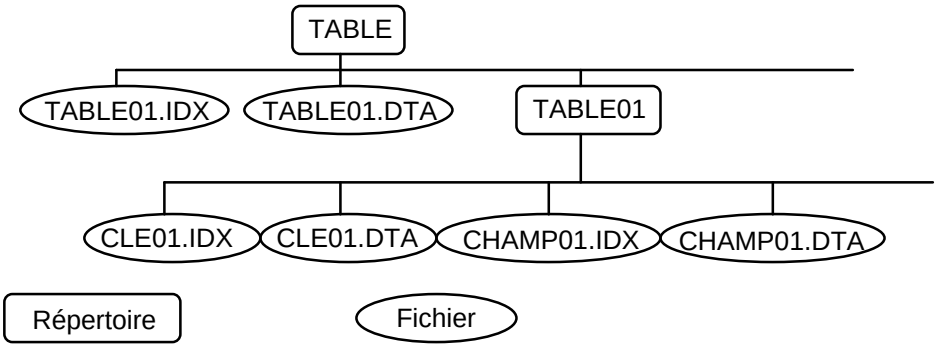
Chaque valeur de champ d' un enregistrement sera un indice sur le fichier de valeur d' un champ qui aura pour objectif, la non-redondance des informations répertoriées dans une base. Ainsi, la taille de chaque champ dans un enregistrement de la table aura la même longueur et par conséquent, la taille d' un enregistrement sera facilement calculable. Le fichier index aura pour mission de garder le fichier table trié selon l' ordre de tri des clés.

Remarque : L' ordre de tri des clés pouvant se scinder en trois catégories :
- Les clés croissantes,
- Les clés décroissantes,
- Les clés aléatoires (sans ordre apparent).

Pour une plus grande facilité de traitements, l' ordre dans lequel se présenteront les enregistrements sera le suivant :

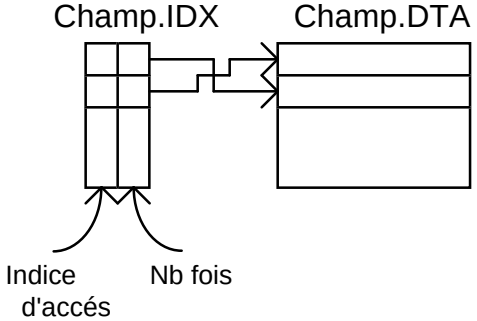
1 - Les clés croissantes
2 - Les clés décroissantes,
3 - Les clés aléatoires,
4 - Les autres champs.

Ainsi physiquement, cela se traduira par :



Ainsi, chaque champ sera stocké en dehors de la table. Chaque champ possédera un index qui le triera selon l' ordre croissant, quel que soit la nature du champ (Clé croissante, clé décroissante, clé aléatoire, champ) afin de pouvoir toujours appliquer l'algorithme de recherche dichotomique.

Chaque index d' un champ associé contiendra deux informations, la première contiendra l' indice où il pourra trouver la valeur recherchée et la deuxième, le nombre de fois qu' est répertoriée l' information dans l' ensemble des relations.



2. Les opérations réalisables sur cet ensemble

2.1. Les opérations sur les tables

Chaque table comportera un nom qui ne pourra pas être redondant, c' est à dire que deux tables ne pourront pas avoir le même nom, et elle contiendra une liste de clés et/ou de champs.

Trois opérations sont envisageables :

- Création d' une table
- Modification du nom d' une table
- Suppression d' une table

Remarque : une table ne comportant pas de clés ni de champs pourra être créée mais ne pourra pas être exploité. Une condition nécessaire à son exploitation (son utilisation) sera la présence au moins d' une clé.

2.1.1. La création d'une table

Une nouvelle table sera insérée en queue de liste.

Physiquement, il y aura la création d' un fichier index, d' un fichier de données et d' un répertoire où seront stockées les valeurs des clés et des champs.

2.1.2. Modification du nom d'une table

Ceci n' apporte pas de grand changement physique, ni logique, seulement une mise à jour du dictionnaire. Il faut prendre soin de la non-existence du nom de la table dans les noms de tables, de clés ou de champs.

2.1.3. La suppression d'une table

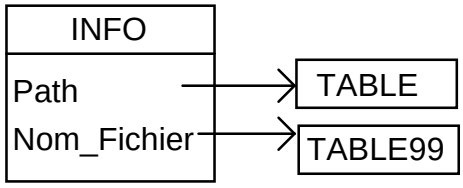
Pour supprimer une table, il faudra avoir supprimé toutes les clés et tous les champs avant, c' est à dire que la table ne soit plus qu une entité constituée d' une seule cellule.

Cette suppression s' accompagnera d' une remise à jour de toutes les cellules TABLES suivantes celle supprimée, cette mise à jour s' effectuant, pour chaque TABLE sur les structures INFO attachées à la TABLE, aux clés et/ou aux champs.

2.1.4. Déclaration d'une table dans le dictionnaire

TABLE99: < nom de la table >

2.1.5. Contenu de la structure INFO



2.2. Les opérations sur les clés et les champs

Il existera deux états d' un champ :

- les champs normaux,
- les champs référencés.

Il existera cinq types de champs :

- Le caractère,
- Le numérique,
- Le réel,
- La date,
- Le mémo.

Il faudra éviter la création d' un cyclage des références, c' est-à-dire un champ référencé qui finit par se référencer sur lui-même ce qui ne permettra pas de connaître le type, ni les informations nécessaires à l' exploitation d' un champ.

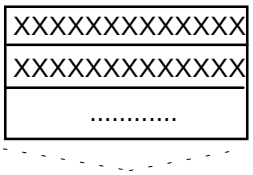
Cinq opérations pourront être réalisées :

- Création d' une clé
- Création d' un champ
- Modification de paramètres d' un champ ou d' une clé
- Suppression d' une clé
- Suppression d' un champ

Le nom d' un champ ne doit pas exister deux fois dans la base car le nom du champ servira de clé d'accès pour la mise à jour.

2.2.1. Sauvegarde des différents types

2.2.1.1. Le caractère



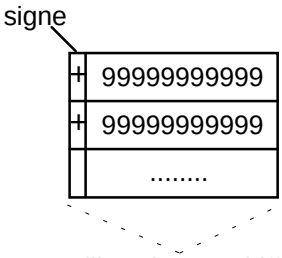
Taille_Champ Caractéres

Fichier séquentiel

Ne contient jamais de caractéres de contrôle

Aligné à gauche

2.2.1.2. Le numérique



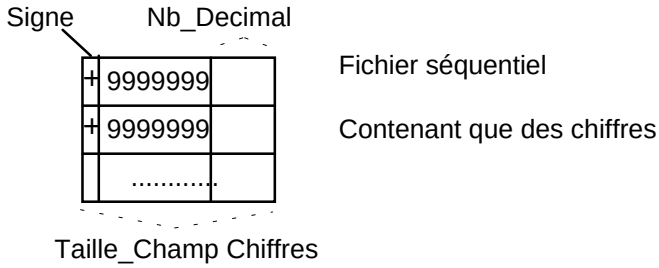
Taille_Champ Chiffres

Fichier séquentiel

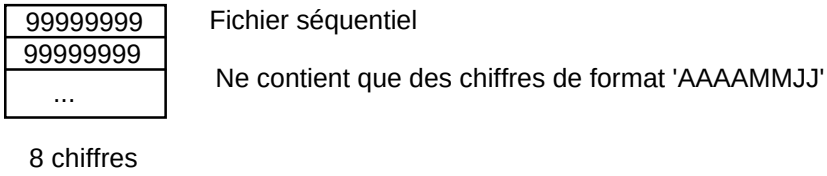
Chaque nombre complété par des '0' à gauche

Aligné à droite

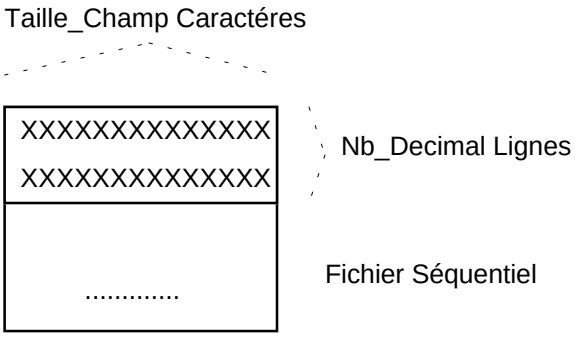
2.2.1.3. Le réel



2.2.1.4. La date



2.2.1.5. Le mémo



2.2.2. Création d' une clé ou d' un champ

2.2.2.1. Création d'une clé

Pour créer une clé, il faudra connaître, au moins, le nom de la table dans laquelle la clé doit être insérée, le nom de la clé et l' ordre de tri de la clé. Les informations par défauts seront :

- le type : caractère
- la taille : 0
- le nombre de décimal : 0
- Domaine de définition : tout (" * ")

Cette clé, ainsi définie, sera insérée dans la liste à la place qui lui est due, sachant que l' ordre d' apparition des clés est :

1- Les clés croissantes
2- Les clés décroissantes
3- Les clés aléatoires

Physiquement, cela se traduira par la création de deux fichiers vides dans le répertoire de la table correspondant à son insertion.

Il faudra que la table indiquée existe, que le nom de la clé ne soit pas déjà existant et que l' ordre de tri soit parmi les trois indiqués (Clé croissante, clé décroissante ou clé aléatoire).

2.2.2.2. Création d'un champ

La création d' un champ nécessite la connaissance du nom de la table dans laquelle le champ doit être inséré. Il ne doit pas être redondant. Suite à quoi, les paramètres de ce champ seront positionnés par défaut :

- Nature : Champ normal
- Type_Champ : Caractère
- Taille_Champ : 0
- Nb_Décimal : 0
- Domaine de définition : tout (" * ")

2.2.3. La modification des paramètres d' un champ

Cette partie est complexe du fait que l' un des buts de ce projet est de ne pas perdre les données, si l' utilisateur modifie le schéma de la base. Cette modification est simple au niveau logique, car cela se traduit par de simples affectations successives sur les paramètres concernés mais au niveau physique, là où sont stockées les données, cela est une autre chose.

Pour des questions de sécurité, il sera prudent de réaliser les modifications demandées dans des fichiers temporaires. Ils permettront une vérification. Si les modifications ont été acceptables, le niveau logique est modifié avant que les fichiers temporaires viennent écraser ceux qui ont subi les modifications.

2.2.3.1. Les différentes modifications pouvant être faites

- Modification du nom du champ ou de la clé
- Modification de la taille du champ ou de la clé
- Modification du type :
    - passage du caractère au numérique
    - passage du caractère à la date
    - passage du caractère au réel
    - passage du caractère au mémo
    - passage du numérique au caractère
    - passage du numérique au réel
    - passage du numérique à la date
    - passage du réel au caractère
    - passage du réel au numérique
    - passage de la date au caractère
    - passage de la date au Numérique
    - passage du mémo au caractère
- Modification du domaine de définition
- Modification de la nature d' un champ ou d' une clé
    - Changement d' ordre
    - Perte de la référence
    - Accès à la référence

2.2.3.2. Analyse des différentes modifications

Pour chaque modification dans un champ, ce dernier subira une épuration (Cf. 2.2.5. Epuration d' un champ).

1. Modification du Nom d' un champ ou d' une clé

Cette modification n' est pas la plus fondamentale, elle ne perturbe pas les données. Elle consiste à mettre à jour le nom de la cellule concernée par cette modification et une simple mise à jour du dictionnaire de l' application.

Remarque : void *Nouveau_Paramétre -> char *Nouveau_Nom

2. Modification de la taille du champ ou de la clé

Cette modification ne perturbe pas la table d' index car les données ne sont pas fondamentalement modifiées. Le Champ est soit allongé, soit écourté. Il faudra juste vérifier que les informations ainsi générées ne sont pas redondantes, auquel cas deux possibilités sont à envisager, c' est à dire :

- Soit, nous devons faire la remise à jour de la table d' index et par conséquent, modifié pour chaque table contenant ce champ, l' ensemble des indices dans les tables,
- Soit, une erreur dans le reformatage du champ sera générée, et les modifications demandées ne seront pas accordées, ceci dans le cas où la taille vient à être réduite.

La première solution peut impliquer de long traitement sur le physique et nécessitera de la place dans le cas où toutes les modifications transiteront par un fichier temporaire, si on désire la mettre en oeuvre. Au niveau logique, ce traitement est très simple et très rapide, une fois les protagonistes réunis. L' inconvénient majeur de cette solution est la perte irrémédiable de données.

La deuxième solution est la plus rapide au niveau physique et la plus simple à mettre en place, elle nécessite un unique fichier temporaire et ne pose pas le problème de la modification de l' index. Les traitements logiques sont rapides. De plus, elle offre une marge de sécurité plus importante car, si l' utilisateur a fait une erreur de traitement, elle sera alors détectable.

La modification de la taille pour un champ dont la taille était nulle et dont la table concernée n' est pas vide, une valeur neutre (ensemble d' espaces) est ajoutée dans le fichier de données du champ et l' index est réévalué.

Remarque : void *Nouveau_Parametre -> int p[2]
(p[0] : Taille_Champ et p[1] : Nb_Decimal)

3- Modification du type de champs

Cette modification n' aporte pas de grands changements dans la majorité des cas, seulement une remise à jour du fichier de données du champ concerné. Cependant, dans deux cas, passage du caractère au numérique ou au réel, l' index du champ peut subir une modification.

Ce changement d' index provoque dont une mise à jour de tous les index des tables concernées par ce champ.

Modification de types possibles

Type Départ	***Format Départ***	***Type Arrivé***	***Format Arrivé***
Caractère	+/-[0-9]*	Numérique	+/-[0-9]*
Caractère	AAAA/MM/JJ	Date	AAAAMMJJ
Caractère	+/-999...,99...	Réel	+/-999...99...
Caractère	*	Mémo	*
Numérique	+/-9999...	Caractère	+/-9999...
Numérique	+/-9999...	Réel	+/-9999...
Numérique	+AAAAMMJJ	Date	AAAAMMJJ
Réel	+/-999...99...	Caractère	+/-999...,99..
Réel	+/-999...	Numérique	+/-999...
Date	AAAAMMJJ	Caractère	AAAA/MM/JJ
Date	AAAAMMJJ	Numérique	+AAAAMMJJ
Mémo	*	Caractère	*

Remarque : void *Nouveau_Parametre -> char Type_Champ

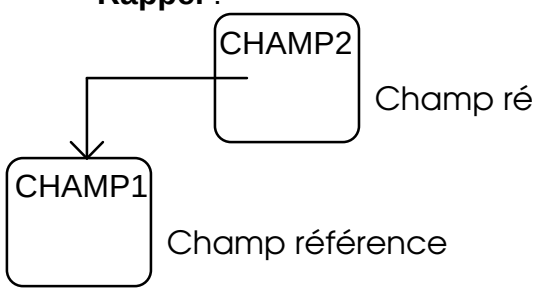
4- Modification du domaine de définition

Cette modification ne modifie pas les données, par conséquent, elle ne modifie pas l' index des données. Cette modification lit chaque valeur du champ en question puis vérifie que le nouveau domaine est bien respecté.

Remarque : void *Nouveau_Parametre -> char *Nouveau_Domaine

5- Modification de la nature d' un champ

Rappel :



Remarque : void *Nouveau_Parametre ->
struct
{
char Nouvelle_Nature;
char *Nom_Référence;
}

5-1- Perte de la référence

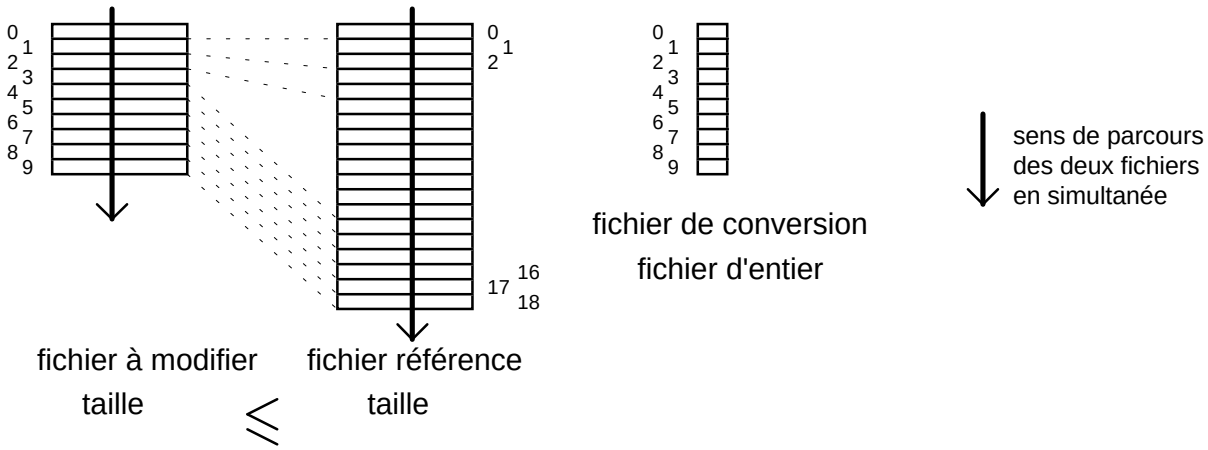
Cette modification n' influe en aucun cas sur les index des différents protagonistes (champ perdant la référence, le champ référence et toutes les tables s' y rapportant). Car il s'agit de recopier les données et la structure à l' endroit réservé au champ référencé. La modification physique nécessite la réévaluation du nombre d' occurrences de chaque donnée pour l' un et l' autre champ (Cf. 2.2.6. Réévaluation du nombre d' occurrences des données d'un champ).

5-2- Accès à la référence

L'accès à la référence doit éviter d' offrir un cyclage des références, c' est à dire, qu' un champ référencé ne devienne le champ référence de lui même.

Cette modification implique l' épuration des deux champs protagonistes, le champ référence et le champ référencé afin d' éliminer les données d' occurrences nulles. Cette épuration apporte un côté pratique car les données sont classées comme l' index de ces données, ceci permet alors de mieux comparer les données par la méthode des poids binaires.

Toutes les données du champ devenant référencé doivent être présentes dans les données du champ qui deviendra la référence.



Le fichier à modifier est le fichier qui veut devenir référencé sur le fichier référence. Le fichier de conversion contient les nouveaux indices dans le fichier de données des données équivalentes.

Le fichier à modifier et le fichier de référence sont parcourus séquentiellement en simultanée suivant la méthode des poids binaires:

- (1) Lecture de la valeur du fichier à modifier
- (2) Lecture de la valeur dans le fichier référence
- (3) si les deux valeurs sont égales,
  l' indice du fichier référence est écrit dans le fichier de conversion.
  puis on recommence en (1) sans recommencer au début des deux fichiers.
- (4) l' algorithme s' arrête quand la fin d' un des deux fichiers est atteint.
- (5) Si la fin du fichier à modifier est atteint avant ou en même temps que le fichier référence, toutes les données sont communes sinon il y a erreur, l'accès à la référence ne peut pas se faire.
- (6) toutes les tables, étant en rapport avec le champ à modifier, sont copiés dans des fichiers TEMPO99 avec l' indice du champ modifié. Le nouvel indice de la donnée est lu dans le fichier de conversion à l' ancien indice de la donnée.

Cette modification provoque l' ajout dans le fichier index du champ référence du nombre d' occurrences déjà présent dans le fichier index du champ à modifier.

Remarque : Les informations des deux champs doivent être compatibles :

- même type de champ
- même taille de champ
- même nombre de décimales

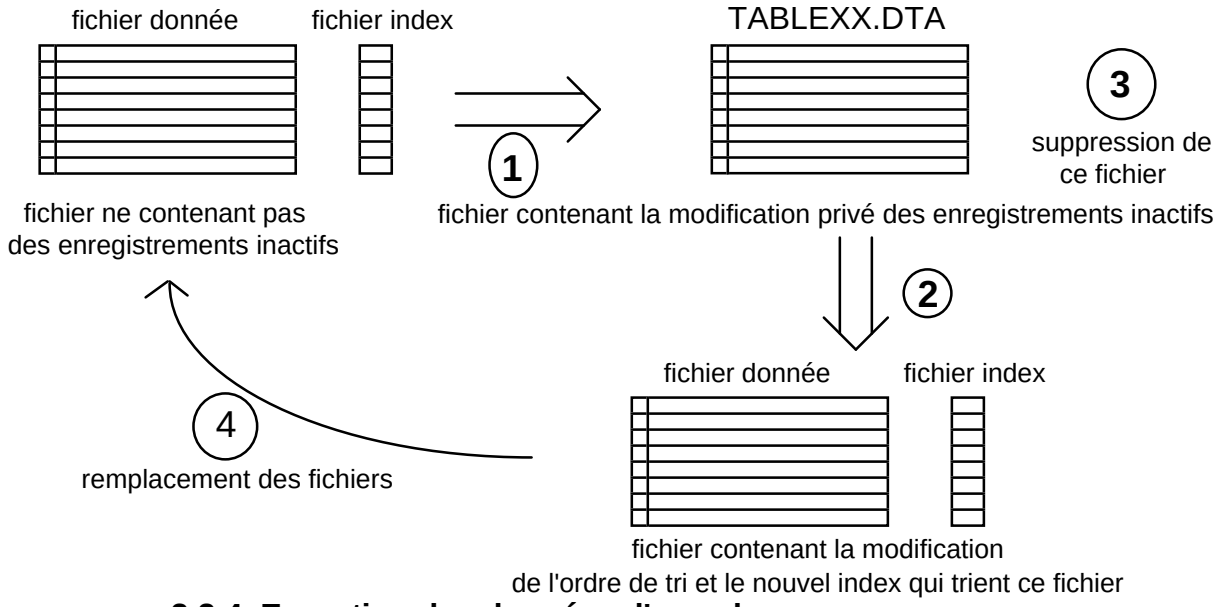
5-3- Modification de l'ordre de tri d'une clé

Cette modification consiste à créer un fichier temporaire ("TEMPOXX.DTA") contenant les mêmes informations que le fichier de bases avec la modification de l' ordre de tri à effectuer, sans recopier l' index de la table concernée. Cette modification est :

- soit une translation vers la gauche,
- soit une translation vers la droite

en fonction de l' ancienne position et de la nouvelle position.

Une fois ce fichier constitué, il est donc nécessaire de reconstruire un index en fonction de la nouvelle configuration. Si cette dernière provoque une redondance d' une clé, la modification ne pourra pas être tolérée.



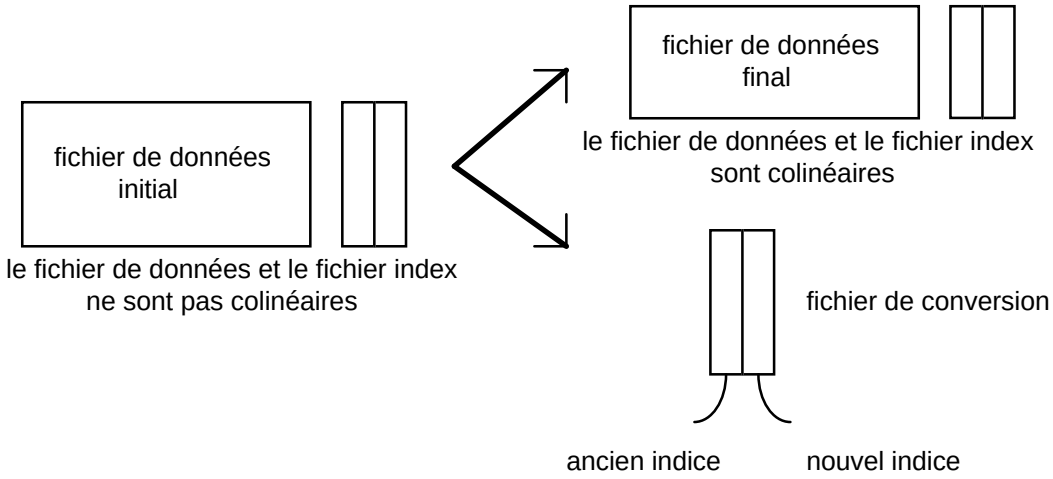
2.2.4. Epuration des données d' un champ

L' épuration des données d' un champ consiste à la suppression des données d' un champ dont le nombre d' occurrences de cette dernière est nulle. Cette modification affecte l' index du champ en question et l' ensemble des enregistrements du champ concerné.

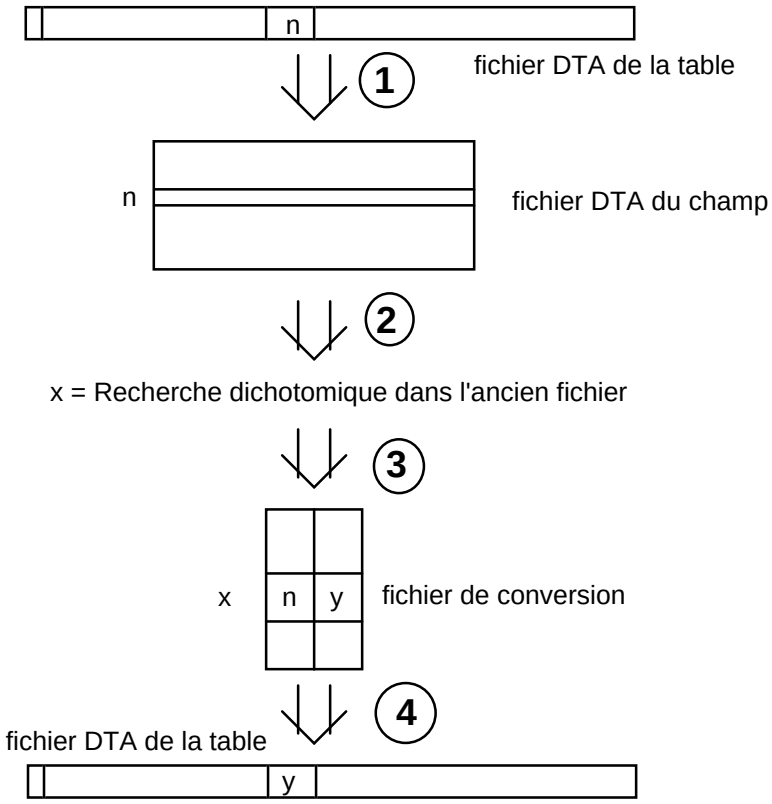
Cette épuration fera donc disparaître, les données d' occurrences nulles, ce qui signifie que les enregistrements inactifs concernés seront aussi détruits. Il sera conseillé à l' utilisation d' effectuer de temps à autre une épuration de l' ensemble de ces champs, si les tables sont sujettes à de nombreuses manipulations.

Cette épuration s' effectue en deux temps :

- Suppression des données d' occurrences nulles et constitution du fichier de conversion.



- Modification des enregistrements actifs de toutes les tables concernées par cette épuration et suppression définitive des enregistrements inactifs.



Cette épuration aura pour conséquence de rendre le fichier de données du champ et le fichier index du champ colinéaire. Mais, aussi, tous les fichiers de données et tous les index des tables concernées par cette épuration. Car les enregistrements et les données ne sont pas lus dans n' importe quel ordre. Pour lire ces informations, la lecture s' effectuera par sa position dans le fichier index correspondant à l' ancien fichier.

2.2.5. Réévaluation du nombre d' occurrences de chaque donnée

Cette réévaluation consiste à recalculer le nombre d' occurrences de la donnée qui a été répertoriée. Ce traitement entre dans le cadre de la perte de la référence mais aussi dans certains autres cas rares. Il se peut qu' au cours d' une recherche d' un enregistrement, une erreur ou un ennui intervienne pour que la valeur contenue soit modifiée. Seulement dans ces cas précis, il est utile de l' utiliser.

Cette réévaluation ne modifiera que les données contenues dans l' index du champ réévalué.

2.2.6. Suppression d' un champ

Cette suppression ne doit pas être acceptée:
- si le champ est un champ référence,
- si le champ est une clé unique pour une table contenant encore des champs

Après ces tests préliminaires, il faudra supprimer cette donnée de tous les enregistrements de la table concernée. La procédure est la même que la modification d' environ tri, seulement, le champ n' est pas replacé, il est purement supprimé.

2.2.7. Déclaration d' une clé ou d' un champ dans le dictionnaire

2.2.7.1. Déclaration d'une clé

1. Déclaration d' une clé non référencée

TABLE99:CLE99:<nom de la clé>:
<ordre de tri>:<type>:<taille>:<Nb_Dec>:<Domaine>

où :
nom de la clé : 32 caractères maxi
Ordre de Tri : 1 caractère
Type : 1 caractère
Taille : 99
Nb_Dec : 99
Domaine : fonction du type

2. Déclaration d' une clé référencée

TABLE99:CLE99:< nom de la clé > : < Ordre > : < nom du champ référence >

2.2.7.2. Déclaration d'un champ

1. Un champ normal

TABLE99:CHAMP99:< nom du champ>:<nature du champ>:
<Type>:<Taille>:<Nb_Dec>:<Domaine>

Composition : Cf. ci-dessus

2. Un champ référencé

TABLE99:CHAMP99:<nom du champ>:< nature >:< nom du champ référence>

2.2.7.3. Détail des éléments

1. Ordre de tri d' une clé

C : Clé croissante
D : Clé décroissante
A : Clé aléatoire
c : Clé croissante référencée
d : Clé décroissante référencée
a : Clé aléatoire référencée

2. Nature d' un champ

N : Champ normal
R : Champ référencé

3. Le type

C : Caractère
N : Numérique
R : Réel
D : Date
M : Mémo
(Taille_Champ : Nb de caractères par ligne)
(Nb_Decimal : Nb de lignes)

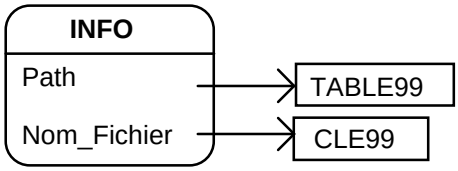
4. Le domaine de définition

Il sera défini comme une expression régulière étendue avec comme cas particulier :

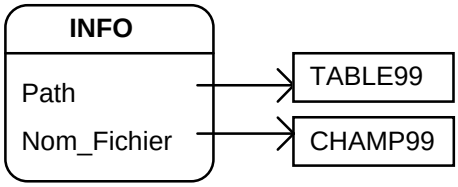
- "*" : tous les caractères ou chaîne vide
- "+" : tous les caractères et chaîne d' au moins un caractère

2.2.8. Contenu de la structure INFO d' un champ ou d' une clé

2.2.8.1. Structure INFO d' une clé

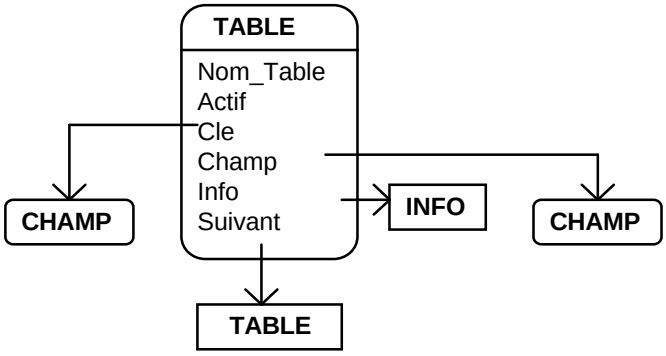


2.2.8.2. Structure Info d' un champ

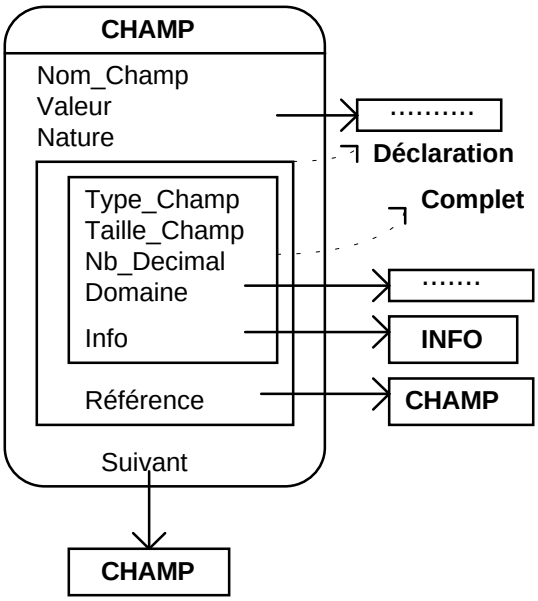


3. Les structures manipulées

3.1. La structure TABLE



3.2. La structure CHAMP



```
/* ************************************************************************ */
/*                Systéme de Gestion de Bases de Données                    */
/*                              Version 1.0                                 */
/*                                                                          */
/* Date : 01/06/95                                                          */
/*                                                                          */
/*                                                        par LESERT Aymeric */
/* ************************************************************************ */

/* ************************************************************************ */
/*               MODULE DE GESTION DU SCHEMA DES RELATIONS                  */
/*                                                                          */
/* Date : 24/04/94                                                          */
/* ************************************************************************ */

/**************************************************************************/
/*                   Partie 1 : Gestion des tables                        */
/**************************************************************************/

/* ---------------------------------------------------------- */
/* int Creer_Table(char *Mot_De_Passe,car *Nom_Table);        */
/* fonction de création d'une table                           */
/* Cette fonction retourne 0 quand il n'y a pas d'erreurs sinon : */
/*    - 1 : Pas d'application ouverte                         */
/*    - 2 : mot de passe non valide                           */
/*    - 3 : Un nom de table ne peut pas être vide             */
/*    - 4 : Le nom de la table est trop long                  */
/*    - 5 : Le nom de la table posséde un caractére incorrect */
/*    - 6 : Le nom de la table existe déjà                    */
/*    - 7 : Allocation impossible                             */
/*    - 8 : Création d'une table impossible, capacité maxi de 99  */
/*    - 9 : Répertoire d'accés à l'application inaccessible   */
/*    - 10: répertoire d'accés aux tables inaccessible        */
/*    - 11: Fichier donnée impossible d'ouvrir en écriture    */
/*    - 12: Fichier index impossible d'ouvrir en écriture     */
/*    - 13 : impossible de créer un répertoire                */
/*      Les erreurs qui suiventne détruisent pas la cellule créée */
/*    - 14 : Répertoire d'accés au dictionnaire impossible    */
/*    - 15 : Dictionnaire impossible d'ouvrir en écriture     */
/*    - 16 : Nom complet de l'application ne doit pas être vide   */
/* ---------------------------------------------------------- */

extern "C" int Creer_Table(char *Mot_De_Passe,char *Nom_Table);
```

```
/* ------------------------------------------------------------------------ */
/* fonction : int Modifier_Table(char *Mot_De_Passe,char *Ancien_Nom,       */
/*                               char *Nouveau_Nom);                        */
/* Cette fonction permet de modifier le nom d'une table parmi les relations */
/* Cette fonction retourne 0 quand pas d'erreur sinon :                     */
/*    - 1 : Pas d'application ouverte                                       */
/*    - 2 : Pas de tables définies                                          */
/*    - 3 : Pas de tables au nom indiquée                                   */
/*    - 4 : Nouveau nom déjà existant                                       */
/*    - 5 : Le nouveau nom de table ne peut pas être vide                   */
/*    - 6 : Le nouveau nom de table est trop long                           */
/*    - 7 : Le nouveau nom de table est mal défini                          */
/*    - 8 : Mot de passe incorrect                                          */
/*    - 9 : Répertoire d'accés au dicitonnaire incorrect                    */
/*    - 10: Dictionnaire impossible d'ouvrir en écriture                    */
/*    - 11: Nom de l'application ne peut pas être vide                      */
/*    - 12: Table impliquée dans une requête ouverte                        */
/* ------------------------------------------------------------------------ */

extern "C" int Modifier_Table(char *Mot_De_Passe,char *Ancien_Nom,
                                                        char *Nouveau_Nom);

/* ------------------------------------------------------------------------ */
/* fonction : int Supprimer_Table(char *Mot_De_Passe,char *Nom_Table);      */
/* Cette fonction a pour but de détruire une table de Nom_Table             */
/* Cette fonction retourne 0 si pas d'erreurs sinon :                       */
/*    - 1 : Mot de passe incorrect                                          */
/*    - 2 : La table demandée n'existe pas                                  */
/*    - 3 : Répertoire d'accés au projet introuvable                        */
/*    - 4 : Répertoire d'accés aux tables introuvable                       */
/*    - 5 : Table impossible à supprimer, il reste des clés ou des champs   */
/*    - 6 : Pas projet ouvert                                               */
/*    - 7 : Mot de passe incorrect                                          */
/*    - 8 : Répertoire d'accés au dicitonnaire incorrect                    */
/*    - 9 : Dictionnaire impossible d'ouvrir en écriture                    */
/*    - 10: Nom de l'application ne peut pas être vide                      */
/*    - 11: Table impliquée dans une requête ouverte                        */
/* ------------------------------------------------------------------------ */

extern "C" int Supprimer_Table(char *Mot_De_Passe,char *Nom_Table);

/* ------------------------------------------------------------------------ */
/* fonction : int Existe_Table(char *Nom_Table);                            */
/*   Cette fonction retourne Vrai si l'application contient une table du    */
/* nom de Nom_Table.                                                        */
/* ------------------------------------------------------------------------ */

extern "C" int Existe_Table(char *Nom_Table);
```

```
/************************************************************************/
/*             Partie 2 : Gestion des clés et des champs                */
/************************************************************************/

/* ---------------------------------------------------------------- */
/* fonction : int Creer_Cle(char *Mot_De_Passe,char *Nom_Table,     */
/*                          char *Nom_Cle,short Ordre);             */
/* Cette fonction a pour but d'ajouter une nouvelle clé à une table */
/* Cette fonction retourne 0 quand il n'y a pas d'erreurs sinon :   */
/*    - 1 : Pas de projet d'ouvert                                  */
/*    - 2 : Mot de passe incorrect                                  */
/*    - 3 : le nom de la clé ne peut pas être vide                  */
/*    - 4 : Le nom de la clé est trop long                          */
/*    - 5 : Le nom de la clé est mal défini                         */
/*    - 6 : Le nom de la clé est redondante                         */
/*    - 7 : Le nom de la table n'existe pas                         */
/*    - 8 : Ordre de tri inconnu                                    */
/*    - 9 : Fichier temporaire impossible d'ouvrir en écriture      */
/*    - 10: Fichier temporaire impossible d'ouvrir en lect/ecrit    */
/*    - 11: Table impossible d'ouvrir en lecture                    */
/*    - 12: Allocation impossible                                   */
/*    - 13: Erreur lors de la transformation des données du schéma  */
/*    - 14: Repertoire d'accés à l'application introuvable          */
/*    - 15: Repertoire des tables introuvable                       */
/*    - 16: Répertoire de la table concernée introuvable            */
/*    - 17: Répertoire d'accés au dictionnaire impossible           */
/*    - 18: Dictionnaire impossible d'ouvrri en écriture            */
/*    - 19: Le nom complet d'un projet ne peut pas être vide        */
/*    - 20: Table impliquée dans une requête ouverte                */
/*    - 21: Il existe des écrans en cours d'exécution               */
/* ---------------------------------------------------------------- */

extern "C" int Creer_Cle(char *Mot_De_Passe,char *Nom_Table,char *Nom_Cle,
                                                          short Ordre);

/* ---------------------------------------------------------------- */
/* fonction : int Creer_Champ(char *Mot_De_Passe,char *Nom_Table,   */
/*                                  char *Nom_Champ);               */
/* Cette fonction a pour but d'ajouter un nouveau champ à une table */
/* Cette fonction retourne 0 quand il n'y a pas d'erreurs sinon :   */
/*    - 1 : Pas de projet d'ouvert                                  */
/*    - 2 : Mot de passe incorrect                                  */
/*    - 3 : le nom du champ ne peut pas être vide                   */
/*    - 4 : Le nom du champ  est trop long                          */
/*    - 5 : Le nom du champ  est mal défini                         */
/*    - 6 : Le nom du champ  est redondant                          */
/*    - 7 : Le nom de la table n'existe pas                         */
/*    - 9 : Fichier temporaire impossible d'ouvrir en écriture      */
/*    - 10: Fichier temporaire impossible d'ouvrir en lect/ecrit    */
/*    - 11: Table impossible d'ouvrir en lecture                    */
/*    - 12: Allocation impossible                                   */
/*    - 13: Erreur lors de la transformation des données du schéma  */
/*    - 14: Repertoire d'accés à l'application introuvable          */
/*    - 15: Repertoire des tables introuvable                       */
/*    - 16: Répertoire de la table concernée introuvable            */
/*    - 17: Répertoire d'accés au dictionnaire impossible           */
/*    - 18: Dictionnaire impossible d'ouvrri en écriture            */
/*    - 19: Le nom complet d'un projet ne peut pas être vide        */
/*    - 20: Table impliquée dans une requête ouvertee               */
/*    - 21: Il existe des écrans en cours d'exécution               */
/* ---------------------------------------------------------------- */
```

```
extern "C" int Creer_Champ(char *Mot_De_Passe,char *Nom_Table,char *Nom_Champ);

/* ------------------------------------------------------------------- */
/* fonction : int Reevaluer_Champ(char *Nom_Champ);                    */
/*   Cette fonction réévalue le nombre d'occurrences pour chaque valeur */
/* du champ passé en paramétre.                                        */
/*   Cette fonction retourne 0 si pas d'erreurs sinon                  */
/*    - 1 : Pas d'application ouverte                                  */
/*    - 2 : Nom de champ doit être non vide                            */
/*    - 3 : Nom de champ trop long                                     */
/*    - 4 : Nom de champ mal défini                                    */
/*    - 5 : Nom de champ inconnu                                       */
/*    - 6 : Répertoire d'accés projet impossible                       */
/*    - 7 : Répertoire d'accés aux tables impossible                   */
/*    - 8 : Répertoire d'accés aux champs de la table impossible       */
/*    - 9 : Allocation impossible                                      */
/*    - 10: fichier index temporaire impossible créer                  */
/*    - 11: fichier index temporaire impossible d'ouvrir I/O           */
/*    - 12: fichier index courant impossible d'ouvrir I/O              */
/*    - 13: fichier donnée courant table impossible d'ouvrir I/O       */
/* ------------------------------------------------------------------- */

extern "C" int Reevaluer_Champ(char *Nom_Champ);

/* ------------------------------------------------------------------- */
/* int Epurer_Champ(char *Mot_De_Passe,char *Nom_Champ);               */
/*   Cette fonction élimine toutes les données d'occurrences nulles    */
/* et remet à jour toutes les tables concernées par ces modifications  */
/*   Cette fonction retourne 0 quand cela a pu être réalisé sinon :    */
/*    - 1 : Pas d'application ouverte                                  */
/*    - 2 : Mot de passe donnée incorrect                              */
/*    - 3 : Nom de champ doit être non vide                            */
/*    - 4 : Nom de champ trop long                                     */
/*    - 5 : Nom de champ mal défini                                    */
/*    - 6 : Nom de champ inconnu                                       */
/*    - 7 : Allocation impossible                                      */
/*    - 8 : Répertoire d'accés projet impossible                       */
/*    - 9 : Répertoire d'accés aux tables impossible                   */
/*    - 10: Répertoire d'accés aux champs de la table impossible       */
/*    - 11: Impossible de créer le fichier de convertion               */
/*    - 12: Fichier index impossible d'ouvrir en lecture/écriture      */
/*    - 13: Fichier de convertion impossible d'ouvrir lecture/écriture */
/*    - 14: Fichier de données du champ à épurer impossible d'ouvrir   */
/*    - 15: fichier index temporaire inouvrable en lecture/ecriture    */
/*    - 16: fichier donnée temporaire inouvrable en lecture/écriture   */
/*    - 17: impossible de créer fichier index temporaire d'une table   */
/*    - 18: impossible de créer fichier données temporaire d'une table */
/*    - 19: impossible ouvrir fichier index temporaire d'une table I/O */
/*    - 20: impossible ouvrir fichier donnée temporaire d'une table I/O */
/*    - 21: impossible ouvrir fichier index courant d'une table I/O    */
/*    - 22: impossible ouvrir fichier données courant d'une table I/O  */
/* ------------------------------------------------------------------- */

extern "C" int Epurer_Champ(char *Mot_De_Passe,char *Nom_Champ);
```

```
/* ---------------------------------------------------------------------- */
/* int Modifier_Champ(char *Mot_De_Passe,char *Nom_Champ,int Commande,  */
/*                         void *Nouveau_Paramétre);                      */
/* Cette fonction apporte les modifications qui seront demanées dans la */
/*      commande avec le nouveau paramétre                                */
/*                                                                        */
/*  Remarque : les données d'occurrence nulle seront alors perdues      */
/*                                                                        */
/* Cette fonction retourne 0 quand pas d'erreurs sinon :                 */
/*    - 1 : Pas d'application ouverte                                     */
/*    - 2 : Mot de passe incorrect                                        */
/*    - 3 : Nom de champ doit être non vide                               */
/*    - 4 : Nom de champ trop long                                        */
/*    - 5 : Nom de champ mal défini                                       */
/*    - 6 : Nom de champ inconnu                                          */
/*    - 7 : Commande inconnue                                             */
/*    - 8 : Nom de champ = Nouveau Nom                                    */
/*    - 9 : Nouveau Nom de champ ne peut pas être vide                    */
/*    - 10: Nouveau Nom de champ trop long                                */
/*    - 11: Nouveau Nom de champ mal défini                               */
/*    - 12: Nouveau Nom de champ déjà existant                            */
/*    - 13: Nouveau domaine de définition ne peut pas être vide         */
/*    - 14: Nouveau domaine de définition mal défini                      */
/*    - 15: Le nom du champ est le nom d'un champ référencé             */
/*    - 16: Modification du domaine non applicable                        */
/*    - 17: Allocation impossible                                         */
/*    - 18: Répertoire d'accés à l'application introuvable              */
/*    - 19: Répertoire des tables introuvable                             */
/*    - 20: Répertoire de la table concernée introuvale                 */
/*    - 21: Fichier de données inexistant                                 */
/*    - 22: Fichier de données impossible d'ouvrir en lecture/écriture  */
/*    - 23: Domaine de définition inapplicable                            */
/*    - 24: Type inconnue                                                 */
/*    - 25: Nouveau type est égal à l'ancien type                         */
/*    - 26: Modification du type incomptabile avec l'existant           */
/*    - 27: Taille_Champ trop grand,il doit etre entre 0 et 78          */
/*    - 28: Le nombre de décimal doit être 0                              */
/*    - 29: Modification de la taille d'une date impossible             */
/*    - 30: Le nombre de décimal d'un réel doit être strictement inf à  */
/*                la taille de ce dernier                                 */
/*    - 31: Le hauteur de la fenêtre du mémo doit être entre 1 et 23    */
/*    - 32: La taille n'a pas subit de modification                       */
/*    - 33: champ temporaire impossible d'ouvrir en lecture/écriture    */
/*    - 34: le fichier index du champ concerne est introuvable          */
/*    - 35: La nouvelle taille ne peut pas être vide                      */
/*    - 36: Perte de données                                              */
/*    - 37: Passage du réel au numérique impossible car Nb_Decimal>0    */
/*    - 38: Taille du mémo trop important pour devenir un caractére     */
/*    - 39: Modification impossible (Caractere -> Numérique) car        */
/*            une donnée n'est pas un numérique                           */
/*    - 40: Caractere -> Numérique , impossible, blanc entre 2 Chiffres */
/*    - 41: Caractére -> Date , impossible longueur champ <> 10         */
/*    - 42: Caractére -> Date , impossible une des valeurs ne peut pas  */
/*            être une date                                               */
/*    - 43: Numérique -> Date , impossible longueur champ <> 8          */
/*    - 44: Numérique -> Date , impossible une des valeurs ne peut pas  */
/*            être une date                                               */
/*    - 45: Taille du champ est nulle,modification de type impossible   */
/*    - 46: Caractére -> Réel , Nb de virgule > 1                         */
/*    - 47: Nouvelle_nature est identique à l'ancienne nature           */
/*    - 48: Changement de nature non réalisé directement                */
```

```
/*    - 51: Répertoire d'accés au dictionnaire impossible               */
/*    - 52: Dictionnaire impossible d'ouvrri en écriture                */
/*    - 53: Le nom complet d'un projet ne peut pas être vide            */
/*    - 54: Fichier de données temporaire impossible de créer           */
/*    - 55: Fichier de données temporaire impossible ouvrir en I/O      */
/*    - 56: Numérique <0 pour date incorrecte                           */
/*    - 57: Clé redondante dans une table                               */
/*    - 58: Un des fichiers clés ou champ n'a pu être ouvert en I/O     */
/*    - 59: La table ne posséde plus de clés                            */
/*    - 60: Fichier de convertion impossible de créer                   */
/*    - 61: Fichier index impossible d'ouvrir en lecture/écriture       */
/*    - 62: Fichier de convertion impossible d'ouvrir lecture/écriture  */
/*    - 63: Fichier de données du champ à épurer impossible d'ouvrir    */
/*    - 64: fichier index temporaire inouvrable en lecture/ecriture     */
/*    - 65: fichier donnée temporaire inouvrable en lecture/écriture    */
/*    - 66: impossible de créer fichier index temporaire d'une table    */
/*    - 67: impossible de créer fichier données temporaire d'une table  */
/*    - 68: impossible ouvrir fichier index temporaire d'une table I/O  */
/*    - 69: impossible ouvrir fichier donnée temporaire d'une table I/O */
/*    - 70: impossible ouvrir fichier index courant d'une table I/O     */
/*    - 71: impossible ouvrir fichier données courant d'une table I/O   */
/*    - 80: Nom de champ référence doit être non vide                   */
/*    - 81: Nom de champ référence trop long                            */
/*    - 82: Nom de champ référence mal défini                           */
/*    - 83: Nom de champ référence inconnu                              */
/*    - 84: Impossible d'apporter l'accés à la référence : Cyclage      */
/*    - 85: Champs incompatibles pour accéder à la référence            */
/*    - 86: une des données n'existe pas dans la référence              */
/*    - 87: Table impliquée dans une requête ouverte                    */
/*    - 88: Il existe des écrans en cours d'exécution                   */
/* ------------------------------------------------------------------- */

extern "C" int Modifier_Champ(char *Mot_De_Passe,char *Nom_Champ,
                                          int Commande,void *Nouveau_Parametre);
```

```
/* ------------------------------------------------------------------- */
/* fonction : int Supprimer_Champ(char *Mot_De_Passe_Structure,        */
/*                                char *Nom_Champ);                    */
/*   Cette fonction procéde à la destruction du champ en question      */
/* Cette fonction retourne 0 quand cette suppression a été réalisée sinon */
/*    - 1 : Un projet n'est pas ouvert                                 */
/*    - 2 : Mot de passe incorrect                                     */
/*    - 3 : Le champ n'existe pas                                      */
/*    - 4 : Non suppression d'une clé unique                           */
/*    - 5 : Non suppression d'un champ référencé                       */
/*    - 6 : Non suppression d'un champ référence                       */
/*    - 7 : Accés au répertoire de l'application impossible            */
/*    - 8 : Accés au répertoire des tables impossible                  */
/*    - 9 : Impossible créer fichier TABLEXX.DTA                       */
/*    - 10: Impossible ouvrir fichier TABLEXX.DTA en I/O               */
/*    - 11: Impossible ouvrir table courante en I/O                    */
/*    - 12: Allocation impossible                                      */
/*    - 13: Accés au répertoire du champ impossible                    */
/*    - 14: Fichier donnée du champ impossible d'ouvrir                */
/*    - 15: Fichier Temporaire des données impossible de créer         */
/*    - 16: Fichier Temporaire des données impossible d'ouvrir en I/O  */
/*    - 17: Fichier Temporaire des index impossible de créer           */
/*    - 18: Fichier Temporaire des index impossible d'ouvrir en I/O    */
/*    - 19: Redondance de la clé                                       */
/*    - 20: Champ utilisé dans un écran                                */
/*    - 21: Table impliquée dans une requête ouverte                   */
/*    - 22: Il existe des écrans en cours d'exécution                  */
/* ------------------------------------------------------------------- */

extern "C" int Supprimer_Champ(char *Mot_De_Passe_Structure,char *Nom_Champ);

/* ------------------------------------------------------------------- */
/* fonction : int Existe_Champ(char *Nom_Champ);                       */
/*    Cette fonction retourne Vrai si le Nom de champ indiqué existe   */
/* dans l'application sinon elle retourne Faux.                        */
/* ------------------------------------------------------------------- */

extern "C" int Existe_Champ(char *Nom_Champ);
```

SYSTEME DE GESTION DE BASES DE DONNEES

MANIPULATION DES DONNEES

Cette partie est la deuxième partie plus importante. Pour vérifier si l'architecture des relations est correcte, elle nécessite sa mise en oeuvre en même temps que la mise en oeuvre de la modification du schéma de la base.

Cette partie se décompose en deux niveaux :

- Le niveau de mise à jour simple des données (pour un champ)
- Le niveau de mise à jour complexe des données (pour les relations)

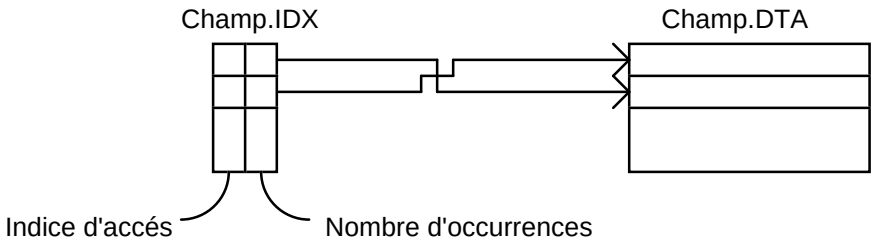
1. Le niveau de mise à jour simple des données (pour un champ)

Ce niveau sera interne à l'application et guère exploitable par l'utilisateur, le deuxième niveau sera celui accessible par tous et celui utilisé pour l'exploitation des données.

Quatre opérations sont demandées à ce niveau :

- Ecriture d'une donnée
- Lecture d'une donnée
- Recherche d'une donnée
- Suppression d'une donnée.

1.1. Rappel de la structure physique d' un champ ou d' une clé



Cette structure a été sélectionnée car le fichier de données sera toujours trié par ordre **strictement croissant** grâce au fichier index. Ce tri permettra avec une grande facilitée l'utilisation de l'algorithme de dichotomie réputé comme étant celui le plus rapide. L'algorithme dichotomique s'appliquera sur le fichier index.

L'indice d'accès est un indice compris entre 0 et la taille du fichier de données divisée par la longueur d'un champ moins un.

1.2. Ecriture d' une donnée

En entrée, la donnée et le fichier suffiront, la donnée devra **impérativement** être formatée car la valeur du champ sauvegardée sera la donnée passée en paramètre.

L'enregistrement d'une donnée s'effectuera en deux temps :

- l'ajout dans le fichier de données,
- l'insertion dans le fichier index (si nécessaire).

1.2.1. Ajout dans le fichier de données

Cet ajout est, en fait, une simple insertion en queue de fichier.

1.2.2. Insertion dans le fichier d'i ndex

On réalisera une copie du fichier d' index dans un fichier index temporaire dans lequel on va réaliser l' insertion de la nouvelle donnée pour éviter lors d' un incident technique la perte partielle ou totale des données.

Cette insertion s' accomplira en deux étapes :

- Recherche de la zone d' insertion,
- Insertion et décalage du reste du fichier (si nécessaire).

1.2.2.1. La recherche de la zone d' insertion

1. Initialisation de l' algorithme

La recherche sera basée sur l' algorithme de dichotomie. On lira la première et dernière valeur du fichier puis on les comparera à la valeur à insérer.

* la première valeur est trop grande,
  la zone d' insertion est en tête du fichier.
* la première valeur est égale,
  il y a simplement l' incrémentation du nombre d' occurrences de la première donnée dans le fichier d' index.
* la dernière valeur est trop petite,
  la zone d' insertion est en queue du fichier.
* la dernière valeur est égale, il y a simplement l' incrémentation du nombre d' occurrence de la dernière donnée dans le fichier d' index.
* Sinon, la donnée est recensée entre la première et la dernière donnée.

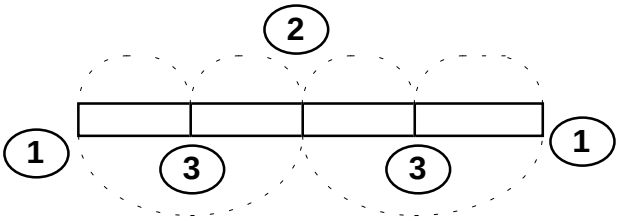
2. Itération

On positionne la marge gauche et la marge droite de la dichotomie aux valeurs de 0 et de ((n / taille_champ) - 1).

On lit la valeur au milieu des deux marges puis on compare, si la valeur lue est égale à la valeur à insérer, l' index de la valeur lue voit son nombre d' occurrences incrémenté de 1. Sinon, si la valeur lue est plus grande, alors la marge droite est positionnée à la valeur du milieu sinon , la marge gauche est positionnée à la valeur du milieu.

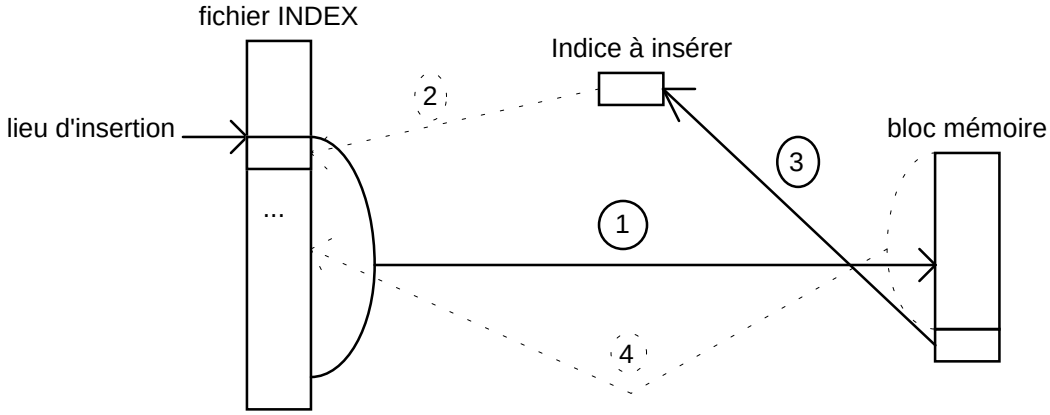
Puis, on recommence jusqu' à ce que l' algorithme ne soit plus applicable, c' est à dire, jusqu' à ce que le milieu soit, soit la marge gauche, soit la marge droite ou soit la donnée à insérer.

3. Schéma



1.2.2.2. Insertion et décalage par bloc des index

Une fois que l' on connaît le lieu d' insertion de la donnée, on lit un bloc que l' on stocke en mémoire, puis on écrit l' index à insérer. L' index à insérer prend la valeur du dernier élément du bloc qui a été lu, puis on écrit le bloc que l' on a stocké en mémoire excepté le dernier indice. On effectue ce cyclage jusqu' à ce que l' on atteigne la fin du fichier.



1.3. Lecture d' une donnée

Pour lire une donnée, on s' intéressera à l' indice de la donnée (indice dans le fichier index) pour déterminer son contenu.

L' indice, demandé ici, est l' indice dans le fichier index. On lit l' index correspondant à l' indice demandé puis on lit la donnée à l' indice de l' index indiquée dans le fichier donnée en multipliant ce dernier par la taille du champ qu' il faudra connaître.

1.4. Recherche d' une donnée

Cette recherche sera la recherche de la zone d' insertion ou la zone d'existence d' une donnée dans le fichier index (ordre strictement croissant). Cette procédure sera exploitée dans le cas de l' écriture de données pour simplifier la démarche de programmation.

Cette recherche se basera sur l' algorithme de dichotomie expliqué ci-dessus.

1.5. Suppression d' une donnée

Il y a deux options de suppression d' une donnée :
- A partir de l' indice dans le fichier index,
- A partir de la donnée, elle-même

1.5.1. A partir de l'indice dans le fichier index.

Cette option lira l' index correspondant à l' indice donné puis, décrémentera le nombre d' occurrences de 1 dans le cas où ce dernier sera strictement positif. Ce traitement est rapide et s' applique au fichier index.

1.5.2. A partir de la donnée, elle-même

Cette option utilisera la recherche d' une donnée qui retournera l' indice de la zone d' insertion ou d'existence dans le fichier index. Si la position retournée correspond à une zone d'existence, nous lirons son index puis nous décrémenterons de un le nombre d' occurrences de la donnée si possible... (nombre d' occurrences déjà existantes strictement positif).

Ce traitement fait dans la première partie que de la lecture et dans la deuxième, la durée sera la même que la suppression d' une donnée à partir de l' indice dans le fichier index.

2. Le niveau de mise à jour complexe des données

Ce niveau sera la partie visible pour l' utilisateur de la base de données. Il constitue les outils de manipulations des données, il réalisera la liaison entre les données physiques à partir du modèle des données logiques.

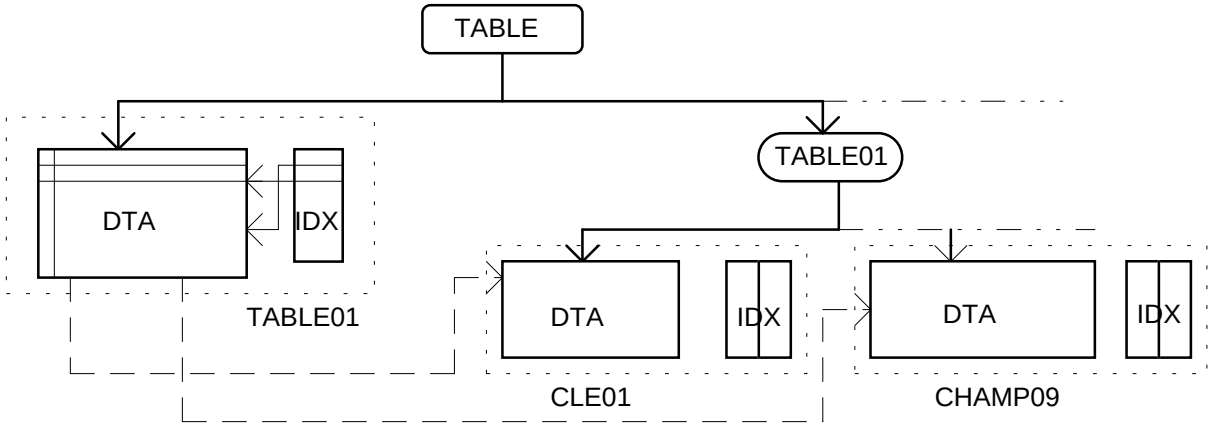
Un mot de passe verrouillera l'accès à ces informations, ce mot de passe pourra être différent du mot de passe pour la modification de la structure des données. Il s'appliquera à l'écriture et la suppression d'enregistrements.

Quatre opérations sont demandées à ce niveau :
- Ecriture d' un enregistrement d' une table
- Lecture d' un enregistrement d' une table
- Recherche d' un enregistrement à partir de la concaténation des clés.
- Suppression d'un enregistrement.

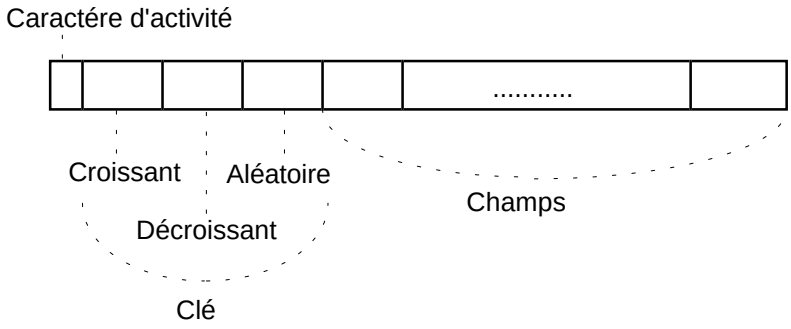
Pour assurer une homogénéité des données qui vont être sauvegardées, deux autres opérations sont à prévoir :
- Affecter une valeur à un champ,
- Visualiser (Récupérer) la dernière valeur de ce champ.

2.1. Rappel de la structure physique d' une table

2.1.1. Organisation physique des liens entre fichiers



2.1.2.Ordre de rangement des champs dans le fichier donnée table



2.2. Affectation d'une valeur à un champ

Cette procédure permettra de transcrire une donnée saisie (format utilisateur) en donnée physique (sauvegarde), en fonction du type du champ, de la taille et du domaine de définition.

Cette procédure aura l' avantage de ne pas compliquer la vie de l' utilisateur quand il désirera sauvegarder un enregistrement. Ceci constituera une interface entre l'utilisateur et la base de données.

TYPES	ENTREES	TRAITEMENTS
Caractère	Chaîne de caractères quelconque	complété par des ' ' à droite
Numérique	+/-[0-9]*	complété par des ' 0' à gauche
Réel	+/-[0-9]* , [0-9]*	Réel sans virgule
Date	JJ/MM/AAAA AAAAMMJJ	si la date est correcte, AAAAMMJJ
Mémo	Chaîne de caractères quelconque	complété par des ' ' à droite

2.3. Visualisation (Récupération) de la valeur d'un champ

Cette procédure permettra de récupérer une valeur affectée à un champ. En fonction d' un code de sortie, l' utilisateur n' aura pas à reformater le champ à sa sortie. Dans le cas où le champ ne serait pas affecté, une valeur N/A sera retournée (pointeur NULL).

TYPES	CODES	SORTIES
Caractère	1	privé des espaces à droite
	2	telle que la valeur est connue
Numérique	1	Avec des ' 0' à gauche (signé)
	2	Sans ' 0' à gauche (signé)
	3	Avec des ' 0' à gauche (non signé)
	4	Sans ' 0' à gauche (non signé)
Réel	1	Avec ' 0' à gauche et droite (signé)
	2	Sans ' 0' à gauche (signé)
	3	Sans ' 0' à gauche et à droite (signé)
	4	Avec ' 0' à gauche et droite (non signé)
	5	Sans ' 0' à gauche (non signé)
	6	Sans ' 0' à gauche et droite (non signé)
Date	1	"JJ/MM/AAAA"
	2	"AAAAMMJJ"
Mémo	1	privé des espaces à droite
	2	telle que la valeur est connue

2.4. Ecriture d'un enregistrement d' une table

Cette écriture se déroulera en quatre étapes :

- Constitution des pointeurs clés de l' enregistrement
- Recherche de la zone d' insertion de l' enregistrement
- Constitution des pointeurs champs de l' enregistrement
- Insertion physique de l' enregistrement

2.4.1. Constitution des pointeurs clés de l' enregistrement

Cette constitution consiste à écrire chaque donnée clé dans le fichier donnée du champ concerné. Les indices d'existence ou d' insertion sont alors mémorisés dans une zone tampon (constitution logique d' un enregistrement physique). Puis, ensuite, chacune des valeurs sera supprimée juste le temps de vérifier que la clé existe ou n' existe pas.

Pour cela, chaque champ concerné doit alors avoir été affecté d' une valeur qui aura été validée par l' affectation.

2.4.2. Recherche de la zone d'i nsertion dans la table

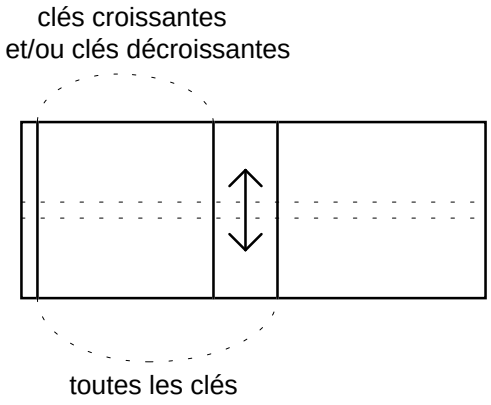
Cette recherche nécessite la connaissance d' une structure simple permettant d' accéder rapidement à chaque champ clé concerné, afin d' éviter d' ouvrir, de fermer des fichiers ou de changer de répertoire (car cela est très coûteux en temps).
Description d' une cellule de la zone tampon :

Indice dans le fichier
Descripteur du fichier
Ordre de rangement
Taille de la donnée

La fin du tableau de cellule tampon sera marquée par le fait que toutes les informations contenues dans cette cellule soient à (-1).

Cette recherche va nécessiter l' utilisation de l' algorithme dichotomique déjà exploité dans l' enregistrement ou la recherche de données. Cet algorithme s' appliquera uniquement sur les clés croissantes et décroissantes.

Une fois la concaténation des clés croissantes et décroissantes trouvées, la recherche consistera à parcourir le fichier de données, d' abord en arrière, puis en suite en avant, tant que les clés croissantes et décroissantes sont égales.



2.4.3. Constitution finale des pointeurs de l' enregistrement

Si l' enregistrement n' existe pas ou n' est pas actif, l' enregistrement des pointeurs sera alors construit et chaque valeur de chaque clé et de chaque champ sera enregistré.

2.4.4. Insertion du nouvel enregistrement

Rappel :

Un enregistrement d' un fichier TABLE peut adapter deux états : l' état actif, enregistrement qui n' a pas été supprimé et l' état inactif, enregistrement toujours répertorié et qui a été supprimé.

Aprés l' application de la procédure de la recherche de la zone d' insertion de l' enregistrement :

Trois possibilités sont envisagées :

- la clé existe et est active, erreur non enregistrement
- la clé existe et est inactive, le nouvel enregistrement écrase le précédent
- la clé n' existe pas, insertion de l' enregistrement en fin de fichier et insertion de son indice dans l' index de la table (Cf. insertion par bloc d' un indice Cf. 1.2.2.2)

2.5. Lecture d' un enregistrement

Il existe deux possibilités de demander à lire un enregistrement :

- En donnant la position dans le fichier index,
- En donnant la clé pour compléter les autres champs.

2.5.1. En donnant la position dans le fichier index

Il faut lire l' indice situé à la position indiquée dans le fichier index, puis lire l' enregistrement concerné (les données numériques). Ensuite, il y aura affectation de tous les champs de la table de la valeur correspondante. Il faudra préciser au retour, si l' enregistrement est actif ou pas (dans la table, Actif ou inactif). Ceci permet d' obtenir une certaine sécurité des données, qui pourront, ainsi, être récupérées sans encombre dans les plus bref délais. Et l' utilisateur devra alors faire la distinction entre un enregistrement actif ou inactif.

2.5.2. En donnant la clé pour obtenir l' enregistrement

Il faudra rechercher par méthode dichotomique (Cf. rechercher zone d' insertion d' un enregistrement 2.4.2.), élément recherché, lire les clés, les convertir en chaînes de caractères (lire la valeur des champs), les comparer et appliquer l' algorithme de la dichotomie en fonction des différentes valeurs.

Une fois trouvée, il faudra affecter chaque clé et/ou chaque champ de leur valeur respective.

2.6. Suppression d' un enregistrement

On pourra supprimer un enregistrement en fonction de deux paramètres, l' indice de position dans le fichier d' index ou en donnant la clé de l' enregistrement à supprimer.

2.6.1. En donnant la position dans le fichier index

Il faudra lire dans le fichier index, l' indice correspondant puis positionner le premier caractère de l' enregistrement concerné à ' X' (inactif) et supprimer chaque valeur de l' enregistrement concerné si l' enregistrement était avant cette opération actif.

2.6.2. En donnant la clé

Pour trouver la clé, on appliquera la méthode dichotomique (Cf. recherche de la zone d' insertion d' un enregistrement 2.4.2.) et dans le cas où l' enregistrement existe, il faudra positionner le premier caractère de l' enregistrement à ' X' (inactif) et supprimer chaque valeur de l'enregistrement concerné si l' enregistrement était avant cette opération actif.

```
/* ********************************************************************** */
/*                Systéme de Gestion de Bases de Données                  */
/*                              Version 1.0                               */
/*                                                                        */
/* Date : 01/06/95                                                        */
/*                                                                        */
/*                                                    par LESERT Aymeric */
/* ********************************************************************** */

/* ********************************************************************** */
/*        unite : permettant l'utilisation d'un fichier de données        */
/*                                                                        */
/* Date : 17/03/94                                                        */
/* ********************************************************************** */

/* ---------------------------------------------------------------------- */
/*  Fonction : int Lecture_Donnees(char *Nom_Fichier,char *Donnee,         */
/*                                 int Taille,int n);                     */
/*  Donnees : Donnee : pointeurs sur une chaine de caractéres qui doit    */
/*                     être préalablement allouée                         */
/*            Nom_Fichier : Nom du fichier champ où il faut lire la donnee */
/*            Taille : Taille d'un enregistrement                         */
/*            n : accés au n iéme enregistrement (ordre croissant)        */
/*  Retour  :  m : pas d'erreurs (Nombre d'occurrences de la données)     */
/*            -1 : l'enregistrement n'existe pas                          */
/*            -2 : le fichier index n'existe pas                          */
/*            -3 : le fichier donnée n'existe pas                         */
/*            -4 : Allocation impossible                                  */
/*            -5 : la taille doit être >0                                 */
/* ---------------------------------------------------------------------- */

extern "C" int Lecture_Donnees(char *Nom_Fichier,char *Donnees,
                                                        int Taille,int n);

/* ---------------------------------------------------------------------- */
/* fonction : int Recherche_Donnees(char *Nom_Fichier,char *Donnees,      */
/*                                  int Taille,int *Position);            */
/* Donnees : Position : au retour contiendra le rang où se situe          */
/*                      l'élèment recherché si le retour est positif ou nul */
/*           Nom_Fichier : Nom du fichier champ où il faut chercher       */
/*           Donnees : Valeur de la donnée dont on désire connaître       */
/*                     l'existance.                                       */
/*           Taille : Taille du champ à rechercher                        */
/* Retour  :  n : nombre d'occurrences (position = zone d'existance)      */
/*           -1 : le fichier d'index n'existe pas                         */
/*           -2 : le fichier table n'existe pas                           */
/*           -3 : la donnée n'existe pas (position=zone d'insertion)      */
/*           -4 : Allocation impossible                                   */
/*           -5 : Taille doit être >0                                     */
/* ---------------------------------------------------------------------- */

extern "C" int Recherche_Donnees(char *Nom_Fichier,char *Donnees,int Taille,
                                                            int *Position);
```

```
/* ------------------------------------------------------------------------ */
/* fonction : int Ecriture_Donnees(char *Nom_Fichier,char *Donnee,          */
/*                                                 int Taille);             */
/* Donnees : Nom_Fichier : Nom du fichier champ où il faut écrire la        */
/*                         donnee.                                          */
/*           Donnee : Valeur à écrire dans le champ                         */
/*           Taille : Taille de la valeur à écrire dans le champ            */
/* Retour :  0 : pas d'erreurs                                              */
/*       -1 : Le fichier d'index n'existe pas                               */
/*       -2 : le fichier de données n'existe pas                            */
/*       -3 : Allocation impossible                                         */
/*       -4 : Taille doit être >0                                           */
/* ------------------------------------------------------------------------ */

extern ″C″ int Ecriture_Donnees(char *Nom_Fichier,char *Donnees,int Taille,
                                                              int *Position);

/* ------------------------------------------------------------------------ */
/* fonction : int Supprime_Donnees(char *Nom_Fichier,char *Donnees,         */
/*                                 int Taille);                             */
/* Remarque : Décrémente le nombre d'occurrences du champ recherché         */
/* Retour : -1 : le fichier IDX n'existe pas                                */
/*          -2 : le fichier DTA n'existe pas                                */
/*          -3 : la donnée n'existe pas dans le fichier                     */
/*          -4 : la donnée existe mais l'occurrence est nulle               */
/*          -5 : Allocation impossible                                      */
/*          -6 : Taille doit être >0                                        */
/*          n : Nombre d'occurrences restantes                              */
/* ------------------------------------------------------------------------ */

extern ″C″ int Supprime_Donnees(char *Nom_Fichier,char *Donnees,int Taille);

/* ------------------------------------------------------------------------ */
/* fonction : int Supprime_Occurrence(char *Nom_Fichier,int n);             */
/* Remarque : Décrémente le nombre d'occurrences du n iéme champ (idx)      */
/* Retour : -1 : le fichier IDX n'existe pas                                */
/*          -2 : la donnée n'existe pas dans le fichier                     */
/*          -3 : la donnée existe mais l'occurrence est nulle               */
/*          -4 : Allocation impossible                                      */
/*          n : Nombre d'occurrences restantes                              */
/* ------------------------------------------------------------------------ */

extern ″C″ int Supprime_Occurrence(char *Nom_Fichier,int n);

/* ------------------------------------------------------------------------ */
/* fonction : int Nombre_Donnees(char *Nom_Fichier);                        */
/*   Cette fonction retourne le nombre de données recensée y compris        */
/* les données d'occurrences nulles sinon :                                 */
/*    - -1 : Fichier impossible d'ouvrir en lecture/écriture                */
/* ------------------------------------------------------------------------ */

extern ″C″ int Nombre_Donnees(char *Nom_Fichier);
```

```
/* ********************************************************************** */
/*                  Module de gestion des données complexes               */
/*                                                                        */
/* Date : 07/08/94                                                        */
/* ********************************************************************** */

/* ********************************************************************** */
/*       Partie concernant la mise à jour des valeurs d'un champ          */
/* ********************************************************************** */

/* ---------------------------------------------------------------------- */
/* int Affectation_Champ(char *Nom_Champ,char *Valeur);                    */
/* Entrées : Nom_Champ : Nom du champ devant affecté la valeur            */
/*           Valeur : Valeur à affecter au champ                          */
/* Sorties :                                                              */
/*   Cette fonction a pour but de mémoriser une valeur pour un champ      */
/* de l'application ouverte.                                              */
/* Cette fonction retourne 0 quand l'affectation a pu avoir lieu sinon    */
/*    - 1 : Pas de projet ouvert                                          */
/*    - 2 : Nom de champ inconnu                                          */
/*    - 3 : Allocation impossible                                         */
/*    - 4 : Domaine de définition incorrect                               */
/*    - 5 : Valeur ne correspondant pas au domaine de définition          */
/*    - 6 : Longueur de la valeur > Longueur du champ                     */
/*    - 7 : fichier index de la référence inexistant                      */
/*    - 8 : fichier donnée de la référence inexistant                     */
/*    - 9 : Affectation d'une valeur référencé n'existant pas             */
/*    - 10: Répertoire du projet inaccessible                             */
/*    - 11: Répertoire des tables inaccessible                            */
/*    - 12: Répertoire de la table du champ inaccessible                  */
/*    - 13: trop de virgules pour un réel                                 */
/*    - 14: Date incorrecte                                               */
/* ---------------------------------------------------------------------- */

extern "C" int Affectation_Champ(char *Nom_Champ,char *Valeur);

/* ---------------------------------------------------------------------- */
/* fonction : char *Connaitre_Valeur(char *Nom_Champ,int Code,            */
/*                                   int *Erreur);                        */
/*   Cette fonction a pour but de connaître la valeur qui a été affectée  */
/* au champ.                                                              */
/*   Les codes autorisés en fonction du type du champ sont :              */
/*   +-----------+----------------------------------------+               */
/*   | Caractére | Code=1 Privé des espaces à droite      |               */
/*   |           | Code=2 Telle que                       |               */
/*   +-----------+----------------------------------------+               */
/*   | Numérique | Code=1 Avec des '0' à gauche '+' / '-'|               */
/*   |           | Code 2 Sans '0' à gauche     '+' / '-'|               */
/*   |           | Code 3 Avec '0' à gauche           '-'|               */
/*   |           | Code 4 Sans '0' à gauche           '-'|               */
/*   +-----------+----------------------------------------+               */
```

```
/*   +-----------+---------------------------------------+              */
/*   |    Réel   | Code=1 Avec '0' à gauche et à droite  |              */
/*   |           | Code=2 Avec '0' à droite              |              */
/*   |           | Code=3 Sans '0' à gauche et à droite  |              */
/*   |           |          '+' / '-'                    |              */
/*   |           | Code=4 Avec '0' à droite et à droite  |              */
/*   |           | Code=5 Avec '0' à gauche              |              */
/*   |           | Code=6 Sans '0' à gauche et à droite  |              */
/*   |           |               '-'                     |              */
/*   +-----------+---------------------------------------+              */
/*   |    Date   | Code=1 : "JJ/MM/AAAA"                 |              */
/*   |           | Code=2 : "AAAAMMJJ"                   |              */
/*   +-----------+---------------------------------------+              */
/*   |    Mémo   | Code=1 Sans espaces à droite          |              */
/*   |           | Code=2 Telle que                      |              */
/*   +-----------+---------------------------------------+              */
/*   Cette fonction retourne un pointeur sur une chaine de caractéres    */
/* contenant la valeur au format demandé. Si un pointeur null a été     */
/* retourné et que l'Erreur est égale à 0, cela signifie N/A            */
/* (pas de valeurs) sinon si null est retourné, erreur contient la      */
/* raison de l'échec :                                                  */
/*    - 1 : Projet non ouvert                                           */
/*    - 2 : Nom de champ inconnu                                        */
/*    - 3 : Code inconnu                                                */
/*    - 4 : Allocation impossible                                       */
/* -------------------------------------------------------------------- */

extern "C" char *Connaitre_Valeur(char *Nom_Champ,int Code,int *Erreur);

/* -------------------------------------------------------------------- */
/* fonction : int Liberation_Valeur_Champ();                            */
/*   Cette fonctin a pour but de liberer tous les espaces alloués par les */
/* champs pour conserver les valeurs                                    */
/*  Cette fonction retourne 0 quand pas d'erreurs sinon :               */
/*    - 1 : Pas de projet ouvert                                        */
/* -------------------------------------------------------------------- */

extern "C" int Liberation_Valeur_Champ();

/* ******************************************************************** */
/*     Partie concernant la sauvegarde et le chargement des valeurs     */
/* ******************************************************************** */
```

```
/* ------------------------------------------------------------------- */
/* fonction : int Ecriture_Enregistrement(char *Mot_De_Passe,          */
/*                                        char *Nom_Table);            */
/*    Cette fonction permet de sauvegarder un enregistrement à sa place */
/* dans le bon fichier.                                                */
/*    Quand une erreur survient, il n'est pas garanti que les fichiers  */
/* ne sont pas à jour surtout si ce sont des erreurs physiques.        */
/*    Cette fonction retourne 0 quand pas d'erreurs sinon :            */
/*      - 1 : Pas de projet ouvert                                     */
/*      - 2 : Mot de passe des données incorrect                       */
/*      - 3 : Nom de la table inconnu                                  */
/*      - 4 : Une des cles ne posséde pas de valeurs                   */
/*      - 5 : Un des champs ne posséde pas de valeurs                  */
/*      - 6 : Allocation impossible                                    */
/*      - 7 : Accés Répertoire Projet impossible                       */
/*      - 8 : Accés Répertoire Table impossible                        */
/*      - 9 : Accés au réertoire de la table du champ impossible       */
/*      - 10: fichier index d'un champ inexistant                      */
/*      - 11: fichier données d'un champ inexistant                    */
/*      - 12: fichier index de la table inexistant                     */
/*      - 13: fichier donnée de la table inexistant                    */
/*      - 14: Enregistrement dont la clé est déjà recensée             */
/*      - 15: Pas de clés pour l'enregistrement                        */
/* ------------------------------------------------------------------- */

extern "C" int Ecriture_Enregistrement(char *Mot_De_Passe,char *Nom_Table);

/* ------------------------------------------------------------------- */
/* fonction : int Lecture_Enregistrement(char *Nom_Table,int n);       */
/*   Cette fonction a pour but de pouvoir accéder pour la n iéme       */
/* valeur des enregistrements de la table indiquée dans l'index [0..Max]*/
/*   Cette fonction retourne 0 et tous les champs sont affectés de leur */
/* valeur respectivement quand tout s'est bien passé sinon :           */
/*    - 1 : Pas de projet ouvert                                       */
/*    - 2 : la table indiquée n'existe pas                             */
/*    - 3 : Le répertoire d'accés au projet est introuvable            */
/*    - 4 : le répertoire d'accés àux tables est introuvable           */
/*    - 5 : le fichier index de la table impossible ouvrir en I/O      */
/*    - 6 : enregistrement à la n iéme position inconnue               */
/*    - 7 : le fichier donnée de la table impossible ouvrir en I/O     */
/*    - 8 : Allocation impossible                                      */
/*    - 9 : Répertoire d'accés à la table d'une clé impossible         */
/* ------------------------------------------------------------------- */

extern "C" int Lecture_Enregistrement(char *Nom_Table,int n);
```

```
/* ------------------------------------------------------------------------ */
/* fonction : int Recherche_Enregistrement(char *Nom_Table,int *Pos);       */
/*   Cette fonction a pour but de retrouver l'enregistrement correspondant */
/* à la clé de la table indiquée.                                           */
/*   Si la clé n'existe pas Position contiendra la position d'insertion     */
/*        dans l'index de la table.                                         */
/*   Si la clé existe, Position contiendra la position de l'enregistrement */
/*        dans l'index de la table.                                         */
/*  Cette fonction retourne :                                               */
/*    - 0 : l'enregistrement existe et actif                                */
/*    - 1 : l'enregistrement existe et inactif                              */
/*    - 2 : l'enregistrement n'existe pas                                   */
/*    - 3 : Pas de projet ouvert                                            */
/*    - 4 : Nom de la table inconnu                                         */
/*    - 5 : Une des cles ne posséde pas de valeurs                          */
/*    - 6 : Accés Répertoire Projet impossible                              */
/*    - 7 : Accés Répertoire Table impossible                               */
/*    - 8 : Allocation impossible                                           */
/*    - 9 : Accés au répertoire de la table du champ impossible             */
/*    - 10: fichier index d'un champ inexistant                             */
/*    - 11: fichier données d'un champ inexistant                           */
/*    - 12: fichier index de la table inexistant                            */
/*    - 13: fichier donnée de la table inexistant                           */
/* ------------------------------------------------------------------------ */
extern "C" int Recherche_Enregistrement(char *Nom_Table,int *Pos);

/* ------------------------------------------------------------------------ */
/* fonction : int Suppression_Enregistrement(char *Mot_De_Passe,        */
/*                                           int Code,char *Nom_Table,  */
/*                                           int Position);             */
/*   Cette fonction supprime un enregistrement de la table indiquée     */
/*   Mot_De_Passe : Mot de passe des données                            */
/* Si Code = 0 , Cela signifie que l'on doit supprimer l'enregistrement */
/*               par sa position dans le fichier INDEX                  */
/* Si Code = 1 , Cela signifie que l'on doit supprimer l'enregistrement */
/*               par sa clé                                             */
/*   Cette fonction retourne 0 quand la suppression a eu lieu sinon :   */
/*    - 1 : Code incorrect                                              */
/*    - 2 : Projet non ouvert                                           */
/*    - 3 : Mot de passe incorrect                                      */
/*    - 4 : la table indiquée n'existe pas                              */
/*    - 5 : Le répertoire d'accés au projet est introuvable             */
/*    - 6 : le répertoire d'accés àux tables est introuvable            */
/*    - 7 : le fichier index de la table impossible ouvrir en I/O       */
/*    - 8 : enregistrement à la n iéme position inconnue                */
/*    - 9 : le fichier donnée de la table impossible ouvrir en I/O      */
/*    - 10: Allocation impossible                                       */
/*    - 11: Répertoire d'accés à la table d'une clé impossible          */
/*    - 12: Enregistrement déjà supprimé                                */
/*    - 13: Enregistrement dont la clé de recherche est inconnue        */
/*    - 14: une des clé ne posséde pas de valeurs                       */
/*    - 15: fichier index d'un champ inexistant                         */
/*    - 16: fichier données d'un champ inexistant                       */
/*    - 17: fichier index de la table inexistant                        */
/*    - 18: fichier donnée de la table inexistant                       */
/* ------------------------------------------------------------------------ */

extern "C" int Suppression_Enregistrement(char *Mot_De_Passe,int Code,
                                          char *Nom_Table,int Position);
```

```
/* ------------------------------------------------------------- */
/* fonction : int Nombre_Enregistrement(char *Nom_Table);        */
/*   Cette fonction retournera le nombre d'enregistrements actifs ou */
/* inactifs contenus dans le fichier donnée de la table.         */
/*   Cette fonction retourne donc une valeur positive ou nulle sinon */
/*      - -1 : Pas de projet ouvert                              */
/*      - -2 : Pas de tables recensée                            */
/*      - -3 : La table n'existe pas                             */
/*      - -4 : Accés Repertoire de l'application impossible      */
/*      - -5 : Accés Répertoire des tables impossible            */
/*      - -6 : Fichier index de la table impossible d'ouvrir     */
/* ------------------------------------------------------------- */

extern "C" int Nombre_Enregistrement(char *Nom_Table);

/* ------------------------------------------------------------- */
/* fonction : int Nombre_Enregistrement_Actif(char *Nom_Table);  */
/*   Cette fonction retournera le nombre d'enregistrements actifs */
/* contenus dans le fichier donnée de la table.                  */
/*   Cette fonction retourne donc une valeur positive ou nulle sinon */
/*      - -1 : Pas de projet ouvert                              */
/*      - -2 : Pas de tables recensée                            */
/*      - -3 : La table n'existe pas                             */
/*      - -4 : Accés Repertoire de l'application impossible      */
/*      - -5 : Accés Répertoire des tables impossible            */
/*      - -6 : Fichier index de la table impossible d'ouvrir     */
/* ------------------------------------------------------------- */

extern "C" int Nombre_Enregistrement_Actif(char *Nom_Table);

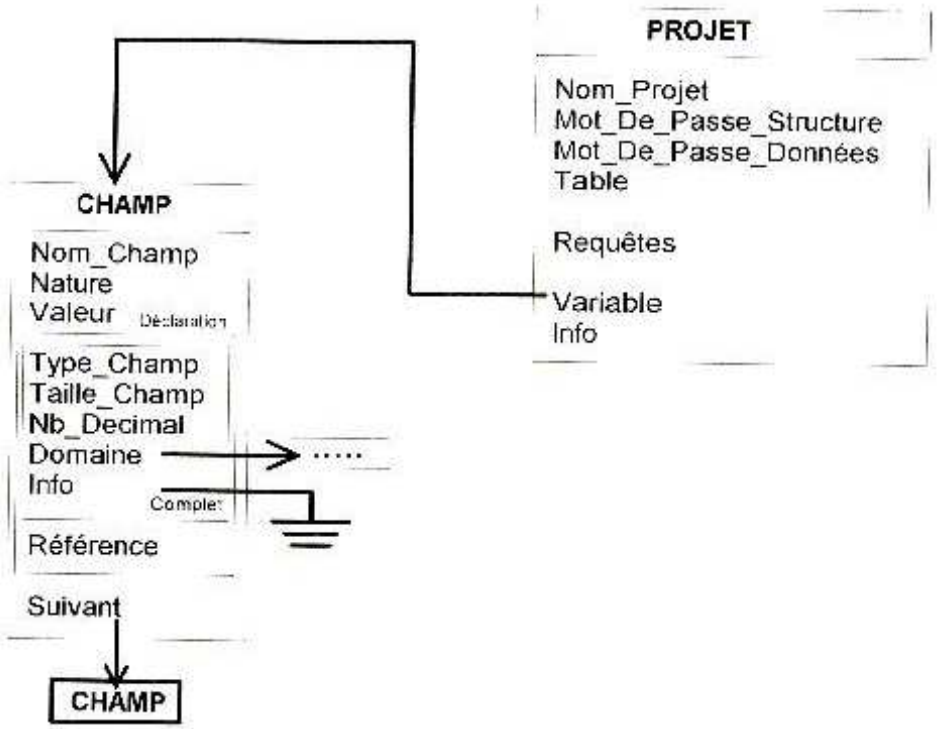
/* ------------------------------------------------------------------------ */
/* fonction : int Enregistrement_Actif(char *Table);                        */
/*   Cette fonction retourne Vrai si l'enregistrement courant de la table  */
/* est Actif sinon Faux.                                                    */
/*   Remarque : il est prudent de réaliser ce traitement aprés une lecture */
/* ------------------------------------------------------------------------ */

extern "C" int Enregistrement_Actif(char *Nom_Table);

/* ------------------------------------------------------------- */
/* fonction : int Nombre_Donnees_Champ(char *Nom_Champ);         */
/*   Cette fonction retournera le nombre de données différentes */
/* contenues dans le fichier de données du champ concerné sinon */
/*      - -1 : Pas de projet ouvert                              */
/*      - -2 : Pas de tables recensée                            */
/*      - -3 : La table n'existe pas                             */
/*      - -4 : Accés Repertoire de l'application impossible      */
/*      - -5 : Accés Répertoire des tables impossible            */
/*      - -6 : Fichier index de la table impossible d'ouvrir     */
/*      - -7 : Accés Répertoire du champ impossible              */
/* ------------------------------------------------------------- */

extern "C" int Nombre_Donnees_Champ(char *Nom_Champ);
```

DICTIONNAIRE DE LA BASE



SYSTEME DE GESTION DE BASES DE DONNEES

GESTION DE VARIABLES

Il est intéressant de pouvoir avoir accès à un plus large éventail d' utilisation du système en manipulant des variables. Les variables permettront de stocker des valeurs temporairement sans affecter le schéma de l' aplication lors de leur manipulation. Les effets les plus importants seront le fait de pouvoir rendre une requête dynamique.

1. Définition d' une variable

Une variable est un champ qui ne possède pas de rattachement à une table et qui ne possède pas d' entité physique. Elle se présente uniquement sous forme logique.

2. Les opérations réalisables

Une variable sera globale à l' application en cours.
Les opérations de bases seront :

- Création d' une variable
- Suppression d' une variable
- Affectation d' une valeur à une variable
- Récupération de la valeur affectée à une variable
- Affectation entre variables et champs de l' application
- Modification des paramètres d'une variable
  (Attention !! Redéfinition complète)

2.1. Création d' une variable globale

Cette opération consiste à mettre en place une structure logique correspondant à une variable. Cette constitution ne nécessite pas de traitements physiques mis à part une mise à jour du dictionnaire. Cette création nécessite la vérification de la non-existence de cette variable.

2.2. Suppression d' une variable

Cette opération libère, logiquement, la mémoire de la structure qu' elle constitue. Elle disparaîtra définitivement de l' application puisque cette variable disparaît du dictionnaire.

2.3. Modification des paramètres d'une variable

Cette opération consiste à modifier tous les paramètres d'une variable existante dans la base. Ce suivra une mise à jour du dictionnaire. Cette opération pourra permettre l'emploi d'une variable à plusieurs fins (Exemple : variable pouvant servir de transfert entre deux tables de la base de données).

2.4. Affectation d' une valeur au champ

Cf. 2.2. *Affectation d'une valeur à un champ*
(Manipulation des données)

2.5. Récupération de la valeur affectée à une variable

Cf. 2.3. *Visualisation de la valeur d'un champ*
(Manipulation des données)

2.6. Affectation entre variables et champs de l' application

Cette opération a été réalisée pour permettre une affectation directe entre deux champs, deux variables, un champ et une variable ou une variable et un champ. Cette affectation nécessite une **parfaite** compatibilité entre les deux éléments, c' est à dire : même taille, même type, même Nombre de décimal.

Pour différencier le nom d'une variable et le nom d' un champ, le nom de la variable devra être précédée par ' &' .

3. Présentation dans le dictionnaire

VARIABLE : < nom de la variable > : < type > : < Taille > :
< Nb_Dec > : < domaine >

où :

Nom de la variable	: 32 caractères maxi
Type	: 1 caractère
Taille	: 99
Nb_Dec	: 99
Domaine	: fonction du type

```
/* ********************************************************************** */
/*                Systéme de Gestion de Bases de Données                  */
/*                              Version 1.0                               */
/*                                                                        */
/* Date : 01/06/95                                                        */
/*                                                                        */
/*                                                    par LESERT Aymeric */
/* ********************************************************************** */

/* ********************************************************************** */
/*             unité permettant la gestion des variables globales         */
/*                                                                        */
/* Date : 22/08/94                                                        */
/* ********************************************************************** */

/* ---------------------------------------------------------------------- */
/* fonction : int Affecter_Variable(char *Nom_Variable,char *Valeur);   */
/*    Cette fonction a pour but d'affecter une valeur à une variable.   */
/*    Cette fonction retourne 0 qund la valeur a pu être affectée sinon */
/*      - 1 : pas de projet ouvert                                        */
/*      - 2 : pas de variables globales                                   */
/*      - 3 : Nom de variable inconnu                                     */
/*      - 4 : Allocation impossible                                       */
/*      - 5 : Domaine de définition non conforme                          */
/*      - 6 : Domaine de définition non respectée                         */
/*      - 7 : Taille de la valeur à affecter trop grande                  */
/*      - 8 : Nombre de virgule > 1                                       */
/*      - 9 : Date invalide                                               */
/* ---------------------------------------------------------------------- */

extern "C" int Affecter_Variable(char *Nom_Variable,char *Valeur);

/* ---------------------------------------------------------------------- */
/* fonction : char *Connaître_Variable(char *Nom_Variable,int Code,    */
/*                                     int *Erreur);                      */
/*    Cette fonction retourne une chaîne de caractéres correspondant à */
/* l'affectation du champ en fonction du Code.                            */
/*    Cette fonction retournera NULL si il y a une erreur :               */
/*      - 0 : N/A Valeur                                                  */
/*      - 1 : pas de projet ouvert                                        */
/*      - 2 : pas de variables globales                                   */
/*      - 3 : Nom de variable inconnu                                     */
/*      - 4 : Allocation impossible                                       */
/*      - 5 : Code inconnu                                                */
/* ---------------------------------------------------------------------- */

extern "C" char *Connaitre_Variable(char *Nom_Variable,int Code,int *Erreur);
```

```
/* ------------------------------------------------------------------- */
/* fonction : int Creer_Variable(char *Mot_De_Passe,char *Nom_Variable,  */
/*                               short Type_Variable,short Taille_Champ, */
/*                               short Nb_Decimal,char *Domaine);        */
/*    Cette fonction crée une variable globale avec les paramétres       */
/* indiqués.                                                             */
/*    Cette fonction retourne 0 quand pas d'erreur sinon :               */
/*      - 1 : pas de projet ouvert                                       */
/*      - 3 : Nom de variable inconnu                                    */
/*      - 4 : Allocation impossible                                      */
/*      - 5 : Type de variable inconnu                                   */
/*      - 6 : Caractére : Taille Champ trop grand (0<x<79)               */
/*      - 7 : Caractére : Nb_Decimal non nul                             */
/*      - 8 : Numérique : Taille Champ trop grand (0<x<79)               */
/*      - 9 : Numérique : Nb_Decimal non nul                             */
/*      - 10: Date : Taille_Champ != 8                                   */
/*      - 11: Date : Nb_Decimal non nul                                  */
/*      - 12: Mémo : Taille Champ trop grand (0<x<79)                    */
/*      - 13: Mémo : Nb_Decimal trop grand (0<y<24)                      */
/*      - 14: Réel : Taille Champ trop grand (0<x<79)                    */
/*      - 15: Réel : Nb_Decimal > Taille_Champ                           */
/*      - 16: Domaine de définition non défini                           */
/*      - 17: Domaine de définition non conforme                         */
/*      - 18: Taille du nom de la nouvelle variable = 0                  */
/*      - 19: Longueur du nom trop long                                  */
/*      - 20: Nom non conforme au domaine de définition des noms de var  */
/*      - 21: Mot de passe incorrect                                     */
/*      - 22: Il existe des écrans en cours d'exécution                  */
/* ------------------------------------------------------------------- */
extern "C" int Creer_Variable(char *Mot_De_Passe,char *Nom_Variable,
                              short Type_Variable,short Taille_Champ,
                              short Nb_Decimal,char *Domaine);

/* ------------------------------------------------------------------------ */
/* fonction:int Supprimer_Variable(char *Mot_De_Passe,char *Nom_Variable); */
/*   Cette fonction détruira la variable globale indiquée.                  */
/*   Cette fonction retournera 0 quand pas d'erreurs sinon :                */
/*      - 1 : pas de projet ouvert                                          */
/*      - 2 : pas de variables globales                                     */
/*      - 3 : Nom de variable inconnu                                       */
/*      - 4 : Mot de passe incorrecte                                       */
/*      - 5 : Variable utilisée dans un écran                               */
/*      - 6 : Il existe des écrans en cours d'exécution                     */
/* ------------------------------------------------------------------------ */
extern "C" int Supprimer_Variable(char *Mot_De_Passe,char *Nom_Variable);

/* ------------------------------------------------------ */
/* fonction : int Nettoyer_Variable(char *Nom_Variable);  */
/*   Cette fonction efface la valeur de la variable.      */
/*   Cette fonction retourne 0 quand pas d'erreurs sinon : */
/*    - 1 : Pas de projet ouvert                          */
/*    - 2 : Pas de variables globales                     */
/*    - 3 : la variable n'existe pas                      */
/*    - 4 : Il existe des écrans en cours d'exécution     */
/* ------------------------------------------------------ */

extern "C" int Nettoyer_Variable(char *Nom_Variable);
```

```
/* ------------------------------------------------------------ */
/* fonction : int Existe_Variable(char *Nom_Variable);          */
/*   Cette fonction retourne Vrai si la variable indiquée existe */
/* sinon Faux.                                                  */
/* ------------------------------------------------------------ */

extern "C" int Existe_Variable(char *Nom_Variable);

/* ------------------------------------------------------------------------ */
/* fonction : int Affecter_Champ_Variable(char *Destination,char *Source); */
/*    Cette fonction permettra d'affecter la valeur d'une variable à un    */
/* champ , d'un champ à une variable, d'un champ à un champ ou vice d'une  */
/* variable à une variable.                                                */
/*    Un nom de variable devra être précédé par un '&'.                    */
/*    Cette fonction retourne 0 quand cette affectation a pu avoir lieu    */
/* sinon :                                                                 */
/*      - 1 : Pas de projet ouvert                                         */
/*      - 2 : Pas de variables dans le projet                              */
/*      - 3 : Nom de la variable destination inconnu                       */
/*      - 4 : Pas de champs dans le projet                                 */
/*      - 5 : Nom du champ destination inconnu                             */
/*      - 6 : Nom de la variable source inconnu                            */
/*      - 7 : Nom du champ source inconnu                                  */
/*      - 8 : Type des deux champs incompatibles                           */
/*      - 9 : Taille des deux champs différente                            */
/*      - 10: Nb_Decimal des deux champs différent                         */
/*      - 11: Valeur source non conforme au domaine de définition dest     */
/*      - 12: Allocation impossible                                        */
/*      - 13: Accés Répertoire de l'application impossible                 */
/*      - 14: Accés Répertoire des tables impossible                       */
/*      - 15: Accés au Répertoire de la table concernée impossible         */
/*      - 16: Fichier index du champ impossible d'ouvrir                   */
/*      - 17: Fichier donnée du champ impossible d'ouvrir                  */
/*      - 18: Donnée Source inexistante chez les données Destinations      */
/* ------------------------------------------------------------------------ */

extern "C" int Affecter_Champ_Variable(char *Destination,char *Source);
```

SYSTEME DE GESTION DE BASES DE DONNEES

OUTILS MATHEMATIQUES

Ces outils mathématiques sont particuliers par le fait même du stockage des données numériques ou réelles dans la base (sous forme de chaînes de caractères).

Ces outils offriront le moyen de réaliser des calculs précis (précision limitée à la taille mémoire). Les calculs offriront donc des limites autres que celles connues en programmation par les entiers et les réels. Cependant, les traitements sont quelques peu lents.

Remarque : Il est donc conseillé aux utilisateurs d' optimiser leur calcul.

Cette partie ne présente pas de traitements particuliers à détailler. Elle est constituée d' opérations élémentaires telles que :

- L' addition;
- La soustraction,
- La multiplication,
- La division réelle,
- La division entière,
- Modulo (reste de la division entière).

Il est important de signaler que cette partie n' est pas le point fort du système de gestions de bases de données car ces routines sont très peu optimisées.

```
/* ************************************************************************ */
/*                Systéme de Gestion de Bases de Données                    */
/*                              Version 1.0                                 */
/*                                                                          */
/* Date : 01/06/95                                                          */
/*                                                                          */
/*                                                        par LESERT Aymeric */
/* ************************************************************************ */

/* ************************************************************************ */
/*       Unité de Gestion mathématique des chaines de caractéres            */
/*                                                                          */
/* Date : 09/09/94                                                          */
/* ************************************************************************ */

/* ------------------------------------------------------------------------ */
/* fonction : int Chaine_Reelle(char *Chaine);                              */
/*   Cette fonction indique si la chaîne de caractéres en entrée est bien */
/* la chaîne de caractéres d'un réel. Elle retourne Vrai ou Faux.           */
/* ------------------------------------------------------------------------ */

extern "C" int Chaine_Reelle(char *Chaine);

/* ------------------------------------------------------------------------ */
/* fonction : char *Partie_Entiere(char *Donnee);                           */
/*   Cette fonction retourne un pointeur sur une zone spécialement allouée */
/* contenant la partie entiére du réel passé en paramétre.                  */
/*   Cette fonction retourne un pointeur NULL si l'allocation n'a pas pu   */
/* se faire.                                                                */
/* ------------------------------------------------------------------------ */

extern "C" char *Partie_Entiere(char *Donnee);

/* ------------------------------------------------------------------------ */
/* fonction : char *Partie_Fractionnelle(char *Donnee);                     */
/*   Cette fonction retourne un pointeur sur une zone spécialement allouée */
/* contenant la partie fractionnelle sans la virgule du réel passé en      */
/* paramétre.                                                               */
/*   Cette fonction retourne un pointeur NULL si l'allocation n'a pas pu   */
/* se faire.                                                                */
/* ------------------------------------------------------------------------ */

extern "C" char *Partie_Fractionnelle(char *Donnee);

/* ------------------------------------------------------------ */
/* fonction : int Addition(char *Resultat,char *Donnee,         */
/*                                      int Taille_Resultat);   */
/*   Cette fonction réalise l'addition de deux réels dont :     */
/*       Resultat = Resultat + Donnee.                          */
/*   Cette fonction retourne 0 quand pas d'erreurs sinon :      */
/*    - 1 : Allocation impossible                               */
/*    - 2 : Résultat n'est pas un réel                          */
/*    - 3 : Donnée n'est pas un réel                            */
/*    - 4 : Taille_Resultat est trop petit                      */
/* ------------------------------------------------------------ */

extern "C" int Addition(char *Resultat,char *Donnee,int Taille_Resultat);
```

```
/* ------------------------------------------------------------ */
/* fonction : int Soustraction(char *Resultat,char *Donnee,     */
/*                                     int Taille_Resultat); */
/*   Cette fonction réalise la soustraction de deux réels dont :*/
/*       Resultat = Resultat - Donnee.                          */
/*   Cette fonction retourne 0 quand pas d'erreurs sinon :      */
/*    - 1 : Allocation impossible                               */
/*    - 2 : Résultat n'est pas un réel                          */
/*    - 3 : Donnée n'est pas un réel                            */
/*    - 4 : Taille_Resultat est trop petit                      */
/* ------------------------------------------------------------ */

extern "C" int Soustraction(char *Resultat,char *Donnee,int Taille_Resultat);

/* ------------------------------------------------------------ */
/* fonction : int Multiplication(char *Resultat,char *Donnee,   */
/*                                     int Taille_Resultat); */
/*   Cette fonction réalise la multiplication de deux réels dont*/
/*       Resultat = Resultat * Donnee.                          */
/*   Cette fonction retourne 0 quand pas d'erreurs sinon :      */
/*    - 1 : Allocation impossible                               */
/*    - 2 : Résultat n'est pas un réel                          */
/*    - 3 : Donnée n'est pas un réel                            */
/*    - 4 : Taille_Resultat est trop petit                      */
/* ------------------------------------------------------------ */

extern "C" int Multiplication(char *Resultat,char *Donnee,int Taille_Resultat);

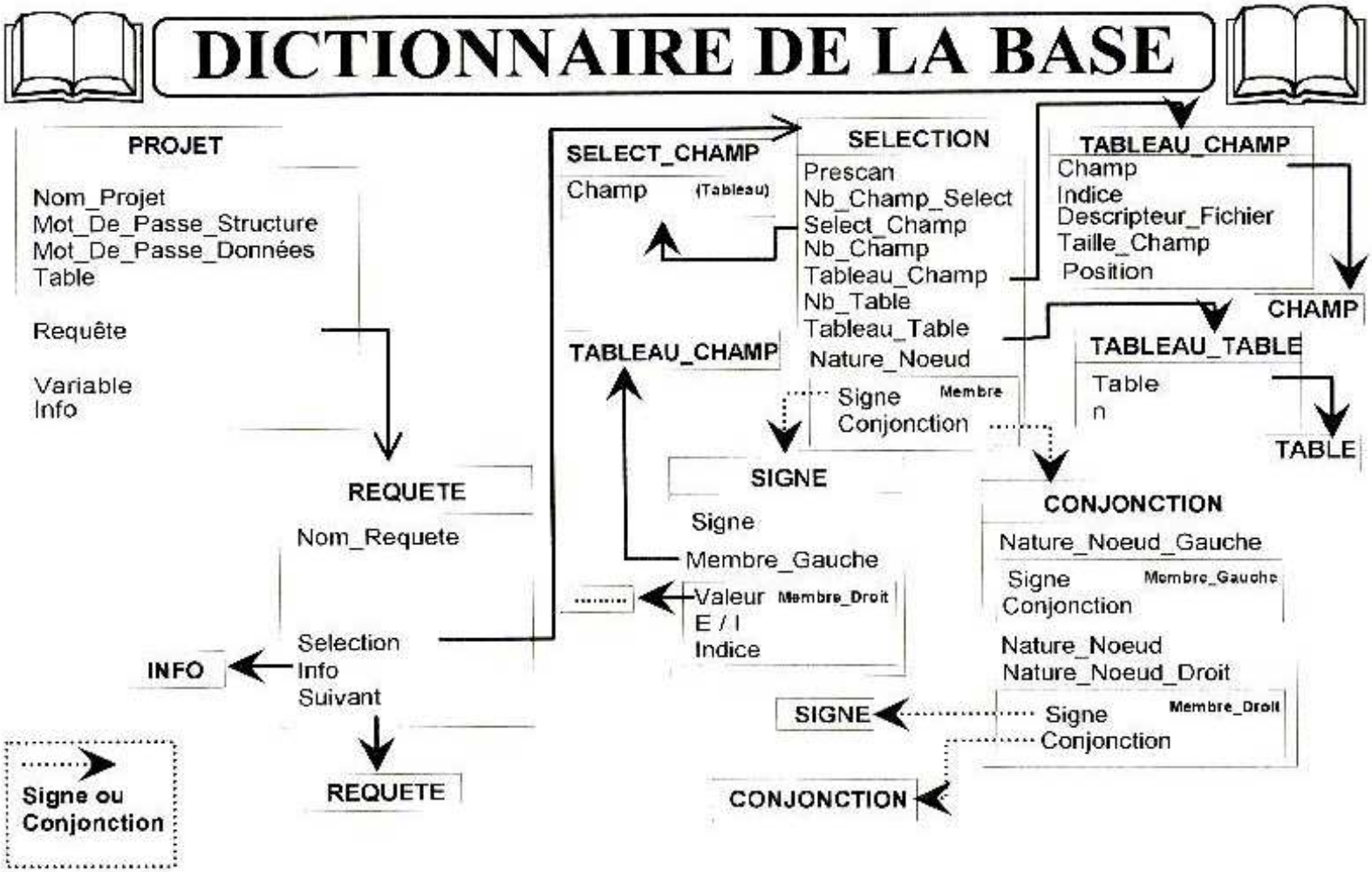
/* ------------------------------------------------------------ */
/* fonction : int Division(char *Resultat,char *Donnee,         */
/*                                     int Taille_Resultat);  */
/*   Cette fonction réalise la division réelle de deux réels dont*/
/*       Resultat = Resultat / Donnee.                          */
/*   Cette fonction retourne 0 quand pas d'erreurs sinon :      */
/*    - 1 : Allocation impossible                               */
/*    - 2 : Résultat n'est pas un réel                          */
/*    - 3 : Donnée n'est pas un réel                            */
/*    - 4 : Taille_Resultat est trop petit                      */
/*    - 5 : Division par 0                                      */
/* ------------------------------------------------------------ */

extern "C" int Division(char *Resultat,char *Donnee,int Taille_Resultat);

/* ------------------------------------------------------------ */
/* fonction : int Division_Entiere(char *Resultat,char *Donnee,  */
/*                                     int Taille_Resultat);  */
/*   Cette fonction réalise la division entiére de deux entiers  */
/*       Resultat = Resultat / Donnee.                          */
/*   Cette fonction retourne 0 quand pas d'erreurs sinon :      */
/*    - 1 : Allocation impossible                               */
/*    - 2 : Résultat n'est pas un réel                          */
/*    - 3 : Donnée n'est pas un réel                            */
/*    - 4 : Taille_Resultat est trop petit                      */
/*    - 5 : Division par 0                                      */
/*    - 6 : un des deux membres n'est pas un entier             */
/* ------------------------------------------------------------ */
extern "C" int Division_Entiere(char *Resultat,char *Donnee,
                                                    int Taille_Resultat);
```

```
/* ------------------------------------------------------------ */
/* fonction : int Modulo(char *Resultat,char *Donnee,           */
/*                                      int Taille_Resultat);   */
/*   Cette fonction réalise le reste de la division entiére de  */
/* deux entiers :                                               */
/*       Resultat = Resultat % Donnee.                          */
/*   Cette fonction retourne 0 quand pas d'erreurs sinon :      */
/*    - 1 : Allocation impossible                               */
/*    - 2 : Résultat n'est pas un réel                          */
/*    - 3 : Donnée n'est pas un réel                            */
/*    - 4 : Taille_Resultat est trop petit                      */
/*    - 5 : Division par 0                                      */
/*    - 6 : un des deux membres n'est pas un entier             */
/* ------------------------------------------------------------ */

extern "C" int Modulo(char *Resultat,char *Donnee,int Taille_Resultat);
```


DICTIONNAIRE DE LA BASE
PROJET
Nom_Projet
Mot_De_Passe_Structure
Mot_De_Passe_Données
Table
Requête
Variable
Info
REQUETE
Nom_Requete
Selection
Info
Suivant
INFO
REQUETE
SELECT_CHAMP
Champ (Tableau)
SELECTION
Prescan
Nb_Champ_Select
Select_Champ
Nb_Champ
Tableau_Champ
Nb_Table
Tableau_Table
Nature_Noeud
TABLEAU_CHAMP
Champ
Indice
Descripteur_Fichier
Taille_Champ
Position
CHAMP
TABLEAU_TABLE
Table
n
TABLE
Signe Membre
Conjonction
SIGNE
Signe
Membre_Gauche
Valeur Membre_Droit
E / I
Indice
CONJONCTION
Nature_Noeud_Gauche
Signe Membre_Gauche
Conjonction
Nature_Noeud
Nature_Noeud_Droit
SIGNE
Signe Membre_Droit
Conjonction
CONJONCTION
Signe ou Conjonction

SYSTEME DE GESTION DE BASES DE DONNEES

INTERROGATION DE LA BASE

Dans les parties précédentes, nous avons posé les bases d' un S.G.B.D., c' est à dire, le stockage des données en mémoire et sur disque. Pour permettre une dynamicité des requêtes, les champs pourront être comparés à des variables. Les variables pouvant contenir le résultat d' une opération arithmétique.

Cette partie offrira des outils permettant une consultation rapide de la base. Son atout sera le fait de pouvoir avoir plusieurs requêtes ouvertes en même temps. La requête à un niveau et les variables pouvant permettre des combinaisons trés importantes.

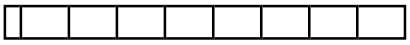
1. Définition d' une requête

Une requête est un outil permettant de consulter tout ou partie de la base. Elle nécessite :

- un nom,
- trois types d' écrans (pouvant être facultatifs),
- un texte indiquant la requête,
- un arbre de résolution.

2. Requête à un niveau (Ne fait intervenir qu' un table)

Rappel :



Voici la structure d' un enregistrement au niveau physique.

La premiére case est un caractére d' activité.

Le reste des cases renferme des nombres entiers contenant les pointeurs sur les données.

Remarque :

Les signes pouvant apparaît dans une comparaison :

"="	: Egal
"!="	: Non égal
">"	: Supérieur
"=>"	: Supérieur ou égal
"<"	: Inférieur
"<="	: Inférieur ou égal

2.1. Grammaire d' une requête de premier niveau

Cette grammaire tente de se rapprocher d' un SQL.

< requête > ::= "SELECT" < liste de champs > "FROM" < table >
< suite requête > ";"

< liste de champs > ::= "*"
::= < ensemble de champs >

< ensemble de champs > ::= < champ > < suite ensemble >

< suite ensemble > ::= "," < champ > < suite ensemble >
::=

< suite requête > ::= "WHERE" < condition >
::=

< condition > ::= < comparaison >
::= < comparaison_suite_et >
::= < comparaison_suite_ou >

< comparaison_suite_et > ::= < comparaison > "AND" <comparaison_suite_et>
::= < comparaison >

< comparaison_suite_ou > ::= < comparaison > "OR" <comparaison_suite_ou>
::= < comparaison >

< comparaison > ::= "(" < condition > ")"
::= "NOT" < comparaison >
::= < champ > < signe > < membre >
::= < membre > < signe > < champ >

< membre > ::= "&" < Variable >
::= " ' " < constante > " ' "

< signe > ::= " > " | " < " | " = " | " => " | " <= " | " >=" | " =<" | " != "

2.2. Construction d' un arbre de résolution de la requête

Cette construction se fera sous la forme d' une analyse sémantique intégrant un traitement des erreurs. La présentation de cette construction sera différente de ce qui a été présentée jusqu' à présent car la requête se formulera sous forme d' un texte (en effet, tout ce qui a été analysé jusqu' à présent nécessitait un découpage du traitement, alors qu' ici, la requête sous forme de texte, ne demande pas à l' utilisateur de déclarer à travers des fonctions, le nom des champs en entrée, de la table concernée et de la suite des conditions de la requête).

Cette construction marquera le fait qu' une requête est ouverte ou pas.

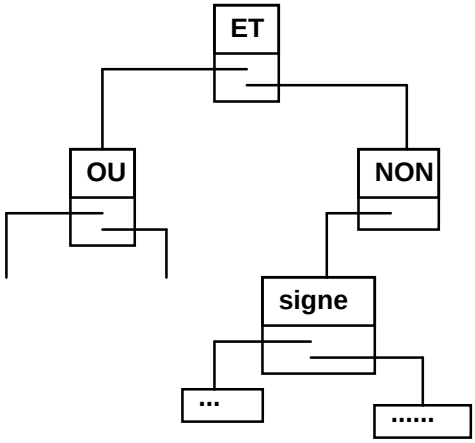
Remarque : L' arbre construit ne contiendra pour des raisons d' optimisation que les opérateurs logiques de bases "AND" et "OR".

2.2.1. Les différentes étapes de la construction de l' arbre

* L' arbre de résolution ne se construira pas du premier coup. Il sera nécessaire de créer un arbre paralléle contenant que les informations élèmentaires de la requête, c' est-à-dire, les mots issus du découpage du texte.
* Cet arbre devra être développé afin de ne présenter que des "AND" et des "OR" dans sa structure.
* Cette construction précaire sera ensuite utilisée pour construire l' arbre de résolution avec une premiére optimisation des conditions simples en les marquant par des Tautologies (toujours vrai) ou des Antilogies (toujours faux).
* Optimisation de l' arbre en supprimant tous les noeuds inutiles

2.2.2. Construction de l' arbre paralléle

Présentation de cet arbre avec ces structures :



Comme vous pouvez le voir, cet arbre sera un arbre binaire.

2.2.3. Développement de cet arbre précaire

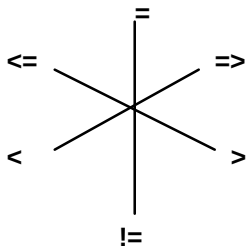
Cet arbre subira en premier lieu la suppression des "NON".

Les calculs de Boole que voici, fait descendre les "NON" en bas de l' arbre :

NON (A et B) = NON (A) ou NON (B)
NON (A ou B) = NON (A) et NON (B)
NON (NON (A)) = A

Le fait qu' il est possible de trouver l' opposer dans les signes d' une comparaison permet de faire disparaître le NON de l' arbre.

Voici les oppositions :



Aprés la suppression des "NON", l' arbre sera alors développé de maniére à présenter les ET en bas de l' arbre (ET étant proritaire sur le OU) :

(A ou B) et C = (A et C) ou (B et C)
A et (B ou C) = (A et B) ou (A et C)
(A ou B) et (C ou D) =
(A et C) ou (A et D) ou (B et C) ou (B et D)

Aprés ces deux opérations, l' arbre est prêt pour être construit.

2.2.4. Construction de l' arbre de résolution

Une fonction permettant d' être exploitée par le programme comme par l' utilisateur sera alors mise en place afin de permettre une dynamique encore plus importante aux requêtes. Ceci offrant la possibilité d' ajouter des conditions en fonction de paramétres extérieurs dépendant de la volonté du programmeur. Cette souplesse nécessite ,ce pendant, une bonne gestion de l' environnement de la part du programmeur.

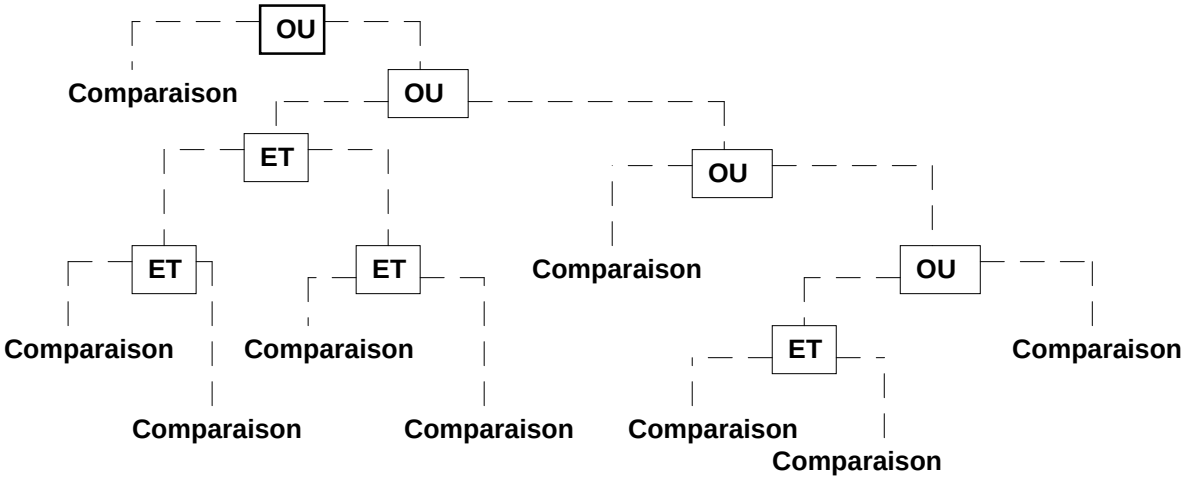
A ce niveau, il peut être étudier un premier niveau d' optimisation (c' est la vérification de la validité d' une comparaison, par le fait qu' elle soit une ***tautologie*** ou une ***antilogie***).

2.2.5. Optimisation de l' arbre de résolution

La structure adoptée pour ce SGBD offre ,en plus d' une trés grande capacité à éviter les redondances d' informations, un moyen d' optimiser au mieux une requête émanant de l' utilisateur. Il est donc facile de repérer qu' elles sont les conditions valides et non valides par une recherche de la valeur à comparer. Cette phase permettra de simplifier encore un peu plus la requête de l' utilisateur.

Tout ceci est réalisé dans le soucis de répondre au plus vite à la requête d' un utilisateur.

2.2.6. Présentation de l' arbre final



Remarque : Cet arbre ainsi construit présente la caractéristique de posséder tous les "OU" sur le noeud droit et les "ET" à partir des branches gauches des "OU".

2.3. Opérations réalisables pour les requêtes

Les opérations réalisables pour les requêtes peuvent être distinguées en deux parties :

- Une gestion directe des requêtes (Construction logique),
- L' utilisation des données dans les requêtes.

Remarque : une requête posséde 2 états : ouverte ou fermée.

* Ouverte : L' arbre de résolution est construit.
* Fermée : L' arbre de résolution n' est pas construit

2.3.1. La gestion directe des requêtes

Les opérations ,correspondantes à cette partie ,s' appliquent à la construction logique d' une requête, c' est à dire:

- création d' une requête,
- Modification des paramétres d' une requête,
- Suppression d' une requête,
- Connaître le texte d' une requête.

2.3.1.1. Création d' une requête

Cette opération consiste à déclarer dans la structure de la base logique et dans le dictionnaire, une nouvelle requête. Il ne peut pas y avoir deux requêtes portant le même nom car ce nom constitue l' identifiant de la requête. La requête ainsi crée nécessite en entrée un texte de base. Le texte de la requête sera alors sauvegardé avec le nom et le répertoire correspondant à la requête dans la base.

2.3.1.2. Modification d' une requête

Deux modifications peuvent être réalisées :

- modification du nom de la requête (code=1),
- modification du texte de la requête (code=2).

Lors de la modification du nom de la requête, nous vérifions que le nouveau nom n' est pas redondant pour éviter toute confusion.

Remarque : void * --> char *

Lors de la modification du texte de la requête, le seul traitement réalisé est l' écrasement de l' ancien texte par le nouveau. Aucune analyse syntaxique n' est réalisée dans cette partie.

Remarque : void * --> char **

2.3.1.3. Suppression d' une requête

Cette suppression nécessite le fait que la requête existe et n' est pas ouverte. Elle détruit logiquement et physiquement la requête en question.

2.3.2. Exploitation des requêtes sur les données

Cette exploitation envisage quelques traitements :

- Ouverture d' une requête,
- Fermeture d' une requête,
- Positionnement du curseur sur le premier enregistrement,
- Positionnement du curseur sur l' enregistrement suivant,
- Positionnement du curseur sur le dernier enregistrement,
- Positionnement du curseur sur l' enregistrement précédent
- Ajout d' une condition
- Prescannage de la requête.

2.3.2.1. Ouverture de la requête.

Cette ouverture positionne la requête d' un état fermé à un état ouvert. Cette ouverture réalise ,en fait ,la construction de l' arbre de résolution de la requête couplée à une analyse syntaxique de cette derniére. Cette phase a été développée en "2.2. Construction de l' arbre de résolution". Pour que cette ouverture soit validée, il faut que la requête existe et n' est pas déjà ouverte.

2.3.2.2. Fermeture de la requête

Cette fermeture réalise la destruction de l' arbre de résolution et le fichier de prescannage, s' il y a lieu, de la requête correspondante.

2.3.2.3. Parcours dans le résultat de la requête

Lorsque la requête est ouverte, il est possible de se déplacer dans la vue ainsi réalisée. Nous pouvons distinguer quatre déplacements :

- Positionnement sur le premier,
- Positionnement sur le suivant,
- Positionnement sur le précédent,
- Positionnement sur le dernier.

Ce déplacement a lieu séquentiellement dans le fichier de données de la table de la requête, chaque enregistrement est testé en fonction de l' arbre de résolution. Cette résolution subit alors un troisiéme niveau d' optimisatio (aprés l' optimisation des comparaisons et l' optimisation des conditions).

Optimisation :

A ou B = C
C est vrai si A est vrai sinon il faut que B soit Vrai
A et B = C
C est faux si A est faux sinon il faut que B soit Faux

Remarque : A chaque parcours, tous les fichiers données des champs dont la nécessité pour la comparaison s' avére nécessaire sont ouverts momentannéement. Ceci nécessite de la part du programmeur de bien vérifier que le nombre de fichiers pouvant être ouverts en même temps correspond aux demandes.

2.3.2.4. Ajouter une condition

Cette fonction intervient directement dans la construction de l' arbre de résolution puisque c' est cette fonction qui crée l' arbre de maniére séquentielle.

Cette fonction n' accepte comme conjonction que les "OR" et les "AND", aucune autre conjonction !!!! Cette fonction tenant compte du fait que les "AND" sont prioritaires sur les "OR", c' est à dire en bas de l' arbre. Cette fonction assure une parfaite validation des données puisqu' elle réalisera une analyse des différents élèments qui consistituent une comparaison.

Une comparaison ne doit pas posséder deux membres champs et toute comparaison doit contenir un champ dans l' un des deux membres. Les variables sont marquées par le fait qu' elles sont précédées par un ' &' . les constantes sont entre deux simples côtes.

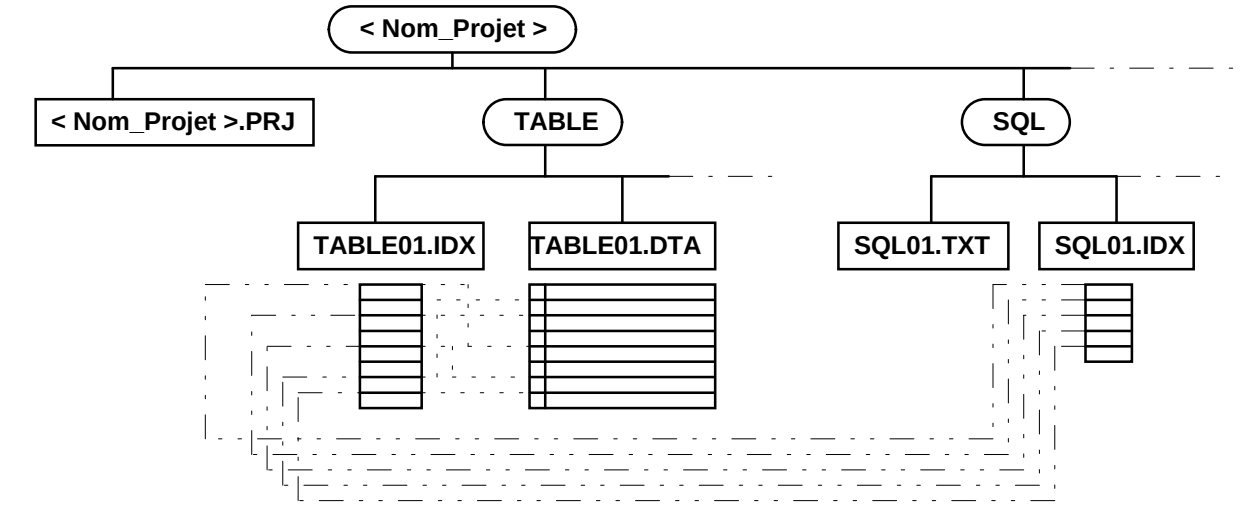
2.3.2.5. Prescannage de la requête

Cette opération est intéressante dans le cas, où le résultat de la requête est utilisée plusieurs fois dans le même traitement. Cette opération consiste à créer un fichier contenant tous les indices des enregistrements de la table concernée par la requête. (Cf représentation physique d' une requête). Une fois, ce prescannage réalisé, tous les ajouts de conditions ne seront pas prises en compte. Il est donc important de réaliser ces ajouts avant de demander un prescannage. Cette opération pourra permettre d' accélerer le parcours des enregistrements répondant à la requête associée.

Ceci permettra de réaliser des "calculs" sur un ensemble d' enregistrements répondant à certains critéres.

Remarque : le retour de cette fonction est le nombre d' enregistrements préscannés correspondant à la requête en question.

2.4. Représentation physique d' une requête

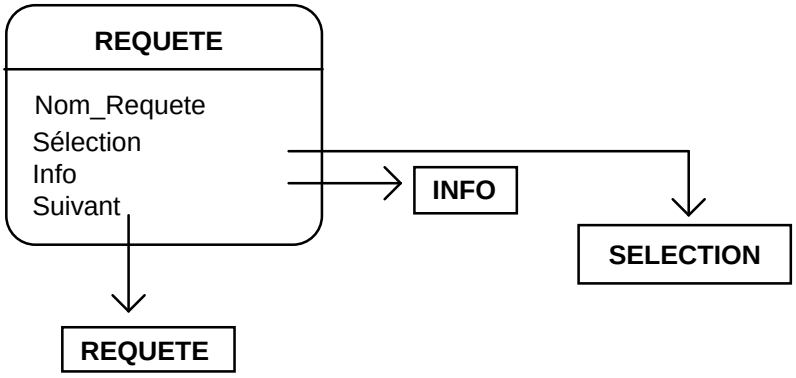


Fichier **Répertoire**

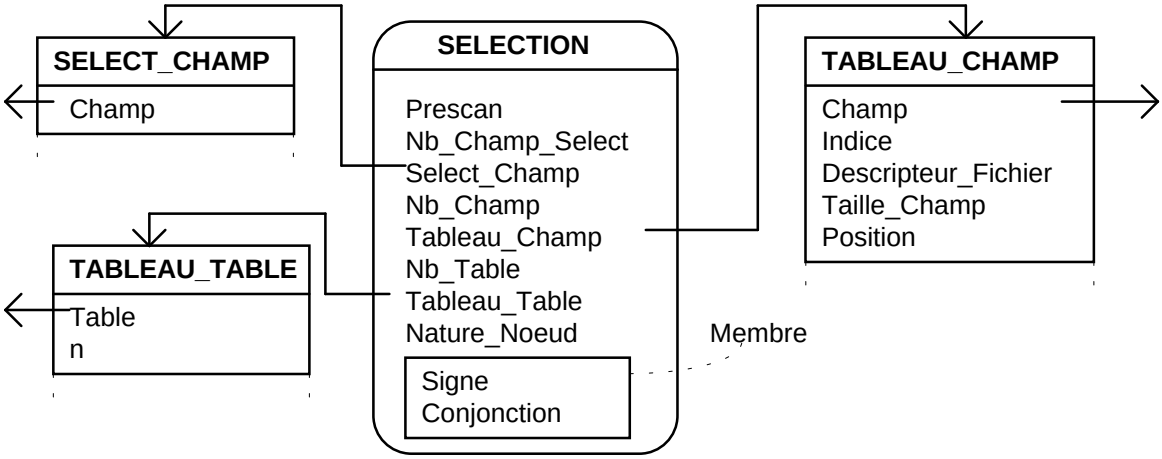
2.5. Représentation logique d' une requête

Une requête se distingue par le fait, qu' elle se compose des paramétres d' une requête , de l' association de celle-ci avec des écrans et de l' arbre de résolution.

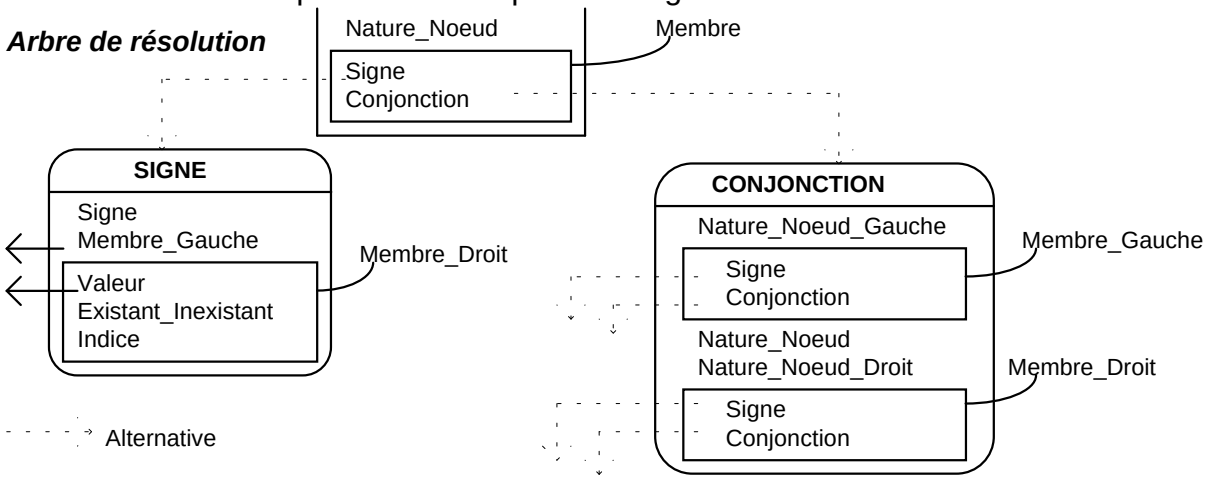
2.5.1. La requête en elle-même



2.5.2. L' arbre de résolution de la requête



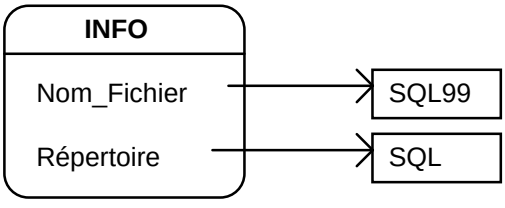
Cette structure regroupe les élèments présents dans l' entête d' une requête. La structure présentée permet de pouvoir associer à une requête plusieurs tables. Cependant, c' est consciemment que la grammaire de la requête ne contient qu' une seule table car cette expension n' est pas envisageable tout de suite.



2.6. Déclaration de la requête dans le dictionnaire

SQL99: < Nom de la requete >

2.7. Contenu de la structure INFO d' une requête



3. Comment concevoir une requête à plusieurs niveaux ?

La structure qui est présentée, au niveau de l' arbre de résolution, propose un stockage de une ou plusieurs tables. C' est volontairement que la grammaire et la programmation tiennent compte d' une seule et unique table. Cependant, il est possible d' élargir les perspectives. Dans ce chapitre, nous allons développer des algorithmes qui permettent de concevoir simplement des requêtes à plusieurs niveaux (entrainant plusieurs tables).

Le principal atout de cette partie du SGBD est le fait que plusieurs requêtes peuvent être ouvertes simultanéement.

3.1. Une requête comparant deux champs d' une même table

```
SELECT *
FROM Table;
```

Cette condition est à réaliser sur chaque enregistrement :

si (Nom_Champ1 == Nom_Champ2) alors Enregistrement valide
sinon Enregistrement invalide

Ceci équivaut à :

```
SELECT *
FROM Table
WHERE Nom_Champ1 = Nom_Champ2 ;
```

3.2. Une requête comparant deux champs de deux tables différentes

```
SELECT *
FROM Table1 ;

&Variable <-- Nom_Champ

SELECT *
FROM Table2
WHERE Nom_Champ2 = &Variable ;
```

Ceci équivaut à :

```
SELECT *
FROM Table1,Table2;
WHERE Nom_Champ = Nom_Champ2;
```

Remarque : Dans cet exemple, il est interessant de faire appel au prescannage de la premiére requête (dans le cas où elle renferme un certain nombre de conditions dans le WHERE). Ceci afin d' éviter l' utilisation de l' arbre de résolution à chaque fois qu' une valeur change (ce traitement pourrait permettre de gagner du temps en évitant de refaire trop souvent le même traitement).

4. Les optimisations réalisables

Remarque :

Tautologie : Toujours Vrai quelque soit la valeur du champ
Antilogie : Toujours Faux quelque soit la valeur du champ

L' optimisation d' une requête passe par les cinq étapes qui sont :

- Optimisation des signes
- Optimisation des conditions ET
- Optimisation des conditions OU
- Optimisation des combinaisons
- Optimisation en utilisant la structure même de la sauvegarde

4.1. Optimisation des signes

Champ = Valeur	Si valeur n' existe pas	Antilogie
	Si valeur a une occurrence nulle	Antilogie
	Si valeur existe et est seule	Tautologie
Champ != Valeur	Si valeur n' existe pas	Tautologie
	Si valeur a une occurrence nulle	Tautologie
	Si valeur existe et est seulle	Antilogie
Champ > Valeur	Si valeur a sa zone **d'insertion** en 0	Tautologie
	Si valeur a sa zone **d'insertion** en fin index	Antilogie
	Si valeur a sa zone **d'existance** en fin index	Antilogie
Champ < Valeur	Si valeur a sa zone **d'insertion** en 0	Antilogie
	Si valeur a sa zone **d'insertion** en fin index	Tautologie
	Si valeur a sa zone **d'existance** en 0	Antilogie
Champ >= Valeur	Si valeur a sa zone **d'insertion** ou **existance** à 0	Tautologie
	Si valeur a sa zone **d'insertion** en fin index	Antilogie
	Si valeur a sa zone **d'existance** en fin index	Champ=Valeur
Champ <= Valeur	Si valeur a sa zone **d'insertion** ou **existance** en fin d' index	Tautologie
	Si valeur a sa zone **d'insertion** en 0	Antilogie
	Si valeur a sa zone **d'existance** en 0	Champ=Valeur

4.2. Optimisation des conditions

4.2.1. Réduction de l' arbre de résolution à une antilogie ou une tautologie

Si un seul membre gauche des "OU" est une tautologie, l' arbre de résolution sera une tautologie. Si tous les membres droits des "OU" sont une antilogie, l' arbre de résolution sera une antilogie.

Si, parmi les "ET" d' une même branche de l' arbre de résolution , il y a une antilogie, cette branche sera alors une antilogie. Si tous les membres des "ET" d' une même branche de l' arbre de résolution comportent que des tautologies, cette branche devient une tautologie.

4.2.2. Optimisation impliquant deux comparaisons

Cette optimisation n' a pas été encore réalisée. Mais, ce paragraphe présente une optimisation poussée qui peut être appliquée à ce SGBD (grâce à la sauvegarde des données et à la séparation des valeurs des champs de la table). Cette optimisation implique un traitement de 72 comparaisons et combinaisons possibles.

* Le tableau qui suit présente une optimisation qui ne nécessite pas l' utilisation de la structure de sauvegarde des données.

Champ=Valeur	AND	Champ=Valeur'	Si valeur=valeur' , Champ=Valeur sinon Antilogie
	AND	Champ!=Valeur'	Si valeur=Valeur' , Antiologie
	AND	Champ > Valeur'	Si Valeur≤Valeur' , Antilogie sinon Champ>Valeur'
	AND	Champ < Valeur'	Si Valeur≥Valeur' , Antilogie sinon Champ<Valeur'
	AND	Champ ≥ Valeur'	Si Valeur<Valeur' , Antilogie sinon Champ≥Valeur'
	AND	Champ ≤ Valeur	Si Valeur>Valeur' , Antilogie sinon Champ≤Valeur'
Champ!=Valeur	AND	Champ=Valeur'	Cf Champ=Valeur AND Champ!=Valeur'
	AND	Champ!=Valeur'	Si Valeur=Valeur' , Champ!=Valeur
	AND	Champ > Valeur'	Si Valeur≤Valeur' , Champ>Valeur'
	AND	Champ < Valeur'	Si Valeur≥Valeur' , Champ<Valeur'
	AND	Champ ≥ Valeur'	Si Valeur=Valeur' , Champ>Valeur' Si Valeur<Valeur, Champ≥Valeur'
	AND	Champ ≤ Valeur'	Si Valeur=Valeur' , Champ<Valeur' Si Valeur>Valeur' , Champ≤Valeur'
Champ > Valeur	AND	Champ=Valeur'	Cf Champ=Valeur AND Champ>Valeur'
	AND	Champ!=Valeur'	Cf Champ!=Valeur AND Champ>Valeur'
	AND	Champ > Valeur'	Si Valeur > Valeur' , Champ > Valeur Si Valeur ≤ Valeur' , Champ > Valeur'
	AND	Champ < Valeur'	Si Valeur > Valeur' , Antilogie
	AND	Champ ≥ Valeur'	Si Valeur ≥ Valeur' , Champ > Valeur Si Valeur < Valeur' , Champ ≥ Valeur
	AND	Champ ≤ Valeur'	Si Valeur ≤ Valeur' , Antilogie
Champ < Valeur	AND	Champ=Valeur'	Cf Champ=Valeur AND Champ<Valeur'
	AND	Champ!=Valeur'	Cf Champ!=Valeur AND Champ<Valeur'
	AND	Champ > Valeur'	Cf Champ>Valeur AND Champ<Valeur'
	AND	Champ < Valeur'	Si Valeur < Valeur' , Champ < Valeur sinon Champ < Valeur'
	AND	Champ ≥ Valeur'	Si Valeur ≤ Valeur' , Antilogie
	AND	Champ ≤ Valeur'	Si Valeur ≤ Valeur' , Champ < Valeur sinon Champ ≤ Valeur'
Champ ≥ Valeur	AND	Champ=Valeur'	Cf Champ=Valeur AND Champ≥Valeur'
	AND	Champ!=Valeur'	Cf Champ!=Valeur AND Champ≥Valeur'
	AND	Champ > Valeur'	Cf Champ>Valeur AND Champ≥Valeur'
	AND	Champ < Valeur'	Cf Champ<Valeur AND Champ≥Valeur'
	AND	Champ ≥ Valeur'	Si Valeur ≥ Valeur' , Champ ≥ Valeur sinon Champ ≥ Valeur'
	AND	Champ ≤ Valeur'	Si Valeur=Valeur' , Champ = Valeur Si Valeur>Valeur' , Antilogie
Champ ≤ Valeur	AND	Champ=Valeur'	Cf Champ=Valeur AND Champ≤Valeur'
	AND	Champ!=Valeur'	Cf Champ!=Valeur AND Champ≤Valeur'
	AND	Champ > Valeur'	Cf Champ>Valeur AND Champ≤Valeur'
	AND	Champ < Valeur'	Cf Champ<Valeur AND Champ≤Valeur'
	AND	Champ ≥ Valeur'	Cf Champ≥Valeur AND Champ≤Valeur'
	AND	Champ ≤ Valeur'	Si Valeur ≤ Valeur' , Champ ≤ Valeur sinon Champ ≤ Valeur'

Champ=Valeur	OR	Champ=Valeur'	Si Valeur=Valeur' , Champ=Valeur
	OR	Champ!=Valeur'	Si Valeur=Valeur' , Tautologie
	OR	Champ > Valeur'	Si Valeur > Valeur' , Champ > Valeur'
	OR	Champ < Valeur'	Si Valeur < Valeur' , Champ < Valeur'
	OR	Champ ≥ Valeur'	Si Valeur ≥ Valeur' , Champ ≥ Valeur'
	OR	Champ ≤ Valeur	Si Valeur ≤ Valeur' , Champ ≤ Valeur'
Champ!=Valeur	OR	Champ=Valeur'	Cf Champ=Valeur OR Champ!=Valeur'
	OR	Champ!=Valeur'	Si Valeur=Valeur' , Champ!=Valeur
	OR	Champ > Valeur'	Si Valeur≤Valeur' , Champ>Valeur'
	OR	Champ < Valeur'	Si Valeur≥Valeur' , Champ<Valeur
	OR	Champ ≥ Valeur'	Si Valeur=Valeur' , Champ>Valeur' Si Valeur<Valeur' , Champ≥Valeur'
	OR	Champ ≤ Valeur'	Si Valeur=Valeur' , Champ<Valeur' Si Valeur>Valeur' , Champ≤ Valeur'
Champ > Valeur	OR	Champ=Valeur'	Cf Champ=Valeur OR Champ>Valeur'
	OR	Champ!=Valeur'	Cf Champ!=Valeur OR Champ>Valeur'
	OR	Champ > Valeur'	Si valeur>valeur' , Champ>valeur' sinon Champ>Valeur
	OR	Champ < Valeur'	Si Valeur=Valeur' , Champ!=valeur Si valeur<valeur' , Tautologie
	OR	Champ ≥ Valeur'	Si valeur<valeur' , Champ>valeur sinon Champ≥valeur'
	OR	Champ ≤ Valeur'	Si Valeur≤Valeur' , Tautologie
Champ < Valeur	OR	Champ=Valeur'	Cf Champ=Valeur OR Champ<Valeur'
	OR	Champ!=Valeur'	Cf Champ!=Valeur OR Champ<Valeur'
	OR	Champ > Valeur'	Cf Champ>Valeur OR Champ<Valeur'
	OR	Champ < Valeur'	Si valeur>valeur' , champ<Valeur sinon champ<valeur'
	OR	Champ ≥ Valeur'	Si Valeur≥valeur' , Tautologie
	OR	Champ ≤ Valeur'	Si valeur>valeur' , champ<valeur sinon champ≤valeur'
Champ ≥ Valeur	OR	Champ=Valeur'	Cf Champ=Valeur OR Champ≥Valeur'
	OR	Champ!=Valeur'	Cf Champ!=Valeur OR Champ≥Valeur'
	OR	Champ > Valeur'	Cf Champ>Valeur OR Champ≥Valeur'
	OR	Champ < Valeur'	Cf Champ<Valeur OR Champ≥Valeur'
	OR	Champ ≥ Valeur'	Si valeur≥valeur' , Champ≥Valeur' sinon Champ≥Valeur
	OR	Champ ≤ Valeur'	Si Valeur≤Valeur' , Tautologie
Champ ≤ Valeur	OR	Champ=Valeur'	Cf Champ=Valeur OR Champ≤Valeur'
	OR	Champ!=Valeur'	Cf Champ!=Valeur OR Champ≤Valeur'
	OR	Champ > Valeur'	Cf Champ>Valeur OR Champ≤Valeur'
	OR	Champ < Valeur'	Cf Champ<Valeur OR Champ≤Valeur'
	OR	Champ ≥ Valeur'	Cf Champ≥Valeur OR Champ≤Valeur'
	OR	Champ ≤ Valeur'	Si valeur>Valeur' , Champ≤Valeur sinon Champ≤Valeur'

* Les traitements qui vont suivre utilisent la structure de sauvegarde des données, ils s'appliquent aprés l'optimisation du tableau ci-dessus :

- Lors d'un encadrement d'un champ par une valeur :

S'il existe qu'une seule valeur comprise dans l'intervalle, cet encadrement devient un égalité. Si aucune valeur n'est comprise dans l'intervalle, cet encadrement devient une antilogie et si TOUTES les valeurs du champs sont contenues dans cet encadrement, il devient une tautologie.

- D' une maniére générale, toutes comparaisons impliquant un signe d' inégalité peuventêtre étendues à un encadrement avec soit le plus grand des élèments soit le plus petit (< , > , <= , =>).

```
/* ************************************************************************ */
/*                Systéme de Gestion de Bases de Données                    */
/*                              Version 1.0                                 */
/*                                                                          */
/* Date : 01/06/95                                                          */
/*                                                                          */
/*                                                        par LESERT Aymeric */
/* ************************************************************************ */

/* ********************************************************************** */
/*             Unité de gestion et d'exploitation des requêtes            */
/*                                                                        */
/* Date : 20/08/94                                                        */
/* ********************************************************************** */

/* ********************************************************************** */
/* *************************** PARTIE I ********************************* */
/*          Partie concernant la gestion directe des requêtes           */

/* ---------------------------------------------------------------------- */
/* fonction : int Creer_Requete(char *Mot_De_Passe,char *Nom_Requete,   */
/*                                                 char **Texte);         */
/*   Cette fonction doit permettre de créer une requête du type SQL     */
/* trés facilement.                                                       */
/*      Le mot de passe utilisé est celui de la structure                 */
/*      Nom Requete : nom de la requête concernée.                        */
/*      Texte : Texte de la requête (tableau de chaînes de caractéres)  */
/*   Cette fonction retournera 0 quand la requête a pu être créée sinon */
/*      - 1 : Pas de projet ouvert                                        */
/*      - 2 : Pas de relations dans l'application                         */
/*      - 3 : Nom_Requete déjà répertorié                                 */
/*      - 4 : Taille du nom de la requête incorrect                       */
/*      - 5 : Nom de la requête non conforme                              */
/*      - 6 : Allocation impossible                                       */
/*      - 7 : Accés Repertoire du projet impossible                       */
/*      - 8 : Accés répertoire des requêtes (SQL) impossible              */
/*      - 9 : impossible de créer le fichier texte de la requête          */
/*      - 10: Mot de passe incorrect                                      */
/* ---------------------------------------------------------------------- */

extern "C" int Creer_Requete(char *Mot_De_Passe,char *Nom_Requete,char **Texte);
```

```
/* ------------------------------------------------------------ */
/* fonction : int Modifier_Requete(char *Mot_De_Passe,int Code,   */
/*                       char *Nom_Requete,void *Parametre);     */
/*   Cette fonction permet de modifier soit le nom de la requête  */
/* (Code=1) , soit la requête elle-même (Code=2).                */
/*   Cette fonction retourne 0 quand cette modification a eu lieu */
/* sinon :                                                       */
/*      - 1 : Pas de projet ouvert                               */
/*      - 2 : Pas de requête recensée                            */
/*      - 3 : Nom de la requête inconnue                         */
/*      - 4 : Code de modification incorrect                     */
/*      - 5 : Nouveau_Nom = Ancien_Nom                           */
/*      - 6 : Nouveau Nom déjà existant                          */
/*      - 7 : Taille du nouveau nom incorrecte                   */
/*      - 8 : Nouveau nom non conforme au domaine de définition  */
/*      - 9 : Accés répertoire du projet impossible              */
/*      - 10: Accés au répertoire des requêtes impossible        */
/*      - 11: Impossible d'écrirer la nouvelle requête           */
/*      - 12: Requête ouverte                                    */
/*      - 13: Mot de passe incorrect                             */
/* ------------------------------------------------------------ */

extern "C" int Modifier_Requete(char *Mot_De_Passe,int Code,char *Nom_Requete,
                                                        void *Parametre);

/* ------------------------------------------------------------------------ */
/* fonction : int Supprimer_Requete(char *Mot_De_Passe,char *Nom_Requete); */
/*   Cette fonction détruira une requête.                                   */
/*   Cette fonction retourne 0 quand cette suppression a eu lieu sinon      */
/*      - 1 : Pas de projet ouvert                                          */
/*      - 2 : Mot de passe incorrect                                        */
/*      - 3 : Pas de requêtes recensées                                     */
/*      - 4 : Nom de requête inconnue                                       */
/*      - 5 : Requête ouverte                                               */
/* ------------------------------------------------------------------------ */

extern "C" int Supprimer_Requete(char *Mot_De_Passe,char *Nom_Requete);

/* ------------------------------------------------------------------- */
/* fonction : char **Lire_Requete(char *Nom_Requete,int *Erreur);      */
/*   Cette fonction permettra de récupérer le texte de la requête.     */
/*   Cette fonction retourne un pointeur sur un tableau de chaînes     */
/* de caractéres contenant le texte sinon , un pointeur NULL et Erreur */
/* affectée de la cause :                                              */
/*      - 1 : Pas de projet ouvert                                     */
/*      - 2 : Pas de requête recensée                                  */
/*      - 3 : La requête n'existe pas                                  */
/*      - 4 : Accés répertoire du projet impossible                    */
/*      - 5 : Accés répertoire des requêtes impossible                 */
/*      - 6 : Allocation impossible                                    */
/*      - 7 : Impossible d'ouvrir en lecture le fichier de la requête  */
/* ------------------------------------------------------------------- */

extern "C" char **Lire_Requete(char *Nom_Requete,int *Erreur);
```

```
/* ------------------------------------------------------------------ */
/* fonction : int Existe_Requete(char *Nom_Requete);                  */
/*   Cette fonction retourne Vrai si la requête indiquée existe sinon */
/* Elle retourne Faux.                                                */
/* ------------------------------------------------------------------ */

extern "C" int Existe_Requete(char *Nom_Requete);

/* ****************************************************************** */
/* *************************** PARTIE II **************************** */
/*            Partie concernant l'utilisation des requêtes             */

/* ------------------------------------------------------------------ */
/* fonction : int Ouvrir_Requete(char *Nom_Requete);                  */
/*    Cette fonction a pour but de construire l'arbre de résolution   */
/* de la requête énoncée dans le fichier TXT.                         */
/*    Cette fonction retourne 0 quand pas d'erreurs et que la struc-  */
/* -ture est construite sinon :                                       */
/*    - 1 : pas de projet ouvert                                      */
/*    - 2 : pas de requetes                                           */
/*    - 3 : la requête n'existe pas                                   */
/*    - 4 : la requête a déjà un arbre de résolution                  */
/*    - 5 : Accés au répertoire du projet impossible                  */
/*    - 6 : Accés au répertoire des requêtes impossible               */
/*    - 7 : Fichier texte de la requête impossible d'ouvrir           */
/*    - 8 : Allocation impossible                                     */
/*    - 9 : Analyse non terminée, argument manquant                   */
/*    - 10: 'SELECT' non localisé                                     */
/*    - 11: 'SELECT' n'a pas d'arguments                              */
/*    - 12: 'FROM' non trouvé                                         */
/*    - 13: Element inconnu aprés le ';'                              */
/*    - 14: pas de fin de requête                                     */
/*    - 15: Nom de la table aprés 'FROM' inconnu                      */
/*    - 16: Allocation impossible                                     */
/*    - 17: Manque un nom de champ avant la virgule dans SELECT       */
/*    - 18: Aprés '*' dans SELECT, il doit y avoir 'FROM'             */
/*    - 19: deux virgules consécutives non conforme                   */
/*    - 20: Nom de champs inexistant aprés SELECT                     */
/*    - 21: Nom de champs redondant aprés SELECT                      */
/*    - 22: Aprés une virgule, il faut un nom de champ dans SELECT    */
/*    - 23: Quand 'WHERE' est énoncé, il faut au moins une condition  */
/*    - 24: Pas de champs concernés dans le WHERE                     */
/*    - 25: Analyse incompléte manque un ";"                          */
/*    - 26: Manque d'une parenthése fermante                          */
/*    - 27: Manque d'une parenthése ouvrante                          */
/*    - 28: Pas de signe de comparaison                               */
/*    - 29: Signe incorrect : !=                                      */
/*    - 30: Conjonction inconnue                                      */
/*    - 31: Probléme de priorité entre AND et OR                      */
/*    - 32: le nom du champ est inconnu                               */
/*    - 33: Signe de comparaison inconnu                              */
/*    - 34: Comparaison de deux constantes impossible                 */
/*    - 35: Comparaison de deux champs impossible                     */
/*    - 36: la variable n'existe pas                                  */
/*    - 37: Type incompatible                                         */
/*    - 38: Taille incompatible                                       */
/*    - 39: Nb_Decimal incompatible                                   */
```

```
/*    - 40: le constante ne posséde pas de cotes                        */
/*    - 41: le domaine de définition n'est pas conforme                 */
/*    - 42: le domaine de définition n'est pas respecté                 */
/*    - 43: Taille de la constante trop grande                          */
/*    - 44: Nb de virgules dans un réel trop important                  */
/*    - 45: Date invalide                                               */
/*    - 46: Autres Erreurs                                              */
/*    - 47: Accés au répertoire du projet impossible                    */
/*    - 48: Accés au répertoire des tables impossible                   */
/*    - 49: Accés au répertoire du champ concerné impossible            */
/*    - 50: fichier index du champ impossible d'ouvrir                  */
/*    - 51: fichier donnée du champ impossible d'ouvrir                 */
/*    - 52: Conjonction inconnue                                        */
/* ---------------------------------------------------------------- */

extern "C" int Ouvrir_Requete(char *Nom_Requete);

/* ---------------------------------------------------------------------- */
/* fonction : int Fermer_Requete(char *Nom_Requete);                      */
/*   Cette fonction a pour but de liberer toute la structure de l'arbre */
/* de résolution.                                                         */
/*   Cette fonction retourne 0 quand la suppression a eu lieu sinon :   */
/*      - 1 : pas de projet ouvert                                        */
/*      - 2 : pas de requetes                                             */
/*      - 3 : la requête n'existe pas                                     */
/*      - 4 : la requête n'a pas d'arbre de résolution                    */
/* ---------------------------------------------------------------------- */

extern "C" int Fermer_Requete(char *Nom_Requete);

/* ---------------------------------------------------------------------------- */
/* fonction : int Enregistrement_Premier(char *Nom_Requete);                    */
/*   Cette fonction positionne les valeurs des champs de la table               */
/* concernée sur le premier enrgistrement correspondant à la requête.           */
/*   Cette fonction retourne >=0 si le premier enregistrement est localisé      */
/* , cette valeur retournée correspondant au n iéme enregistrement de la        */
/* table, sinon :                                                               */
/*    - -1 : Pas de projet ouvert                                               */
/*    - -2 : Pas de requêtes recensées                                          */
/*    - -3 : Nom de la requête inexistante                                      */
/*    - -4 : Requete non ouverte                                                */
/*    - -5 : Accés au répertoire de l'application impossible                    */
/*    - -6 : Accés au répertoire des tables impossible                          */
/*    - -7 : Fichier index de la table inouvrable                               */
/*    - -8 : Fichier donnée de la table inouvrable                              */
/*    - -9 : Allocation impossible                                              */
/*    - -10: Début de fichier atteint                                           */
/*    - -11: Fin de fichier atteint                                             */
/*    - -12 : accés au répertoire de la table du champ concerné impossible      */
/*    - -13 : fichier de données du champ impossible d'ouvrir                   */
/*    - -14 : Allocation impossible                                             */
/* ---------------------------------------------------------------------------- */

extern "C" int Enregistrement_Premier(char *Nom_Requete);
```

```
/* ------------------------------------------------------------------------ */
/* fonction : int Enregistrement_Suivant(char *Nom_Requete);                */
/*   Cette fonction positionne les valeurs des champs de la table           */
/* concernée sur l'enregistrement suivant correspondant à la requête.       */
/*   Cette fonction retourne une valeur positive si l'enregistrement        */
/* suivant correspondant est trouvée, cette valeur est l'indice de          */
/* l'enregistrement répondant à la requête, sinon :                         */
/*    - -1 : Pas de projet ouvert                                           */
/*    - -2 : Pas de requêtes recensées                                      */
/*    - -3 : Nom de la requête inexistante                                  */
/*    - -4 : Requete non ouverte                                            */
/*    - -5 : Accés au répertoire de l'application impossible                */
/*    - -6 : Accés au répertoire des tables impossible                      */
/*    - -7 : Fichier index de la table inouvrable                           */
/*    - -8 : Fichier donnée de la table inouvrable                          */
/*    - -9 : Allocation impossible                                          */
/*    - -10: Début de fichier atteint                                       */
/*    - -11: Fin de fichier atteint                                         */
/*    - -12 : accés au répertoire de la table du champ concerné impossible  */
/*    - -13 : fichier de données du champ impossible d'ouvrir               */
/*    - -14 : Allocation impossible                                         */
/* ------------------------------------------------------------------------ */

extern "C" int Enregistrement_Suivant(char *Nom_Requete);

/* ------------------------------------------------------------------------ */
/* fonction : int Enregistrement_Precedant(char *Nom_Requete);              */
/*   Cette fonction positionne les valeurs des champs de la table           */
/* concernée sur l'enregistrement précédant correspondant à la requête.     */
/*   Cette fonction retourne une valeur positive si l'enregistrement        */
/* précédant correspondant est trouvée, cette valeur est l'indice de        */
/* l'enregistrement répondant à la requête, sinon :                         */
/*    - -1 : Pas de projet ouvert                                           */
/*    - -2 : Pas de requêtes recensées                                      */
/*    - -3 : Nom de la requête inexistante                                  */
/*    - -4 : Requete non ouverte                                            */
/*    - -5 : Accés au répertoire de l'application impossible                */
/*    - -6 : Accés au répertoire des tables impossible                      */
/*    - -7 : Fichier index de la table inouvrable                           */
/*    - -8 : Fichier donnée de la table inouvrable                          */
/*    - -9 : Allocation impossible                                          */
/*    - -10: Début de fichier atteint                                       */
/*    - -11: Fin de fichier atteint                                         */
/*    - -12 : accés au répertoire de la table du champ concerné impossible  */
/*    - -13 : fichier de données du champ impossible d'ouvrir               */
/*    - -14 : Allocation impossible                                         */
/* ------------------------------------------------------------------------ */

extern "C" int Enregistrement_Precedent(char *Nom_Requete);
```

```
/* ------------------------------------------------------------------------ */
/* fonction : int Enregistrement_Dernier(char *Nom_Requete);                */
/*   Cette fonction positionne les valeurs des champs de la table           */
/* concernée sur le dernier enrgistrement correspondant à la requête.       */
/*   Cette fonction retourne >=0 si le dernier enregistrement est localisé */
/* , cette valeur retournée correspondant au n iéme enregistrement de la    */
/* table, sinon :                                                           */
/*    - -1 : Pas de projet ouvert                                           */
/*    - -2 : Pas de requêtes recensées                                      */
/*    - -3 : Nom de la requête inexistante                                  */
/*    - -4 : Requete non ouverte                                            */
/*    - -5 : Accés au répertoire de l'application impossible                */
/*    - -6 : Accés au répertoire des tables impossible                      */
/*    - -7 : Fichier index de la table inouvrable                           */
/*    - -8 : Fichier donnée de la table inouvrable                          */
/*    - -9 : Allocation impossible                                          */
/*    - -10: Début de fichier atteint                                       */
/*    - -11: Fin de fichier atteint                                         */
/*    - -12 : accés au répertoire de la table du champ concerné impossible */
/*    - -13 : fichier de données du champ impossible d'ouvrir               */
/*    - -14 : Allocation impossible                                         */
/* ------------------------------------------------------------------------ */

extern "C" int Enregistrement_Dernier(char *Nom_Requete);

/* ------------------------------------------------------------------------ */
/* fonction : int Enregistrement_N(char *Nom_Requete,int Rang);             */
/*   Cette fonction positionne les valeurs des champs de la table           */
/* concernée sur l'enregistrement Rang correspondant à la requête.          */
/*   Cette fonction retourne >=0 si le dernier enregistrement est localisé */
/* , cette valeur retournée correspondant au n iéme enregistrement de la    */
/* table, sinon :                                                           */
/*    - -1 : Pas de projet ouvert                                           */
/*    - -2 : Pas de requêtes recensées                                      */
/*    - -3 : Nom de la requête inexistante                                  */
/*    - -4 : Requete non ouverte                                            */
/*    - -5 : Accés au répertoire de l'application impossible                */
/*    - -6 : Accés au répertoire des tables impossible                      */
/*    - -7 : Fichier index de la table inouvrable                           */
/*    - -8 : Fichier donnée de la table inouvrable                          */
/*    - -9 : Allocation impossible                                          */
/*    - -10: Début de fichier atteint                                       */
/*    - -11: Fin de fichier atteint                                         */
/*    - -12 : accés au répertoire de la table du champ concerné impossible */
/*    - -13 : fichier de données du champ impossible d'ouvrir               */
/*    - -14 : Allocation impossible                                         */
/* ------------------------------------------------------------------------ */

extern "C" int Enregistrement_N(char *Nom_Requete,int Courant);
```

```
/* ------------------------------------------------------------------- */
/* fonction : int Ajouter_Condition(char *Nom_Requete,char *Conjonction, */
/*                       char *Membre1,char *Comparaison,char *Membre2); */
/*   Cette fonction ajoute une nouvelle condition à la requête indiquée, */
/* Les valeurs que peuvent prendre les arguments sont :                  */
/*        Conjonction : "et" ou "ou"                                     */
/*        Membre1 ou                                                     */
/*        Membre2 : &Nom_Variable, Nom_Champ ou '<constante>'            */
/*        Signe : "=","!=",">","<",>=","<=","=>" ou "=<"                 */
/*   Cette fonction retourne 0 quand la condition a été acceptée sinon : */
/*    - 1 : Pas de projet ouvert                                         */
/*    - 2 : pas de requête recensée                                      */
/*    - 3 : le nom de la requête inconnu                                 */
/*    - 4 : la requête n'est pas ouverte                                 */
/*    - 5 : Allocation impossible                                        */
/*    - 6 : le nom du champ est inconnu                                  */
/*    - 7 : Signe de comparaison inconnu                                 */
/*    - 8 : Comparaison de deux constantes impossible                    */
/*    - 9 : Comparaison de deux champs impossible                        */
/*    - 10: la variable n'existe pas                                     */
/*    - 11: Type incompatible                                            */
/*    - 12: Taille incompatible                                          */
/*    - 13: Nb_Decimal incompatible                                      */
/*    - 14: le constante ne posséde pas de cotes                         */
/*    - 15: le domaine de définition n'est pas conforme                  */
/*    - 16: le domaine de définition n'est pas respecté                  */
/*    - 17: Taille de la constante trop grande                           */
/*    - 18: Nb de virgules dans un réel trop important                   */
/*    - 19: Date invalide                                                */
/*    - 20: Autres Erreurs                                               */
/*    - 21: Accés au répertoire du projet impossible                     */
/*    - 22: Accés au répertoire des tables impossible                    */
/*    - 23: Accés au répertoire du champ concerné impossible             */
/*    - 24: fichier index du champ impossible d'ouvrir                   */
/*    - 25: fichier donnée du champ impossible d'ouvrir                  */
/*    - 26: Conjonction inconnue                                         */
/* ------------------------------------------------------------------- */

extern "C" int Ajouter_Condition(char *Nom_Requete,char *Conjonction,
                                 char *Membre1,char *Comparaison,char *Membre2);
```

```
/* ------------------------------------------------------------------- */
/* fonction : int Prescannage_Requete(char *Nom_Requete);              */
/*   Cette fonction a pour but de créer un fichier SQL99.IDX contenant */
/* le résultat de la requête. Cette option permettra donc de connaître */
/* à l'avance l'ensemble des enregistrements concernés.                */
/*   Cette fonction retournera >=0 quand le préscannage a fonctionné   */
/* sinon :                                                             */
/*      - -1 : pas de projet ouvert                                    */
/*      - -2 : pas de requêtes recensées                               */
/*      - -3 : La requête n'existe pas                                 */
/*      - -4 : La requête n'est pas ouverte                            */
/*      - -5 : Allocation impossible                                   */
/*      - -6 : Accés au répertoire de l'application impossible         */
/*      - -7 : Accés au répertoire des requête impossible              */
/*      - -8 : Fichier Index de la requête impossible créer            */
/*      - -9 : Fichier Index de la requête impossible ouvrir           */
/*      - -10: Accés au répertoire des tables impossible               */
/*      - -11: Fichier index de la table concernée impossible ouvrir   */
/*      - -12: Fichier donnée de la table concernée impossible ouvrir  */
/*      - -13: accés répertoire de table du champ concerné impossible  */
/*      - -14: fichier de données du champ impossible d'ouvrir         */
/* ------------------------------------------------------------------- */

extern "C" int Prescannage_Requete(char *Nom_Requete);
```